

Computer Systems and Programming

CHAPTER 1: Introduction to Linux

Chapter 1: Introduction to Linux and UNIX Architecture

1.2 What is Linux?

At its core, Linux is a "free" operating system heavily inspired by the foundational concepts of Unix. It was originally created by Linus Torvalds, but its immense growth has been driven by the collaborative support of a multitude of developers around the world. Because Linux is intimately related to the GNU suite—which provides essential tools like the compiler, linker, and debugger—the operating system is most accurately referred to as GNU/Linux.

The system's development is governed by the GNU General Public License (GPL). This guarantees that the source code is entirely free and freely available to anyone who wishes to inspect, modify, or distribute it. While Linux was initially developed specifically for Intel 386 microprocessors, its portability has skyrocketed. Today, it is available on all of the most popular computing architectures, ranging from powerful servers to embedded systems such as routers, firewalls, mobile phones, and video recorders.

1.3 The History and Evolution of UNIX and Linux

To understand Linux, one must understand the expansive history of UNIX, which spans from 1969 to the modern era of computing (up to 2023 in the current taxonomy). This historical evolution tracks the progression of operating systems across three distinct paradigms: Open source, Mixed/shared source, and Closed source systems.

The Bell Labs Foundation

The entire family tree began in 1969 at Bell Labs (owned by AT&T), spearheaded by computing pioneers Ken Thompson and Dennis Ritchie, the latter being the creator of the C programming language. Their initial creation was an unnamed

PDP-7 operating system (1969), which rapidly evolved into Unix Version 1 to 4 (1971 to 1973), Version 5 to 6 (1974 to 1975), and subsequently Unix Version 7 and Unix/32V in 1979. Bell Labs continued their internal development through Unix Version 8 (1985) up to Unix 9 and 10 in 1989, which marked their final versions.

From these early Bell Labs versions, the operating system world split into two massive, distinct lineages: the **BSD (Berkeley Software Distribution) Family** and the **System V Family**, alongside independent educational branches.

The BSD Family Lineage

Spearheaded initially by figures like Bill Joy, the BSD family began branching off from the original Unix codebase.

- **Core BSD Evolution:** It progressed from BSD 1.0 to 2.0 (1978-1979), moving through 3.0 to 4.1 (1980-1981), BSD 4.2 (1983), and BSD 4.3 (1986). Subsequent refinements included BSD 4.3 Tahoe (1988) and BSD 4.3 Reno (1990). This culminated in BSD 4.4-Lite and Lite Release 2 between 1994 and 1995.
- **The Open Source BSDs:** Stemming from BSD Net/1 and Net/2, several highly successful open-source operating systems were born. NetBSD launched versions 0.8 to 1.0 starting in 1993, eventually evolving up to the 9.x series today. OpenBSD branched out around 1996 (versions 1.0 to 2.2), leading up to the 7.2 series by 2023. 386BSD (1992) paved the way for FreeBSD, which released versions 1.0 to 2.2x in the early 90s, and has since progressed through iterations to the modern 13.x series. Furthermore, DragonFly BSD (1.0 to 6.4) emerged in the early 2000s.
- **SunOS and Apple MacOS:** In the early 1980s, the Stanford University Network developed SunOS (1 to 1.1, moving to 1.2 to 3.0, and eventually SunOS 4 in 1989). Concurrently, NextStep emerged (1989), transitioning into OpenSTEP (1.0 to 4.2 by the mid-90s). This lineage directly birthed the modern Apple ecosystem via Darwin (1.2.1 to 22), leading to Mac OS X Server, Mac OS X, OSX, and the modern macOS (10.x to 13.x).

The System V Family Lineage

While BSD focused heavily on academia and open systems, the System V family catered extensively to enterprise and commercial environments. Stemming from

PWB/Unix (1978) and System III (1981), it formalized into System V (R1 to R2 in 1983, R3 in 1987, and R4 in 1989).

- **Xenix and SCO:** Microsoft's initial Unix foray, Xenix (1.0 to 2.3), began in 1980, reaching version 3.0 before evolving into SCO Xenix and its variants (V/286 and V/386) throughout the late 80s. This evolved into SCOUNIX 3.2.4 (1993), OpenServer (5.0 to 5.0.7 in the late 90s), and eventually OpenServer 6.x (2005-2007). UnixWare also emerged in this branch (1.x to 2.x based on System V R4.2, and 7.x based on System V R5).
- **Commercial Giants:** Several major tech companies built their proprietary closed-source systems on System V roots. IBM developed AIX (version 1.0 in 1986, moving from 3.0 to 7.3 by 2023). Hewlett-Packard created HP-UX (1.0 to 1.2 in 1984, advancing through versions 2.0 to 11.10, and 11i v1 to v3 today). Sun Microsystems developed Solaris (2.1 to 9 starting in 1992, Solaris 10 in 2005, and 11.0 to 11.4 more recently), which also spawned OpenSolaris and derivatives like Illumos. Silicon Graphics developed IRIX (SGI) starting in 1988.

The Birth of Linux and Minix

Separate from the direct corporate lines, Andrew Tanenbaum created Minix for educational purposes, evolving from versions 1.x (1987) to 2.x (1997) and 3.1.0 to 3.4.0 in the modern era. Richard Stallman concurrently launched the GNU Project in the mid-1980s, producing vital tools and the GNU/Hurd 0.2 kernel. Inspired by Minix and utilizing GNU tools, Linus Torvalds released Linux 0.0.1 in 1991. This open-source kernel rapidly evolved to version 0.9 by 1993, and has since grown immensely through the 2.0 to 6.x versions dominating the present day.

1.4 Unix and Linux Features and Design Principles

Linux inherited and refined a multitude of robust features from its Unix predecessors.

- **Multiuser and Multitasking:** The system allows multiple users to operate simultaneously and run independent tasks concurrently without conflict.
- **Tool Composition:** It features a rich ecosystem of tools at the user level that can be composed and chained together (acting as filters) to perform complex tasks simply.

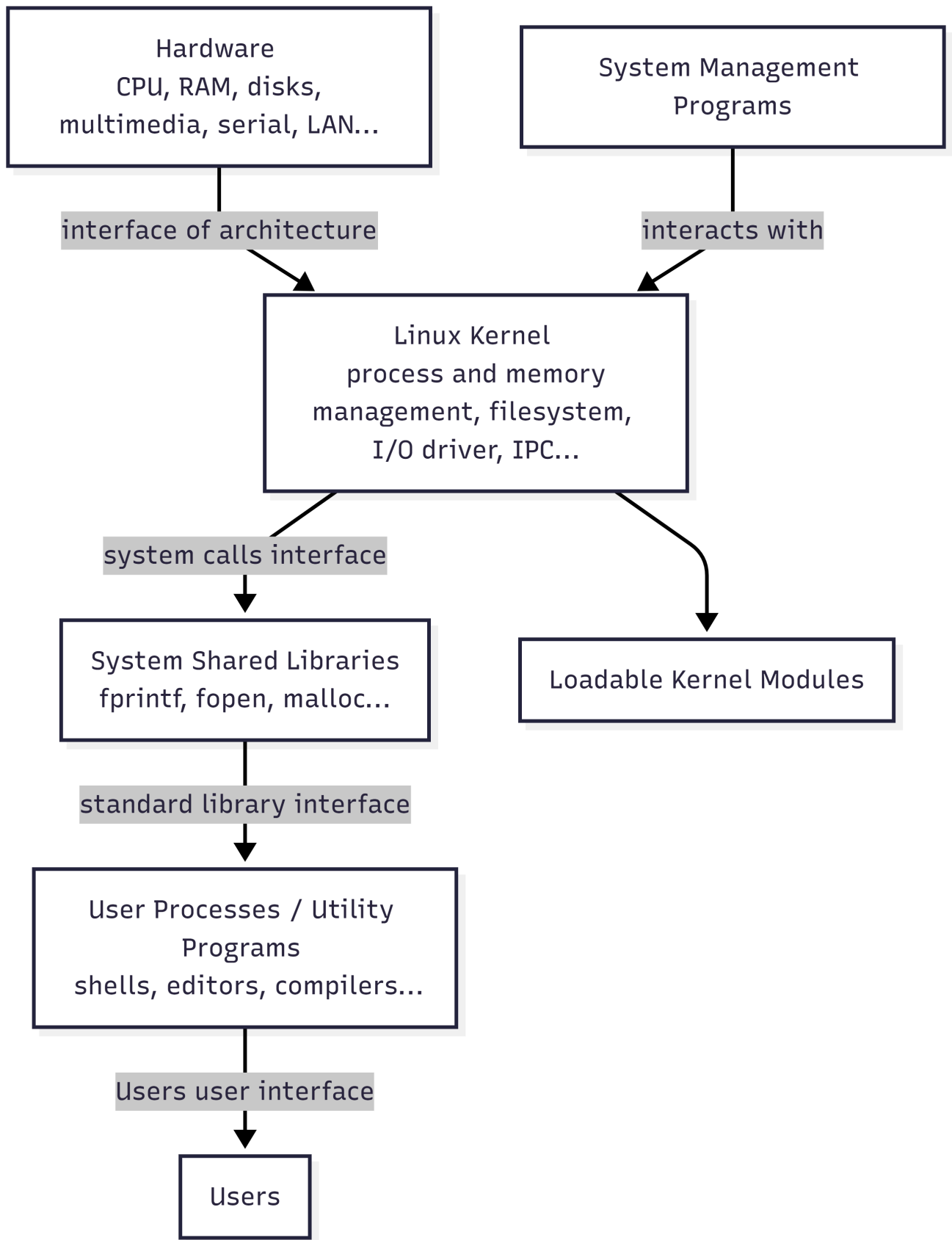
- **Programmer-Centric:** It is an environment fundamentally designed by programmers, for programmers, prioritizing control and flexibility.
- **"Everything is a File":** A core Unix philosophy adopted by Linux is treating almost all hardware devices and data streams as files, though this rule has minor exceptions ("+/-") in modern contexts.
- **Kernel Architecture:** It utilizes a monolithic kernel, meaning the core operating system runs in a single large memory space, but it is highly extensible via loadable modules.
- **Language:** Like Unix, Linux is written almost entirely in the C programming language.

Core Design Principles

The primary design principles driving Linux development are speed, efficiency, and standardization. The operating system maintains tight relationships with other UNIX variants by being "compliant" with POSIX (Portable Operating System Interface) specifications. Its Application Programming Interface (API) is heavily based on UNIX SVR4 (System V Release 4) standards, while many of its tools and libraries are derived directly from the BSD lineage. Above all, it is deeply integrated with the GNU system.

1.5 Linux Architectural Layers

The architecture of Linux is structured into distinct, communicating layers, separating hardware management from user interaction.



Schema Translation: At the absolute foundation lies the **Hardware**, encompassing the CPU, RAM, disks, multimedia devices, serial ports, and LAN. Interfacing directly with the architecture of this hardware is the **Linux Kernel**. The Kernel itself is a monolithic block that handles process and memory management, the filesystem, I/O drivers, and Inter-Process Communication (IPC). It can be dynamically expanded using **Loadable Kernel Modules**.

Above the Kernel sits the **System calls interface**, which connects the core OS to **Standard (Shared) Libraries** (containing fundamental functions like `fprintf`, `fopen`, `malloc`) and **System Management Programs**. These libraries provide a standard interface to the next layer: the **User Processes and Utility Programs**, which include shells, text editors, and compilers. Finally, at the very top, is the **Users user interface**, where humans interact directly with the utility programs.

Chapter 2: The Kernel and System Execution

2.1 The Role and Nature of the Kernel

At the heart of the Linux operating system is the **Kernel**, a specialized software program that executes directly on the system's hardware. The primary responsibility of the kernel is to implement the system's process model and provide essential underlying services. Physically, the kernel resides as a file on the storage disk, typically found in boot paths such as `/boot/zImage` or `/boot/vmlinuz`. It is loaded into RAM during the system boot sequence and remains persistently in memory until the machine is formally shut down.

While Linux utilizes a monolithic kernel design, it maintains immense flexibility by supporting dynamic Loadable Kernel Modules (LKM). These modules are distinct sections of kernel code that can be independently compiled, loaded, and unloaded without requiring a reboot or affecting the rest of the active kernel. Typically, a kernel module is used to implement a specific device driver, a new file system, or a networking protocol.

The modular interface is structurally and legally advantageous. It allows third-party developers to write and distribute device drivers under their own licensing terms, providing a mechanism for proprietary code that cannot be distributed under the stringent GNU General Public License (GPL) to function alongside Linux. Furthermore, modules enable advanced system management capabilities, such as applying dynamic patches to a live operating system.

2.2 Execution Modes and the Process Context

There is only one instance of the kernel operating on the system at any given time. This single instance manages global system resources, such as the system stack, memory address spaces, and per-process objects. To track and manage active applications, the kernel utilizes global data structures to store detailed information about every process, paying special attention to the current running process.

The system relies on a rigid security boundary governed by two primary execution modes:

- **Kernel Mode:** A highly privileged state reserved strictly for core kernel functions.
- **User Mode:** A non-privileged state where standard user applications and functions execute safely without direct access to hardware.

Whenever a user application requires a core service (like reading a file or opening a network port), it must invoke a "system call," which triggers an immediate and controlled mode switch from User Mode to Kernel Mode.

The Execution Mode and Context Matrix

The operating environment can be visualized as a matrix intersecting Execution Mode (User vs. Kernel) with Execution Context (Process vs. System).

	Process Context (Specific to a task)	System Context (Global operations)
User Mode	Application (User) Code: Software operates here, with access restricted to process space only.	Not Allowed: This intersection is architecturally forbidden.
Kernel Mode	System Calls & Exceptions: Code here can access both process and global system space.	Interrupts & System Tasks: Code here accesses system space only, managing core hardware responses.

2.3 The x86 Booting Process

The initialization of a Linux system on standard x86 computing architecture follows a strict, sequential process:

1. **BIOS Initialization:** The Basic Input/Output System (BIOS) initiates and loads the boot block from the system's active partition.
2. **Bootloader Execution:** The initial boot code loads a second-level bootloader, traditionally GRUB (GRand Unified Bootloader).
3. **Kernel Loading:** GRUB possesses the capability to read file systems directly. It locates and starts the kernel, often loading a temporary RAM disk into memory alongside it to facilitate initial setups.

4. **Device Initialization:** The kernel assumes control, initializes the connected hardware devices, switches the root file system from the RAM disk to the actual hard drive, and executes the system's first fundamental process, which is traditionally `init` (or its modern equivalent, `systemd`).
5. **Service Spawning:** The `init` process is assigned PID 1 (Process ID 1). It takes over the boot sequence by starting all necessary system services. Eventually, it spawns the `getty` process (which manages physical and virtual terminal lines), which in turn launches the `login` process.
6. **User Access:** Finally, `login` handles user authentication and, upon success, starts the user's shell environment.

2.4 User Login and Logout Mechanisms

The login procedure serves three distinct purposes: to securely authenticate the user, to configure a customized environment for that specific user, and to start the command shell. If the provided username and password are correct, the user is presented with the shell prompt—a visual message indicating that the system is ready to accept commands.

Example of a standard login prompt and success sequence:

```
Fedora Core release 3 (Heidelberg) Kernel 2.6.9-1.667 on an i386
antarctic login: penguin
Password:
Last login: Thu Aug 18 17:13:26 on :0
[penguin@antarctic ~]$ echo $SHELL
/bin/bash
```

Conversely, the logout procedure systematically ends a user's session. It ensures that any system resources allocated to the user are released, kills all processes belonging to that user by sending termination signals, and officially ends the work session. In a command-line environment, users can initiate this sequence by typing `logout`, typing `exit`, or by using the keyboard shortcut `<Ctrl-d>`.

Chapter 3: System Interfaces and Command Basics

Linux provides two fundamentally different paradigms for users to interact with the underlying operating system: the command interface (managed by a shell program) and the graphical interface (managed by an X Windows server).

3.1 The Command Interface (CLI)

In the command interface, the user interacts with the system by directly typing and sending textual commands. Each command is formulated as a sequence of alphanumeric characters that is officially submitted for execution by pressing the `<ENTER>` (or `<Return>`) key.

These inputs must adhere to a well-defined syntax. A program called the command interpreter—most commonly the `bash` shell in Linux—is responsible for parsing this syntax. The shell verifies the structural correctness of the string, executes the required actions by communicating with the OS, and finally generates output results or error messages directly onto the video terminal. For example, if a user attempts to run a non-existent command like `by -sh`, the shell will return a `command not found error`, or if they type `du -sh (/home/verola`, the shell will return a `syntax error near unexpected token` alert.

3.2 The Graphical Interface (GUI)

Developed to address the need for greater ease of use and a shallower learning curve, the graphical interface relies on icons, graphic panels, and predefined menus for data input and system navigation. In addition to a keyboard, it requires a graphic display terminal and a mouse.

In the Linux ecosystem, the GUI is fundamentally based on the **X Window System** (frequently referred to as X11 or simply X). The X Window System is a client/server architecture explicitly dedicated to providing the graphical primitives required to construct user interfaces.

- **The X Server:** The server component manages the physical hardware (mouse and keyboard) and is responsible for drawing pixels within the graphic windows reserved for specific X Clients. It displays the output data that those clients request to be drawn.
- **The X Client:** The client component waits for input events (like clicks or keystrokes) relayed from the X Server, processes this data, and sends the resulting graphical updates back to the server to be rendered.

To govern how these windows actually appear and behave, Linux uses **X window managers**. These are specialized software programs that control the positioning, core functionality, and overall graphical style of the windows. A key strength of Linux is that users can choose between various X window managers, differing wildly in their level of customization and how tightly they integrate with the broader desktop environment.

3.3 Essential Linux Commands

The basic commands and main utilities found in modern Linux are direct derivatives of Unix. The core toolkit every user must be familiar with includes:

- **Filesystem Creation and Movement:** `ls` (list contents), `cd` (change directory), `pwd` (print working directory), `mkdir` (make directory), `rmdir` (remove directory), `tree`, `touch`, `cp` (copy), `rm` (remove), `find`.
- **Viewing and Editing Text Files:** `more`, `ed`, `vi`, `emacs`, `ex`.
- **Text and Data Processing:** `echo`, `cat`, `grep`, `sort`, `uniq`, `sed`, `awk`, `tail`, `tee`, `head`, `cut`, `tr`, `split`, `printf`.
- **File Comparison Tools:** `comm`, `cmp`, `diff`, `patch`.
- **Miscellaneous Utilities:** `test`, `xargs`, `find`.
- **System Administration:** `chmod` (permissions), `chown` (ownership), `ps` (process status), `on`, `who`.
- **Network Communication:** `mail`, `telnet`, `ftp`, `finger`, `ssh`.
- **Command Shells Available:** `sh`, `bash`, `csch`, `ksh`, `tcsh`.

Academic Note: To begin mastering the system, users should open a terminal and explore basic window manager functions (minimize, maximize, resize, move, close) and analyze the mechanics of the text interface by observing the command prompt, executing simple commands, and reading the resulting output. Executing `man man` is highly recommended to understand the built-in manual system.

Chapter 4: The Linux Filesystem Architecture

4.1 Filesystem Abstractions and Core Functions

The filesystem is an essential operating system component explicitly dedicated to storing information on mass memory devices (such as hard drives). Within the Linux ecosystem, there are dozens of different types of filesystems available, including `ntfs`, `fat`, `ext2`, `ext3`, and `jfs`. Linux is highly versatile in this

regard, allowing several different file systems to be used—or "mounted"—simultaneously on a single machine.

Fundamentally, the filesystem serves as an abstraction layer. Its main functions are designed to shield the user and applications from the complexities of hardware management:

- **Simplifying Access to Mass Storage:** To read and write data, users and programs do not need to specify physical hardware addresses (using head, sector, and cylinder parameters); instead, they simply refer to intuitive file and directory names.
- **Structured Organization:** It offers the user a structured and efficient way to logically organize their personal and system data.
- **Dynamic Space Management:** The filesystem manages space on physical devices, autonomously assigning and releasing memory areas based on the active requests of the programs utilizing it.

Beyond these basics, the Linux filesystem interface provides several advanced functions. It implements critical data protection schemes, such as managing access rights to files and directories, and enforcing policies to limit the use of disk space (known as quotas). The filesystem also acts as the interface for direct access to peripheral hardware devices (typically mapped in the `/dev` directory) and for accessing real-time system and kernel information (mapped in the `/proc` directory). Furthermore, it seamlessly allows access to remote data over a network (using protocols like Samba or NFS) and facilitates reading data from other operating systems (such as Microsoft's NTFS or FAT).

4.2 The "Everything is a File" Philosophy

One of the most famous adages in UNIX/Linux design is: *"On a UNIX/Linux system, everything is a file"* (well, almost). This design philosophy held strictly true until modern computing required adding more complex "features" that did not fit the traditional file paradigm. Exceptions to this rule today include the socket API for networking, certain POSIX standard additions, and the X11 graphical system. Because of these additions, Linux now requires upwards of 300 different system calls and counting to handle operations that fall outside basic file I/O.

While there have been more "orthodox" operating systems that attempted to strictly maintain this philosophy (such as Plan 9 from Bell Labs), none of them

achieved widespread commercial success. They ultimately failed due to a lack of compatibility and a corresponding lack of software applications. Today, the abstraction is managed by higher-level programming; almost nobody programs operating systems using raw, direct system calls anymore.

4.3 File Types and Logical Organization

In Linux, the namespace is not limited to "regular" files (like text documents, binary executables, source programs, or images). There are also "special" entries within the namespace that represent entirely different systemic objects, including active processes, I/O hardware devices, communication channels (such as pipes and sockets), and specific kernel memory areas.

A regular file, at the operating system level, is simply a sequence of bytes. The Linux OS does not interpret the internal structure or the contents of a file; there is no innate "record structure". The interpretation of the data is entirely delegated to the user applications reading it.

A directory is simply a special type of file that contains a list of names of other files and/or other directories (referred to as subdirectories). The Linux file system is organized logically as a hierarchical tree:

- The **root directory** (indicated by the forward slash `/`) constitutes the very base or root of the tree.
- **Directories** act as the structural nodes of the tree.
- **Files** (and empty directories) constitute the terminal leaves of the tree.

Standard File Identifiers

When executing commands to inspect the filesystem, Linux utilizes a specific one-character flag to denote the exact type of a file. Using the `-l` (long format) option of the `ls` command displays this file type as the very first character of the output line:

- `-` : Denotes a **regular file** (data in text/binary format, executables, source code, images).
- `d` : Denotes a **directory** (a file containing a list of other files).
- `l` (lowercase L) : Denotes a **symbolic link** (a file that merely acts as a pointer to another file).
- `c` : Denotes a **character device** (an I/O device organized into single, unbuffered bytes).

- **b** : Denotes a **block device** (an I/O device organized into fixed-length, buffered blocks).
- **p** : Denotes a **named pipe** (a mechanism used for Inter-Process Communication, or IPC, within the kernel).
- **s** : Denotes a **socket** (a dedicated communication channel used for network transmissions).

Example of `ls -l` output analysis:

Plaintext

```
drwxr-xr-x 2 penguin birds 4096 Aug 13 11:07pm Desktop
-rwxrwxr-x 1 penguin birds 6240 Aug 31 09:35 main
-rw-rw-r-- 1 penguin birds 337 Aug 31 09:35 main.c
lrwxrwxrwx 1 penguin birds 6 Aug 31 00:04 prog.c -> main.c
prw-rw-r-- 1 penguin birds 0 Aug 31 17:40 myfifo
```

In this output block, `Desktop` is a directory (`d`), `main` and `main.c` are regular files (`-`), `prog.c` is a symbolic link (`l`) pointing to `main.c`, and `myfifo` is a named pipe (`p`).

4.4 Filename Rules and Conventions

In principle, a Linux file name can be of any length (though the actual maximum length is strictly dependent on the underlying filesystem being used). A filename can utilize any printable character, with one absolute exception: the forward slash `/`, which is reserved as the directory separator.

- **Uniqueness and Sensitivity:** File names must be completely unique within the same directory. They are also strictly case-sensitive (e.g., `File.txt` is distinct from `file.txt`).
- **Hidden Files:** Any file or directory name that begins with a period `.` is considered a hidden file/directory. It will not be shown by the standard `ls` command unless the user explicitly invokes the `-a` option.
- **Hyphen Conflicts:** A file name should *never* begin with a hyphen `-`. Many terminal commands treat any string starting with a hyphen as a command-line option or flag, which will cause errors. If a user must delete a file that mistakenly starts with a hyphen (e.g., `-file`), specialized command structures or absolute paths must be used to bypass the shell's option parsing.

- **Extensions:** Unlike some operating systems, a file or directory name in Linux does not possess a system-defined structure. File extensions (such as `.c` , `.h` , or `.txt`) have only a conventional, human-readable value; they hold no intrinsic systemic value or execution meaning to the Linux kernel.

Handling Special Characters in Filenames

Extreme caution must be used if a file name contains special shell characters such as `&` , `*` , `\` , `$` , `?` , or spaces (`<space>`). If these exist in a filename, they must be "escaped" by preceding them with a backslash `\` , or the entire filename must be enclosed in single quotes `' '` . This prevents the command interpreter (the shell) from improperly interpreting the characters as systemic commands. For instance, attempting to read a file named `filename with spaces` by simply typing `file filename with spaces` will result in errors, as the shell interprets "filename", "with", and "spaces" as three separate, non-existent files. It must be executed as `file 'filename with spaces'` or `file filename\ with\ spaces` . The same applies to wildcard/variable symbols, such as `fileWith$Dollar` , which requires execution as `file fileWith\$Dollar` to be properly recognized.

4.5 Pathnames and The Standard Directory Hierarchy

To locate and identify a specific file or directory, a "pathname" is constructed using directory names sequentially separated by the `/` character. The final name in the string represents the target file or directory.

Pathnames are categorized into two types:

1. **Absolute Pathname:** Always begins with a `/` , explicitly defining the route starting from the absolute root of the filesystem (e.g., `/usr/src/linux`).
2. **Relative Pathname:** Begins with a directory or file name, defining the route starting directly from the active (or "working") directory of the running process.

In relative path navigation, the symbol `.` is used to explicitly indicate the current active directory, while the symbol `..` designates the parent directory (the directory immediately containing the active directory). For example, if a user's working directory is `/usr/etc` , the relative path `../src/linux` will navigate up to `/usr` and then down into `/src/linux` .

Standard Linux Directories Under /

Linux distributions generally adhere to a standardized filesystem hierarchy originating from the root / :

- `/bin` : Contains the basic, essential commands and utilities required for all users.
- `/boot` : Houses the Kernel image files and bootloader configurations.
- `/dev` : Contains device files used to access system peripherals and hardware storage systems.
- `/etc` : Stores critical system configuration files.
- `/home` : The primary directory containing the "Home directories" of all individual users.
- `/lib` : Holds shared system libraries crucial for the operation of compiled programs.
- `/proc` : A highly specialized *virtual* file system. It has no real disk space allocated to it; rather, it is used by the system to provide dynamic system information, particularly data directly related to the running kernel and active processes.
- `/sbin` : Contains sensitive commands intended exclusively for system administration and management.
- `/sys` : Contains files relating to systemic hardware (HW) information and devices.
- `/tmp` : The designated directory for temporary files created by applications and users.
- `/usr` : A massive directory containing the majority of user-level programs, applications, and source code existing on the system.
- `/var` : Designated for variable data that changes rapidly, such as system logs and print queues.

4.6 Core Filesystem Management Commands

Navigating and managing this hierarchy relies on a core set of terminal commands:

- **Directory Navigation & Viewing:**
 - `pwd` : Print Working Directory (shows the current absolute path).
 - `cd directory` : Change working Directory (navigates to the specified location).

- `ls` **directory** : LiSt directory contents. Critical options include `-a` to view all (including hidden) files, `-l` for a long detailed listing, and `-g` for group information. Practical applications also include using the `-u` and `-c` options to sort and view files based on access and modification chronologies, or using specific flags to print the physical inode numbers of files.
- `tree` **directory** : Displays the directory structure visually as a branching tree.
- **Creation & Deletion:**
 - `mkdir` **directory** : MaKe (create) a DIRectory. Advanced usage includes options allowing the creation of a child directory alongside any non-existent parent directories simultaneously in a single command.
 - `rmdir` **directory** : ReMove an empty DIRectory.
 - `rm` : ReMove (delete) files. When deleting files, options can be applied to strictly require user confirmation before the deletion is executed.
- **Manipulation & Observation:**
 - `cp` : Copy files or, using specific options, recursively copy entire directory trees.
 - `mv` : MoVe a file (which is also the systemic method used to rename a file).
 - `ln` : Create a LiNk to a file (either symbolic or hard).
 - `less` (or `more`, `pg`) : Page through a large text file one screen at a time.
 - `test` : A utility program (often used in scripts) to evaluate file types and compare values, which can be run on directories.
 - `df` [options] [directory] : Displays the available free space on mounted disks. (e.g., `df -Tm` to show filesystems, types, and sizes in megabytes).
 - `du` [options] [directory] : Estimates and displays the disk space usage of a specific directory or file. (e.g., `du -s` for a summary, or `du -h` for human-readable formats). The `stat` command can also be utilized to extract and analyze highly detailed chronological and structural information provided directly by the file's inode.

Chapter 5: Physical Storage and the System V Filesystem

5.1 Logical Organization vs. Physical Reality

While files and directories are presented to the user logically as an ordered, hierarchical tree structure, the computer hardware fundamentally does not understand the meaning of this abstract representation.

In physical reality, permanent data is stored sequentially on disk blocks of a fixed size. A physical hard disk is usually divided into distinct logical partitions. Before a partition can hold data, a filesystem must be mathematically created and mapped onto it using commands like `newfs` (in BSD UNIX) or `mkfs` (in System V and Linux). Each partition thus contains its own individual, structured filesystem.

To unify the system, these independent filesystems are attached—or "mounted"—onto specific directories known as "mountpoints". When a filesystem is mounted, the original mountpoint directory is covered by the root of the newly mounted file system. The Linux Kernel diligently maintains complex data structures, such as the mount table and virtual file system (vfs) list, to track these attachments. The overall tree structure with which the user interacts is actually the unified sum of all these individual mounted filesystems.

5.2 The Inode (Index Node) Concept

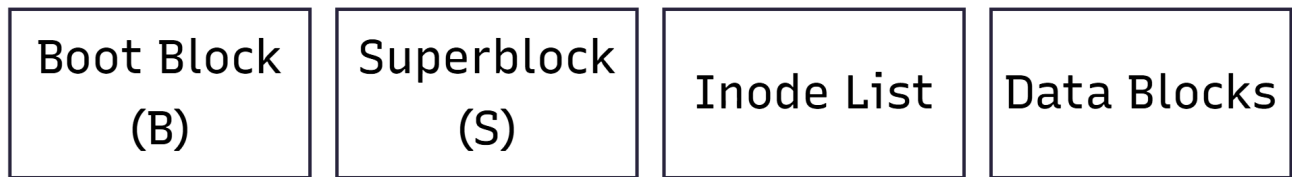
Just as the computer is blind to the tree structure, it is also blind to filenames. File names and paths are strictly for human convenience; the UNIX/Linux operating system identifies files exclusively by their "inodes".

An inode number is a unique integer identifier directly associated with a core data structure (the inode itself). This inode structure contains all the metadata regarding the properties of a file, as well as the map to the exact physical disk blocks where the file's data is actually stored. Because each partition acts as an independent filesystem, each partition possesses its own dedicated set of inodes. The physical space for these inodes is pre-allocated and generated during the initial filesystem formatting and installation process.

5.3 The System V File System (s5fs) Architecture

To understand how physical blocks translate into a filesystem, we examine the foundational System V File System (s5fs) layout. An s5fs partition is divided

sequentially into four distinct structural zones:



1. **Boot Block (B):** The very beginning of the partition, reserved for initial booting instructions.
2. **Superblock (S):** A highly critical "special" block. Because of its importance, it is stored in multiple backup locations across the disk for redundancy. The superblock maintains the global state of the filesystem, containing:
 - The total size (in blocks) of the entire filesystem.
 - The total size of the allocated inode list.
 - The current number of free (available) blocks and free inodes.
 - A Free block list and a Free inode list (Note: while s5fs used explicit lists, all modern filesystems optimize this by using bit-level bitmaps to track free space).
3. **Inode List:** The pre-allocated sector of the disk containing the sequential array of all available inode structures.
4. **Data Blocks:** The vast majority of the partition, consisting of the actual physical blocks where file data is written.

Internal Inode Structure and Data Pointers

When examining a specific file's associated inode, the data structure (represented in C code as `struct dinode` in s5fs) contains a wealth of critical information:

- The overall **file length** in bytes.
- The **ID of the device** containing the file.
- The **user ID** of the file owner and the **group ID** of the file.
- The unique **inode number** itself.
- The **file mode**, which mathematically determines the file type as well as the access and execution permissions.
- Chronological timestamps: The date of last modification of the file data (`mtime`), the last access of the file (`atime`), and the last modification of the inode metadata itself (`ctime`).
- The **link number**, which tracks the exact number of hard-links actively pointing to this specific inode.

To connect this metadata to the actual file content, the inode structure contains an array of direct and indirect memory pointers. These pointers map directly from the inode entry in the "Inode List" to the scattered, non-contiguous physical blocks located in the "Data Blocks" section of the partition, weaving the fragmented physical data into a single logical file stream for the user.

Chapter 6: File Attributes, Permissions, and Ownership

6.1 Inspecting File Attributes

To fully understand how Linux manages data, one must be able to read a file's attributes directly from its inode. This is achieved using the command `ls -lis`, which requests a long-format list that includes the index number and size.

When executing a command such as `ls -lis lec1.pdf`, the output might appear as `10945 188 -rwxr----- 1 danilo danilo 189602 Sep 12 19:11 lec1.pdf`. This string of data is broken down into eight critical fields:

1. **Index:** The file's unique inode number (e.g., 10945).
2. **Blocks:** The number of physical data blocks utilized by the file, where one block typically equals 1024 bytes (e.g., 188).
3. **Permissions:** The file type and its associated access mask (e.g., `-rwxr-----`).
4. **Hard-link Count:** The number of directory entries pointing to this specific inode (e.g., 1).
5. **Ownership:** The username of the file's owner followed by the groupname associated with the file (e.g., `danilo danilo`).
6. **Size:** The exact size of the file in bytes (e.g., 189602).
7. **Timestamp:** The date and time of the last modification (e.g., Sep 12 19:11).
8. **Name:** The human-readable file name (e.g., `lec1.pdf`).

6.2 The Concept of Owners and Groups

Linux security is fundamentally tied to the concept of user and group ownership. Every file in the system is definitively associated with a specific user who owns it, and a specific group (a defined set of users) that shares special access rights to it.

Internally, the operating system identifies users and groups using integer values known as the User ID (UID) and Group ID (GID), while presenting them to

humans as string-based usernames and groupnames. When a user creates a new file, the operating system automatically associates that file with their UID and their current active group. A user's initial default group is predetermined by the system administrator during account creation.

To view the groups to which a user belongs, the `groups [username]` command is used. If invoked without arguments, it lists the groups of the currently logged-in user; if a username is provided, it returns the groups associated with that specific account. Users can temporarily change their active current group (effectively opening a new shell session under that group) using the `newgrp <groupname>` command.

File ownership can also be transferred. The group associated with a file can be modified by the owner using `chgrp <groupname> <file(s)>`. The primary user ownership of a file can be changed using `chown newUserId file(s)`, though this command is normally restricted to system administrators in environments that manage disk space quotas, preventing users from offloading their storage limits onto others.

6.3 The Permission System

Every file is guarded by nine specific flags, collectively known as "Permission bits". These nine bits are divided into three triplets, determining the rights of the **User** (the owner), the **Group**, and **Others** (everyone else).

Each triplet specifies three distinct rights: Read (R), Write (W), and Execute (X). Crucially, the behavior of these rights differs depending on whether they are applied to a regular file or a directory:

- **Read (R):** For a regular file, this grants the ability to read the file's contents. For a directory, it grants the ability to read the list of files contained within it.
- **Write (W):** For a regular file, this allows the user to edit its contents. For a directory, it allows the user to add, remove, or rename files within that directory structure.
- **Execute (X):** For a regular file, this allows the system to run the file as a program or script (assuming it makes sense to do so). For a directory, the execute bit is required to `cd` (change directory) into it or to access files within it via a defined path.

Effective vs. Real IDs in Permission Logic

Permission management is nuanced by the existence of Real and Effective IDs. When a process runs, it holds both a Real User/Group ID (ruid, rgid) and an Effective User/Group ID (euid, egid). The Effective ID allows a process to execute "on behalf" of another user. When a process is initially created, its real and effective IDs are identical.

When a process attempts to access a file, the system evaluates permissions using a strict, cascading logic:

1. If the effective user ID of the process matches the file's owner ID, the **User permissions** are applied.
2. Otherwise, if the effective group ID matches the file's owner GID, the **Group permissions** are applied.
3. If neither matches, the **Others permissions** are enforced.

6.4 Modifying Permissions with chmod

Permissions are modified using the `chmod` command, which supports two distinct syntax paradigms: Absolute (Octal) and Relative (Symbolic).

Absolute (Octal) Method: This method uses an octal number to define the exact state of all nine permission bits simultaneously using the format `chmod octal-number file(s)`. The values are calculated by assigning 4 to Read (R), 2 to Write (W), and 1 to Execute (X).

- `chmod 755 public_html`: Assigns read, write, and execute rights ($4+2+1=7$) to the user, while granting only read and execute rights ($4+1=5$) to both the group and others.
- `chmod 644 .procmailrc`: Assigns read and write rights ($4+2=6$) to the user, and only read rights (4) to the group and others.

Relative (Symbolic) Method: This method uses letter codes to add, remove, or set specific permissions without altering the others, using the syntax `chmod [ugo][+ -=][rwxXstugo] file(s)`.

- `chmod u+x script.sh`: Adds (+) the execute (x) flag specifically for the user/owner (u) of the script.
- `chmod -R ug+rwX src/*`: Recursively (-R) adds read and write rights (rw) for the user and group (ug) to all files in the `src/` directory. The capital X intelligently adds execute rights only to directories, not standard files.

- `chmod -R o-rwx $HOME` : Recursively removes (-) all read, write, and execute rights (`rwX`) from others (`o`), locking down the home directory.

Chapter 7: The Link System

Linux allows multiple filenames to point to the same data, utilizing two different types of links: hard links and symbolic (soft) links.

7.1 Hard Links

A hard link is created using the syntax `ln file hlink`. This creates a new directory entry (named `hlink`) in the current directory that points to the exact same inode number as the original file. Because a new entry is pointing to the inode, the link number stored within the inode is incremented by 1. Regular users are strictly prohibited from creating hard links to directories to prevent circular loops in the filesystem structure.

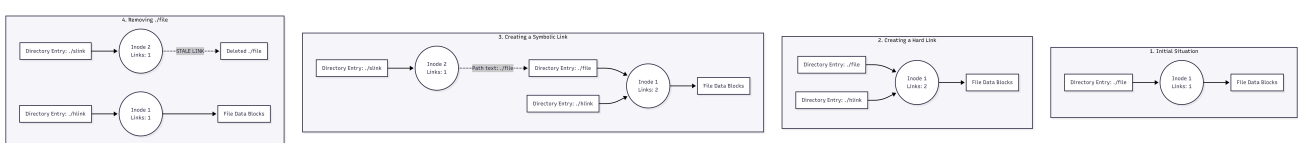
If the original file name is deleted, the data is not lost; the inode's link number is simply decremented. The physical data blocks on the disk are only truly deleted when the link number reaches 0.

7.2 Symbolic Links

A symbolic link is created using the syntax `ln -s file slink`. This creates a completely new, special file (named `slink`) with its own unique inode. Instead of pointing directly to the data blocks, the contents of this new file contain the pathname to the original file. Consequently, creating a symbolic link does not increase the link number of the original file's inode. While the shell has built-in mechanisms to break them, symbolic links pose a risk of creating infinite loops in the filesystem if linked improperly.

If the original target file is removed, the symbolic link becomes "stale" or broken, as it is now pointing to a pathname that no longer exists.

7.3 Link Architecture Visualized



Schema Translation: The diagram illustrates the lifecycle of links. Initially, a single directory entry points to Inode 1. Creating a hard link adds a second directory entry pointing to Inode 1, increasing its link count to 2. Creating a symbolic link

generates a new Inode (Inode 2) whose data points back to the string path of `./file`. If `./file` is deleted, the hard link `./hlink` survives and points to Inode 1 (whose count drops to 1), keeping the data intact. However, `./slink` becomes a stale link pointing to nowhere.

Chapter 8: Process Management

8.1 Process Attributes

Just as files have attributes, every running program—known as a process—has a detailed set of attributes monitored by the kernel.

- **PID (Process Identifier):** The unique integer ID assigned to the active process.
- **PPID (Parent PID):** The ID of the parent process that spawned or created the current process.
- **Nice Number:** The static priority of the process, determining its share of CPU time.
- **TTY:** The specific terminal device associated with the process.
- **Identity Markers:** The Real UID, Effective UID, Real GID, and Effective GID, which dictate the process's access permissions.
- **Resource Metrics:** Additional data tracking the amount of memory and CPU currently being consumed.

8.2 Process Monitoring and Priority

The primary tool for monitoring active processes is the `ps` command, which reports current system statuses. Using a flag combination like `ps alx` provides a highly detailed output including the Process Status (STAT), Virtual Memory Size (VSZ), and CPU prioritization metrics for deep system diagnosis. Alternatively, the `top` utility can be used for periodic, real-time monitoring of system processes.

Process priorities can be manipulated using the "nice" value, which ranges from -20 (the highest possible maximum priority) to 19 (the absolute minimum priority), with 0 being the standard default.

- **nice:** Used to execute a brand-new command with a specific static priority different from the default (e.g., `nice -n 19` command launches the task with minimum priority).

- **renice:** Used to dynamically change the priority of a process that is already running (e.g., `renice +10 -p 1234`).

Processes are terminated or communicated with using the `kill [-SIGNUM] processId` command. This command sends specific POSIX signals to a process. While many signals exist, some (like the aggressive `-9` or SIGKILL flag, e.g., `kill -9 3426`) immediately force the process to terminate. The `kill` utility can also be used simply to verify the existence of a process if the PID is known, without actually terminating it, by sending a benign signal.

8.3 Job Control and The Shell

The shell acts as the ultimate controller of user processes, dividing them into Foreground and Background execution states.

- **Foreground Processes:** These processes seize control of the terminal from which they were launched. At any given time, only one process can occupy the foreground per terminal window.
- **Background Processes:** These are executed silently without an "attached" terminal, allowing the user to continue utilizing the command line while the process runs behind the scenes.

The shell provides robust "Job Control" commands to fluidly move processes between these two states:

- `&` : Appending this to the end of a command (e.g., `long_cmd &`) launches the process directly into the background.
- `Ctrl+z` : A keyboard interrupt that instantly suspends (stops) the current foreground process. (Other interrupts include `Ctrl+c` to terminate and `Ctrl+\` to quit).
- `jobs` : Lists all currently running or stopped background processes associated with the shell.
- `%n` : A syntax used to directly reference the n-th background process in the job list (e.g., `kill %1` terminates the first background job).
- `fg %n` : Switches a specific process from the background to the foreground.
- `bg %n` : Restarts a previously stopped process, forcing it to resume execution in the background.

Chapter 9: Practical Academic Scenarios

Based on the systemic exercises provided in the curriculum.

Filesystem Traversal and Analysis: To apply directory commands, a user might change their working directory, verify the move with `pwd`, and return to their home directory. When inspecting files in directories like `/etc`, one can use `ls -a` to view hidden files, `ls -R` for recursive output showing all subdirectories, `ls -r` to list files in reverse alphabetical order, and `ls -li` to print the underlying inode numbers of the files. Exploring the output of the `stat` command provides deeper insight into a file's inode metadata.

Permissions and Links in Practice: When analyzing permissions, it is vital to know that the octal representation of a `-rwxr-----` mask is `740`, while an octal `755` translates symbolically to `-rwxr-xr-x`.

- If a directory is made non-executable but readable, a user attempting to `cd` into it will be denied access, but running `ls` on it will still display the contained filenames.
- Conversely, if a directory is unreadable but executable, the user can successfully `cd` into it (if they know the exact paths inside), but `ls` will fail to list the contents.
- When practicing links, creating a circular dependency (where `symlink1` points to `symlink2`, which points back to `symlink1`) will result in a shell error when attempting to read the file, as the system detects the infinite loop.

Process Trajectory: To master job control, a student might launch a foreground delay process (e.g., `sleep 30`), immediately suspend it using `Ctrl+z`, verify its suspended state using `jobs`, reactivate it silently using `bg`, suspend it again, and finally pull it back to the active terminal using `fg`.

CHAPTER 2: System Calls

Module 2 - Section A

Computer Systems & Programming (Academic Year 2025-2026)

1. System Programming and The Kernel

To understand system programming, one must first understand the core of the operating system: the kernel. The kernel is the central component of the operating system responsible for managing all hardware resources, which includes the memory, disk drives, and I/O channels. It acts as the ultimate arbiter,

coordinating the use of these resources between various competing applications and users to ensure stability and efficiency. Because it is responsible for such critical, low-level management, the kernel code is always resident in the system's memory.

To allow programmers to interact with these protected hardware resources, the kernel provides a specialized set of functions known as **system calls**. These system calls can be thought of as the fundamental primitives of the operating system. Together, the entire set of system calls forms the Application Programming Interface (API), which acts as the kernel's interface facing the programmer. Therefore, system programming fundamentally consists of writing code that makes use of these kernel system calls to interact with the machine's hardware and operating system features.

2. Deep Dive: System Calls

System calls are ubiquitous in operating system environments; all programs that run under Linux ultimately rely on this specific interface to function. Even higher-level system libraries heavily rely on system calls under the hood to perform their tasks.

The number of available system calls has grown as the Linux operating system has evolved over time. For example, in the Linux kernel version 2.6, there were approximately 280 system calls available, whereas in kernel version 5.15, this number increased to about 450. You can find the exact definition of this limit (`NR_SYSCALLS`) located in the `/usr/include/asm-generic/unistd.h` file.

Often, a system call corresponds directly to a standard C library function at a higher level. A classic example of this relationship is the standard C library function `fopen()` , which relies on the lower-level system call `open()` to actually interact with the file system.

When writing code, the invocation of a system call cleanly follows the standard calling syntax of a normal C function, making it intuitive for the programmer. Each of these system calls has a defined prototype, which is located in the system include files (typically found within the `/usr/include` directory and its various subdirectories). An example of a system call prototype is `pid_t fork(void);` .

It is also important to note that security permissions apply to this interface; some system calls require elevated privileges and can only be successfully invoked if the calling process is running with root access. To explore the specific prototype and

necessary include files for any given system call, programmers can utilize the manual pages by typing `man system_call_name` in their terminal.

3. The C Standard Library vs. System Calls

While system calls are the kernel's primitives, programmers frequently use the **C standard library** (`libc`). This library contains a variety of utility functions designed to provide general-purpose services to the programmer, significantly speeding up development.

Crucially, these standard C library functions are *not* direct calls to kernel services. Standard C library functions generally reside in a dynamic library rather than in the kernel itself. While some library functions (like the aforementioned `fopen()`) make use of system calls to implement their specific services, many others do not interact with the kernel at all. For instance, utility functions like `strcpy()` (used for string copying) and `atoi()` (used for converting ASCII strings to integers) perform purely computational tasks in memory and do not invoke any system calls.

Understanding the Distinction

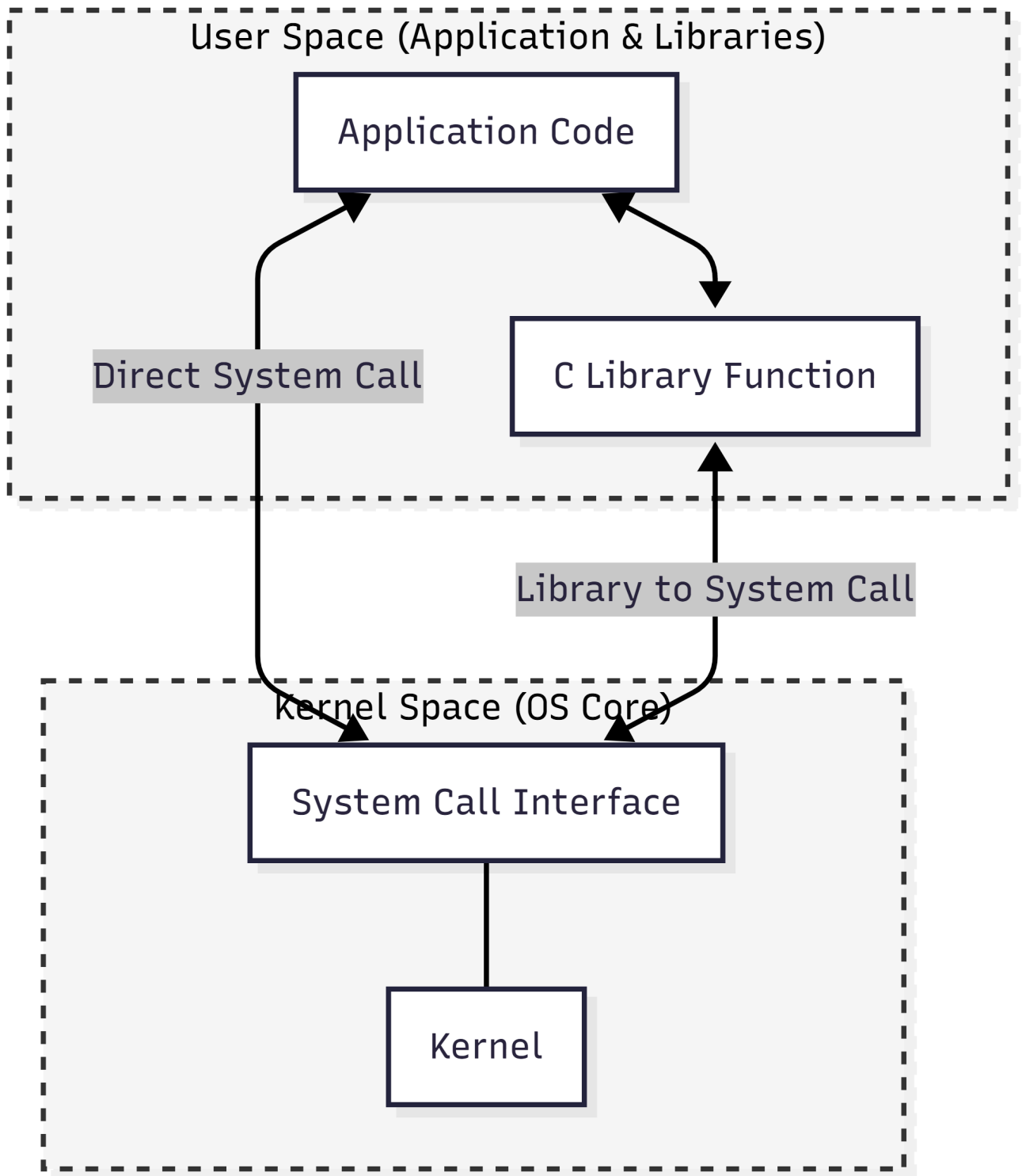
From a standard programmer's perspective, there is no major visible distinction between a library function and a system call, as both are invoked simply as functions in the code. However, from the perspective of an operating system implementer, the difference is monumental.

The primary distinction lies in replaceability and execution space. In general, it is entirely possible to replace a standard library function with a custom function that performs the identical task, perhaps optimizing it or achieving it in a different way. For example, a programmer could easily write their own custom version of `atoi()` or `sqrt()` . Conversely, it is not easily possible to replace system calls. System calls are heavily operating system dependent and must be securely executed by the kernel itself.

Architectural Integration (User Space vs. Kernel Space)

To visualize how these components interact, we can separate the operating environment into two distinct privilege levels: **User Space** and **Kernel Space**.

- **User Space:** This area contains the application code and the C library functions. Application code can communicate back and forth with a C library function, or it can communicate directly with a system call, bypassing the C library entirely.
- **Kernel Space:** Separated by a rigid privilege boundary, this space houses the system calls and the core kernel. C library functions from the user space cross this boundary to communicate with the system calls in the kernel space.



Schema Translation: The diagram illustrates the boundary between user space and kernel space. Application code and C library functions exist in user space, while the

system calls and the kernel exist in kernel space. Bidirectional arrows show that application code can interact with C library functions or directly with system calls. C library functions also interact directly with system calls across the user/kernel space boundary.

4. The `syscall()` Interface and Mechanics

When a system call is made, it utilizes the `syscall()` interface. This interface is fundamentally a request directed to the kernel to perform a specific action.

Triggering this interface allows the system to securely switch from user mode into kernel mode.

This entire process runs within the context of the calling process and follows a strict sequence of events:

1. **Preparation:** The system copies the necessary arguments and saves the current hardware context onto the secure kernel stack.
2. **Dispatch:** It uses a specific system call number to index into a dispatch vector, determining exactly which kernel function to execute.
3. **Return:** Once the kernel finishes the task, it returns the results in the CPU registers, restores the previously saved hardware context, switches back to user mode, and returns control to the library routine.

Assembly Implementation Example (64-bit x86 Linux)

At the hardware level, this process requires precise register management. On a 64-bit x86 Linux machine, the designated system call number must be loaded into the `rax` register. Programmers can find the mapping of these system call numbers in the `/usr/include/asm/unistd_64.h` file.

To illustrate, if a programmer wants to write a message to standard output using raw assembly, the following sequence of instructions is executed:

- `mov $1, %rax` : Loads the system call number (1 for write) into the `rax` register.
- `mov $1, %rdi` : Sets the file descriptor for standard output (`stdout`) in the `rdi` register.
- `mov $msg, %rsi` : Passes the pointer to the message buffer into the `rsi` register.
- `mov $len, %rdx` : Passes the length of the message into the `rdx` register.

- `syscall` : Triggers the actual switch to kernel mode to execute the operation.

5. System Call Errors and Error Handling

System calls can fail for a multitude of reasons during their execution. A failure might occur because the system has completely exhausted the availability of a specific resource, or because a request has exceeded the limit allowed for a single process. Furthermore, the kernel will block execution if the calling process lacks the necessary permissions to request a particular operation. Errors also arise when the arguments passed to a system call are invalid, or when a requested hardware device is missing or not working. Additionally, an interrupt triggered by an external event, such as the reception of a signal, can cause a system call to terminate prematurely. Because of these diverse potential failure points, it is critically important for programmers to always detect and, if necessary, manage any error returning from a system call.

Most system calls communicate their status directly through their return values. Typically, a return value of `0` indicates success, whereas a return value of `-1` signifies that an error has occurred. When a failure happens, the system not only returns `-1` but also assigns a more specific error code to a global variable declared as `extern int errno;`. It is crucial to note that if a system call executes successfully, this `errno` variable is not set or updated.

The specific symbolic names for these numerical error codes are defined within the `errno.h` header file. Among the standard error codes are:

- `EPERM` (value 1): Indicates an "Operation not permitted".
- `ENOENT` (value 2): Indicates "No such file or directory".
- `ESRCH` (value 3): Indicates "No such process".
- `EINTR` (value 4): Indicates an "Interrupted system call".
- `EIO` (value 5): Indicates an "I/O error".
- `ENXIO` (value 6): Indicates "No such device or address".

When an error is detected, the programmed response can take several forms. A developer might choose to display an informative error message, cancel the operation currently in progress, or terminate the program completely by triggering an `exit`. Alternatively, the program might be designed to attempt a re-execution of the failed system call, or in some scenarios, simply ignore the error and continue.

To assist in displaying descriptive error messages, developers use the `perror()` function, which possesses the prototype `void perror(const char *str)`. This function converts the numerical code currently stored in `errno` into a human-readable message, and prints it alongside the custom string `str` placed right before it. For example, if a program executes `fd = open("nonexist.txt", O_RDONLY);` and the file does not actually exist, checking `if (fd == -1)` and subsequently calling `perror("main");` will output the string: `--> main: No such file or directory`.

6. Categorization of System Calls

The operating system offers a wide variety of system calls that can be categorized by their core functions. These categories include system calls dedicated to process control, memory management, and the management of files and file systems. The kernel also provides specific calls for handling signals, managing networking protocols, and facilitating Inter-Process Communication (IPC). Programmers seeking comprehensive documentation can utilize the terminal commands `man 2 intro` and `man 2 syscalls` to access detailed information regarding all the system calls available under Linux.

7. Standardization and Portability

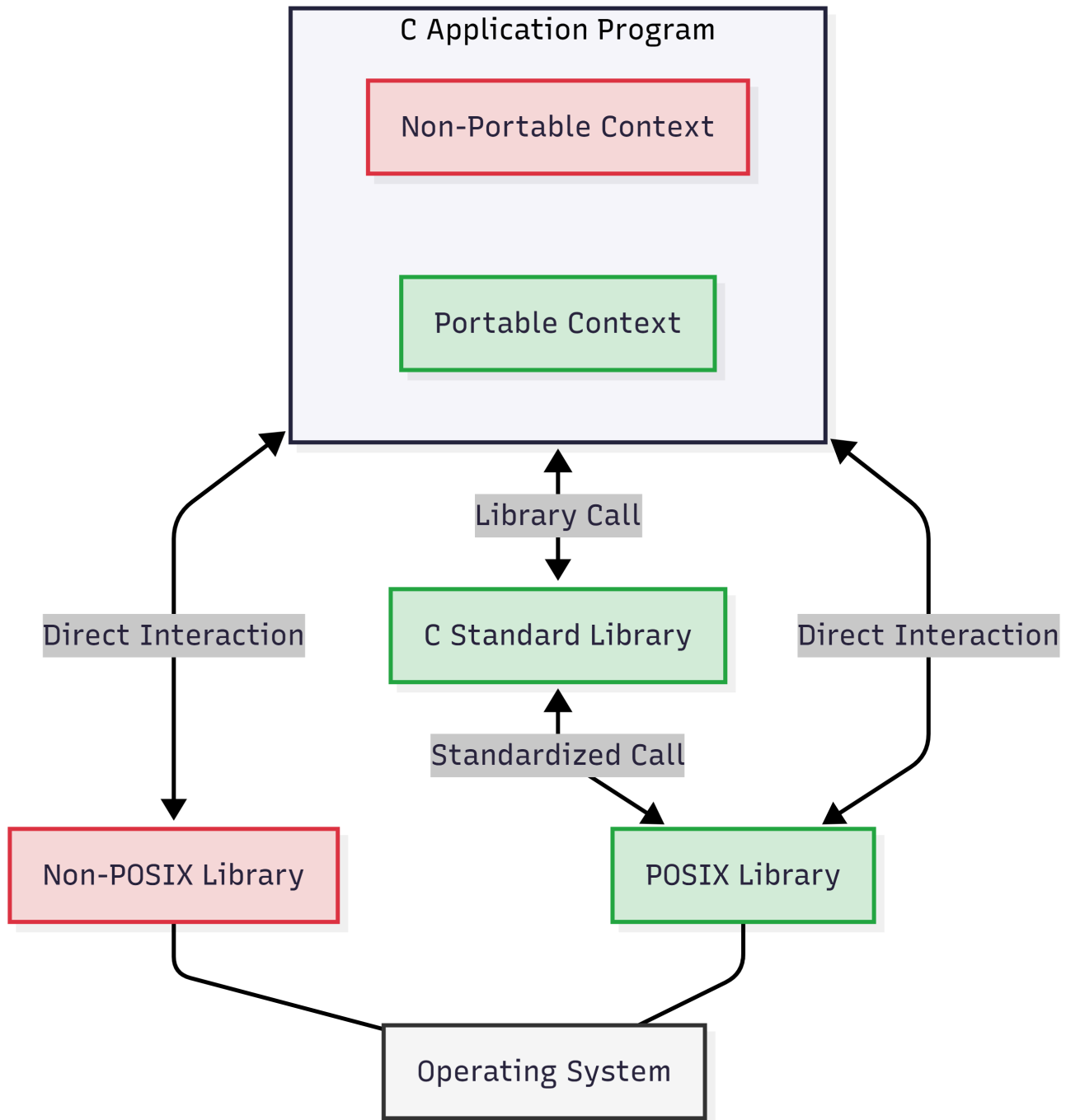
To facilitate the seamless porting of applications between different operating systems, a reference standard has been defined for the system calls that a kernel should offer to a programmer. This crucial standardization effort is known as POSIX, which stands for Portable Operating System Interface. POSIX represents a family of standards developed by the IEEE.

The key components of this standard are:

- **IEEE 1003.1 (POSIX.1):** Focuses on the definition of system calls.
- **IEEE 1003.2 (POSIX.2):** Focuses on shells and utilities.
- **IEEE 1003.7 (POSIX.7):** Focuses on system administration.

[Photo: System Call Standardization and Portability Architecture] The architecture of system call standardization illustrates how an application maintains its portability across different operating systems. At the top level resides the C application program. This application can interact with lower-level components through three distinct pathways: interfacing directly with a Non-POSIX Library, interfacing directly with the POSIX Library, or utilizing the C Standard Library, which internally communicates with the POSIX Library. Both the POSIX Library

and the Non-POSIX Library sit directly on top of the base Operating System. Crucially, the architectural diagram highlights that only the portions of the C application program that rely strictly on the C Standard Library and the POSIX Library are designated as "portable". Bypassing these standardized interfaces to use a Non-POSIX Library strictly binds the application to a specific operating system architecture, thereby breaking its portability.



CHAPTER 3: File Management

1. Introduction to File Management

The manipulation and management of files are fundamental operations within computer systems. The system calls utilized for these tasks within Linux

environments have direct equivalents in the C standard I/O library, with their corresponding function prototypes defined in the `<stdio.h>` header file. When working with these system calls, it is crucial to understand the concept of atomic operations; because files can be shared between multiple concurrent processes, ensuring that operations happen indivisibly prevents data corruption and race conditions.

Before a program can interact with a file, the file must be explicitly opened using the `open` system call. To the operating system's kernel, every successfully opened file is tracked and identified using a file descriptor.

When the `open` system call is invoked, the kernel performs several critical steps:

- It locates the specific file within the file system hierarchy using its provided pathname.
- It copies the file's i-node (index node) from the disk into the system's memory. This caching into memory greatly speeds up subsequent accesses but introduces potential synchronization problems if the file is modified by different processes simultaneously, as the memory copy and disk copy must be kept consistent.
- It associates a non-negative integer—known as the file descriptor—with the opened file. This integer is subsequently used by all other file-related system calls to reference this specific open instance.

To provide a high-level overview, the main system calls for file management are detailed in the following reference table:

System Call	Meaning
<code>open</code>	Opens a file for reading and/or writing or creates a new file.
<code>creat</code>	Creates a new file.
<code>close</code>	Closes a previously opened file.
<code>read</code>	Reads data from a file.
<code>write</code>	Writes data to a file.
<code>lseek</code>	Moves the read/write pointer to a specified byte within the file.
<code>unlink</code>	Removes a file.
<code>ioctl</code>	Allows the manipulation of specific device parameters.
<code>dup</code>	Duplicates an existing file descriptor.

System Call	Meaning
fcntl	Controls the attributes associated with a file and allows for file locking.
stat	Returns detailed status information about a file.

Once a process no longer requires access to a file, it must be explicitly closed. The `close()` system call takes the file descriptor and makes it available for future use, essentially freeing up an array entry in the process's descriptor table.

It is worth noting that some files do not require explicit opening by the programmer because the operating system shell automatically opens them when a program executes. These pre-opened streams are associated with fixed file descriptor numbers: `0` for standard input (stdin), `1` for standard output (stdout), and `2` for standard error (stderr). To adhere to the POSIX.1 standard and ensure code portability, developers should replace these hardcoded numbers with their related symbolic constants defined in the `<unistd.h>` library: `STDIN_FILENO` (`0`), `STDOUT_FILENO` (`1`), and `STDERR_FILENO` (`2`).

2. The Architecture of File Sharing

Linux natively supports the sharing of open files across different, independent processes. To master how file sharing mechanisms operate, one must examine the specific data structures that the kernel maintains to track open files.

The kernel retains a centralized "process table" in memory containing information about every active process on the system. Within this structure, a dedicated memory area called a *process table entry* is reserved for each individual process. For tracking open files, three distinct, interconnected tables are maintained by the kernel:

1. **The Table of Open File Descriptors (Process-Wide):** Located inside each process table entry, this table is private to the individual process. It holds the open file descriptors specific to that process. Associated with each file descriptor in this table are the file descriptor flags and a direct pointer to a corresponding entry in the global file table.
2. **The File Table (System-Wide):** This is a global table tracking all files currently opened by all processes running on the system. Each record within it is known as a *file table entry*. A file table entry contains the file status flags (such as read, write, append, sync, and nonblocking), the current logical offset

(read/write pointer) for that specific file instance, and a pointer to an entry in the v-node table.

- 3. The V-node Table (System-Wide):** As part of the virtual file system interface, this table houses *v-nodes*. Each entry in the v-node table contains foundational file type information, pointers to functions capable of operating on the file, and typically the actual i-node of the file (or a pointer to it). The i-node holds critical disk-level data such as the file's owner, its total size, and pointers connecting to the raw data blocks on the storage device.

To fully appreciate the complexity of the internal structures, consider the Linux `struct inode` defined in the kernel headers

(`/usr/src/kernels/.../include/linux/fs.h`). This structure houses comprehensive metadata needed by the kernel. While highly technical, key fields within the `struct inode` include list heads for hashing and superblocks (`i_hash`, `i_list`, `i_sb_list`, `i_dentry`), basic identifiers (`i_ino`, `i_mode`, `i_uid`, `i_gid`), sizing and block data (`i_size`, `i_blkbits`, `i_blksize`, `i_blocks`, `i_bytes`), access timestamps (`i_atime`, `i_mtime`, `i_ctime`), and pointers to function operations (`i_op`, `i_fop`). It also includes synchronization mechanisms like spinlocks (`i_lock`), semaphores (`i_sem`, `i_alloc_sem`), and structures handling quotas, pipes, and block devices.

2.1 Visualizing File Sharing Scenarios

The way these three tables interact changes depending on how processes open and interact with files.

Scenario A: A Single Process Opening Two Separate Files

If a single process opens two entirely different files, the process table will utilize two distinct file descriptors (e.g., index 3 and 4). Each descriptor points to its own unique file table entry, which tracks its independent status flags and current offset. In turn, each file table entry points to a completely separate v-node table entry containing the specific i-node information for each respective file.

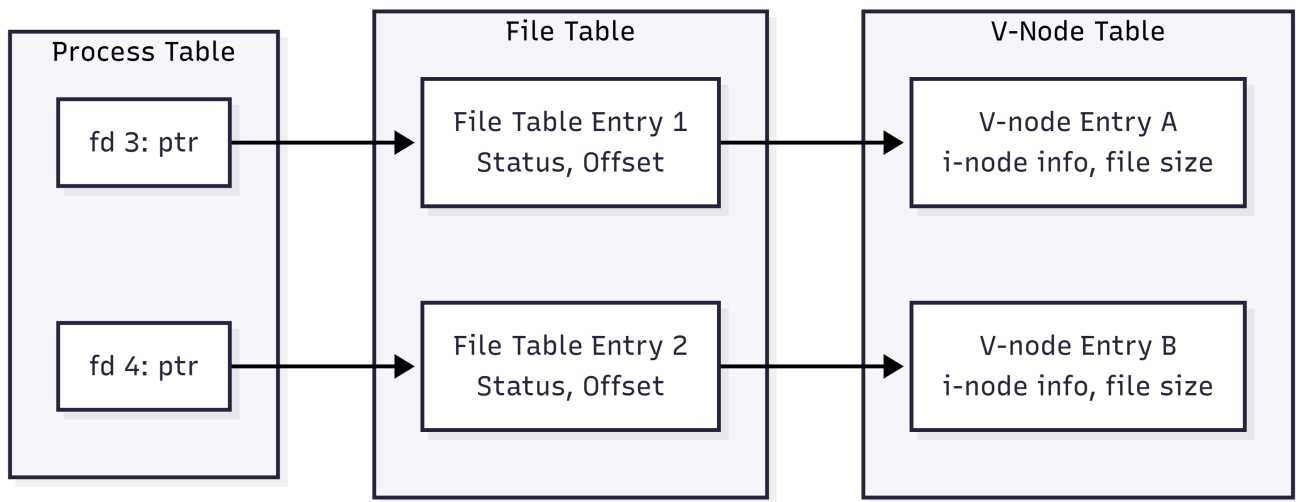


Photo: Schema translating the tables when a process opens two separate files, demonstrating independent pointers across all three kernel tables.

Scenario B: Two Independent Processes Opening the Same File

If two separate processes independently call `open()` on the exact same file, they will receive two distinct file table entries. Consequently, each process maintains its own independent current offset for reading and writing. However, because they are accessing the same physical file on disk, both of their separate file table entries will ultimately point to the exact same v-node table entry.

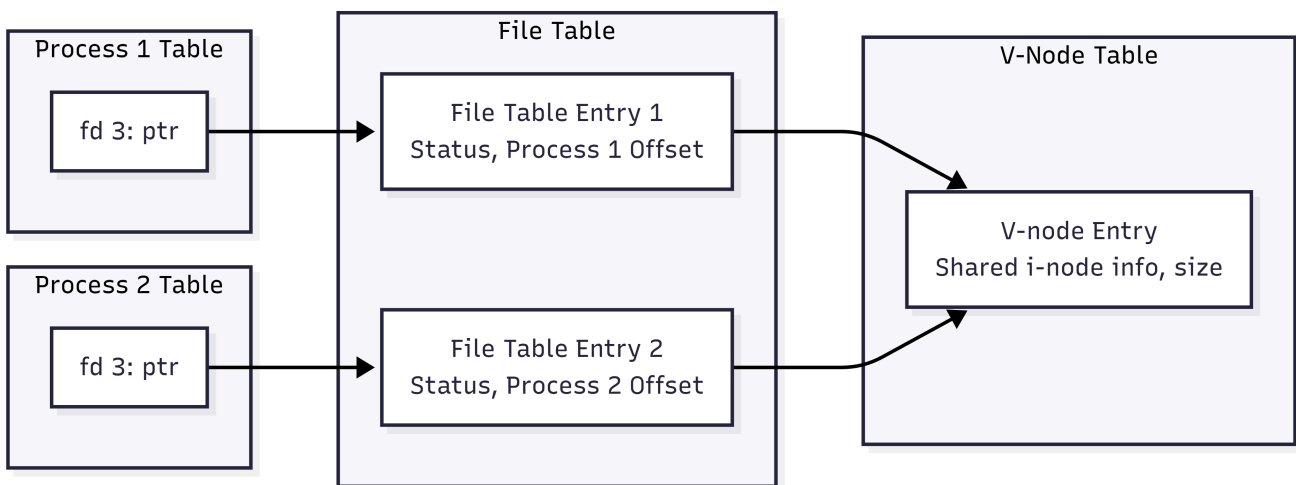


Photo: Schema translating two processes opening the same file. They feature separate File Table Entries, ensuring isolated offsets, but converge on a single V-Node Table Entry.

Scenario C: Parent and Child Processes Sharing the Same File

When a parent process opens a file and subsequently spawns a child process using mechanisms like `fork()`, the child inherits the parent's file descriptors. In this case, both the parent and child process tables possess a pointer that directs to the *exact same file table entry*. Because they share the file table entry, they also share the current offset; if the child reads data and advances the offset, the parent's next

read will begin from that newly advanced position.

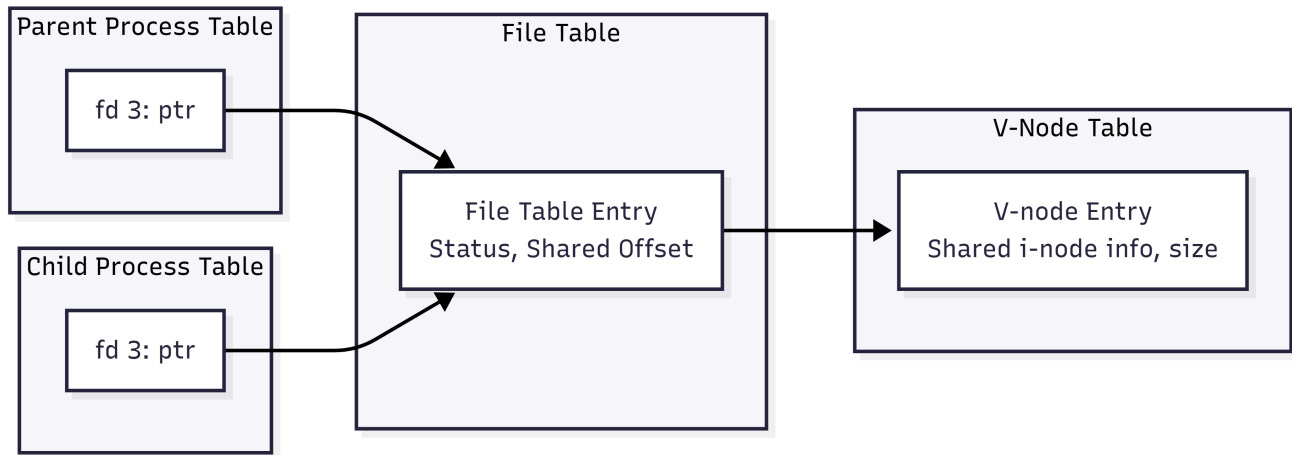


Photo: Schema illustrating a parent and child process utilizing the same file. They point to the same File Table Entry, directly sharing the file offset and flags.

2.2 Implications of Table Structures on Operations

Understanding this triple-layered organization clarifies how standard operations impact the file system:

- **Write Operations:** After a `write` operation, the current offset within the specific file table entry is incremented by the amount of data written. If this new offset exceeds the current recorded size of the file, the file size metric within the v-node table entry is correspondingly incremented.
- **Appending (`O_APPEND`):** If a file is opened with the `O_APPEND` flag, this flag is set inside the file table entry. During every write operation, the system automatically forces the current offset to match the total file size (found in the v-node table entry) before writing, ensuring data is always added to the absolute end of the file.
- **Seeking (`lseek`):** The `lseek` operation strictly updates the current offset variable located in the corresponding file table entry; it does not alter the underlying file size or v-node.
- **Descriptor Duplication:** It is entirely possible for multiple file descriptors to point to a single file table entry. This can happen within the same process by using the `dup()` system call, or between two different processes following a `fork()` operation.

Because flags govern file behavior, it is vital to distinguish between the two types of flags:

1. **File Descriptor Flags:** These are tied to the file descriptor itself and are therefore particular and isolated to a specific process.

2. **File Status Flags:** These are tied to the file table entry, meaning they can be shared among multiple processes if those processes share the entry (such as parent/child processes).

Programmers have the ability to modify these flags using the `fcntl()` system call. However, due to the high degree of structural sharing permitted by this architecture, developers must be extremely mindful of concurrency issues, as modifying shared data structures can lead to unintended consequences if not managed properly.

3. Basic File Operations: Opening and Creating Files

To manipulate data on a storage device, a process must first request access through the operating system. This is fundamentally achieved using the `open` system call, which either opens an existing file or creates a new one.

The `open` function is invoked with the signature `int open(const char *pathname, int flags);`. It takes a `pathname`—which can be either an absolute or relative path—and an integer representing the access `flags`. If successful, it returns the file descriptor that will be used to refer to the file in subsequent operations. If an error occurs, it returns `-1`.

The `flags` argument determines the access mode. A program must specify exactly one of the following base constants:

- `O_RDONLY`: Opens the file for read-only access (represented by the value 0).
- `O_WRONLY`: Opens the file for write-only access (represented by the value 1).
- `O_RDWR`: Opens the file for both reading and writing (represented by the value 2).

Beyond these primary access modes, developers can apply additional behavioral constants by combining them using a bitwise OR operation. For instance, `O_CREAT` dictates that a new file should be created if it does not already exist. If `O_EXCL` is used in conjunction with `O_CREAT`, the `open` call will intentionally fail and return an error if the file is already present on the system, preventing accidental overwriting. If a developer wishes to discard the previous contents of an existing file upon opening it, they can use the `O_TRUNC` flag, which truncates the file to zero bytes.

Other specialized flags include `O_APPEND`, which forces all writes to occur at the absolute end of the file; `O_NONBLOCK`, which sets non-blocking I/O operations for special files like pipes and FIFOs; and `O_SYNC`, which enforces synchronous writing. When `O_SYNC` is active, every write operation blocks the process until the data has been physically written to the underlying hardware device, rather than just being buffered in the kernel's memory.

If the `O_CREAT` flag is utilized, the `open` function requires a third argument to define the file's initial permissions: `int open(const char *pathname, int flags, mode_t mode);`.

Alternatively, the `creat` system call can be used specifically for file generation. Its signature is `int creat(const char *pathname, mode_t mode);`. This function creates a new regular file with the specified pathname and automatically opens it for writing. If the file already exists, it is truncated, though its owner and permissions remain unchanged. The `mode` argument specifies the initial access permissions using an octal constant (for example, `0644` corresponds to read and write permissions for the owner, and read-only permissions for the group and others, often notated as `rw-r--r--`). When created, the file's owner is automatically set to the effective user-ID of the calling process. Functionally, calling `creat(pathname, mode)` is entirely equivalent to calling `open(pathname, O_WRONLY | O_CREAT | O_TRUNC, mode)`. Both functions return a new file descriptor or `-1` on error.

4. Closing, Reading, and Writing Data

Once a process has finished its operations, it must release the file descriptor using the `close` system call: `int close(int fildes);`. This function closes the file associated with `fildes` and returns `0` upon success or `-1` on failure. While the Linux kernel automatically closes all open files when a process terminates, it is considered excellent programming practice to explicitly close files to prevent resource exhaustion, especially in long-running applications. It is also important to note that a single file can be opened multiple times by the exact same process, meaning multiple distinct file descriptors can refer to the same underlying file.

To extract data from an opened file, the `read` system call is used: `ssize_t read(int fildes, void *buf, size_t nbyte);`. This function attempts to read a sequence of `nbyte` bytes from the file identified by `fildes`, starting precisely from the current file offset. The retrieved data is saved into the memory buffer pointed to by `buf`. Upon completion, `read` automatically updates the

current file offset to reflect the amount of data consumed. It returns the number of bytes actually read, or `-1` in the event of an error. Notably, the number of bytes returned can be less than the number requested. This partial read typically occurs when the end of a regular file is reached before filling the buffer, or when reading from a terminal, network socket, or pipe where the requested amount of data is not currently available.

Conversely, the `write` system call pushes data to a file: `ssize_t write(int filedes, const void *buf, size_t nbyte);`. This function writes `nbyte` bytes from the memory buffer `buf` into the file starting at the current read/write offset. Like `read`, it updates the current file offset upon completion and returns the actual number of bytes written, or `-1` on an error (a return value of `0` indicates no bytes were written). Developers must be aware that a successful `write` operation—one that completes without returning an error—does not mathematically guarantee that the data has been physically committed to the disk. The kernel often caches writes for performance; to force a physical disk update, the `fsync` function must be utilized.

5. File Navigation, Offsets, and "Holes"

Every open file maintains a "current file offset"—a non-negative integer that tracks the position of the read/write pointer. It measures the exact number of bytes from the absolute beginning of the file. As previously mentioned, `read` and `write` operations naturally begin at this offset and increment it by the number of bytes processed. When a file is initially opened, the offset is set to `0`, unless the `O_APPEND` flag is used, which places the pointer immediately after the file's final byte.

To manually navigate within a file without reading or writing data, programs use the `lseek` system call: `off_t lseek(int filedes, off_t offset, int whence);`. This function explicitly repositions the read/write pointer by moving it `offset` bytes relative to a reference point dictated by the `whence` parameter:

- `SEEK_SET` : The offset is evaluated relative to the beginning of the file.
- `SEEK_CUR` : The offset is evaluated relative to the current pointer location.
- `SEEK_END` : The offset is evaluated relative to the end of the file.

The `lseek` function returns the newly updated file offset, or `-1` if an error occurs. The `offset` argument can be negative, allowing the pointer to move backward as long as it starts from a valid `whence` reference. However, any

attempt to navigate to a position before the physical beginning of the file generates an error.

Interestingly, POSIX systems allow the pointer to be moved *beyond* the current end of the file. If a program seeks past the end of a file and subsequently writes data, it creates a "hole". A hole is an empty expanse of logical space within the file that contains no physical data. If a process attempts to `read` from this unwritten region between the old file end and the new data, the operating system transparently returns ASCII null characters (`\0`).

Because holes do not take up actual storage space, the logical size of a file and the physical number of disk blocks it occupies may drastically differ. For example, inspecting a "sparse file" with a large hole using the `ls -l` or `stat` command might report a file size of over a gigabyte (e.g., 1,073,741,832 bytes), but the actual block allocation (seen via `df` or the `Blocks:` field in `stat`) might be a mere 32 blocks.

To append data, programmers typically employ one of two strategies:

1. **Manual Seek:** Use `lseek(filedes, (off_t)0, SEEK_END);` to move the pointer to the end, followed by a `write()` call.
2. **Flag Enforcement:** Use the `open` function with the `O_APPEND` flag (`O_WRONLY | O_APPEND`), which guarantees the kernel inherently positions the pointer at the end of the file prior to every single write operation.

6. Practical Application Scenarios (Study Exercises)

Mastering these file IO concepts requires practical application. Standard programming exercises in systems architecture often involve building utilities based entirely on these primitives:

- **Stream Processing:** Writing a basic echo program that reads a line of input from `stdin` (file descriptor 0) and pushes it to `stdout` (file descriptor 1), terminating when an empty line is detected.
- **Data Partitioning:** Creating a program that takes an input file and a specified block size (in bytes or megabytes) and splits the source file into multiple, automatically named sequential output files.
- **Concurrency and Sharing:** Examining how file descriptors are shared between parent and child processes. If a parent opens a file and then uses

`fork()` to create a child, both processes share the same file table entry, meaning their writes will naturally append to one another. However, if the `open()` call is moved *after* the `fork()`, the parent and child acquire separate file table entries; unless `O_APPEND` is used, their separate offsets will cause them to overwrite each other's data.

7. Advanced File Control: The `fcntl` System Call

While standard operations like `read` and `write` handle the basic flow of data, complex applications often require fine-grained manipulation of a file's underlying properties after it has already been opened. This is achieved using the `fcntl` (file control) system call, which allows a program to exercise a significant degree of control over active file descriptors.

The `fcntl` function is defined in the `<fcntl.h>` header and takes three different forms depending on the specific command being executed:

1. `int fcntl(int fd, int cmd);`
2. `int fcntl(int fd, int cmd, long arg);`
3. `int fcntl(int fd, int cmd, struct flock *lock);`

The first argument, `fd`, is always the descriptor of the open file to operate on. The `cmd` argument specifies the operation to execute. If the command requires additional parameters, the third argument acts as a catch-all, taking either an integer value (`arg`) for general cases or a pointer to a `flock` structure (`lock`) when managing record locks.

The operations available through `fcntl` are grouped into several categories:

- **File Descriptor Duplication (`F_DUPFD`):** This command duplicates the file descriptor specified by `fd` and returns a new file descriptor. The system guarantees that the newly allocated file descriptor will be the lowest available integer value that is greater than or equal to the third argument, `arg`.
- **Managing File Descriptor Flags (`F_GETFD` and `F_SETFD`):** These commands retrieve or modify the flags associated strictly with the file descriptor (which are private to the process). Currently, the primary file descriptor flag defined in Unix-like systems is `FD_CLOEXEC` (close-on-exec). If `FD_CLOEXEC` is set to a non-zero (true) value, the operating system will automatically close the file descriptor when the process executes an `exec`

system call to launch a new program. `F_SETFD` uses the `arg` parameter to establish the new set of flags.

- **Managing File Status Flags (`F_GETFL` and `F_SETFL`):** Unlike descriptor flags, status flags are tied to the system-wide file table entry. `F_GETFL` returns the file status flags that were originally established when the file was opened via `open()`. Because the access mode flags (`O_RDONLY`, `O_WRONLY`, `O_RDWR`) are not mathematically distinct bits that can be isolated simply, a program must use the bitwise AND operator (`&`) with the `O_ACCMODE` mask (which has a value of 3) to isolate the access mode. Once isolated, the result can be compared to determine the read/write state. Other status flags, such as `O_APPEND`, `O_NONBLOCK`, and `O_SYNC`, can be checked using standard bitwise testing. When using `F_SETFL` to modify status flags, the program must pass the new flags via `arg`. However, a program can only change the `O_APPEND`, `O_NONBLOCK`, `O_ASYNC`, and `O_DIRECT` flags; the fundamental access mode (read/write permissions) established at opening must remain entirely unchanged.
- **Asynchronous I/O and File Locking:** `fcntl` also supports getting and setting asynchronous I/O ownership (`F_GETOWN`, `F_SETOWN`) and managing record locks to prevent concurrent modification collisions (`F_GETLK`, `F_SETLK`, `F_SETLKW`).

Because `fcntl` alters flags, developers must be careful not to accidentally overwrite existing flags when setting new ones. To behave correctly when turning a flag on, a program should first use `F_GETFL` to retrieve the current integer value of the flags, apply a bitwise OR operation (`|=`) to integrate the new flag, and finally use `F_SETFL` to save the combined updated value back to the file table entry.

8. Device Manipulation and Forced Synchronization

While system calls like `read`, `write`, and `lseek` provide a standardized interface for interacting with regular files, operating systems must also communicate with hardware devices that have highly specialized behaviors. The `ioctl` (Input/Output Control) system call acts as a catch-all interface to collect all behaviors and configurations that cannot be contained within the standard file-based paradigm.

The signature is `int ioctl(int fd, int request, ...);`. It is primarily used to manipulate device parameters, encompassing a wide variety of operations such as disk label I/O, specialized file I/O, magnetic tape configurations, socket networking protocols, and terminal I/O. For example, if a developer needs to programmatically determine the window size of a terminal emulator, they cannot simply read the terminal file. Instead, they must call `ioctl` on the standard input file descriptor (`STDIN_FILENO`), passing the terminal-specific request constant `TIOCGWINSZ`, and a pointer to a `winsize` structure where the operating system will populate the current row and column dimensions.

Another critical edge case in file management is data synchronization. As previously established, for the sake of system efficiency, the kernel often buffers data in physical memory for several seconds before actually writing it to the underlying disk. If the system loses power during this window, data modified via `write` will be lost. To prevent this in critical applications (like databases), developers invoke the `fsync` system call: `int fsync(int fd);`. This function strictly commands the kernel to "flush" all data currently buffered in memory for the specified file descriptor directly to the corresponding hardware device. It halts process execution until the physical write is complete, returning `0` upon success and `-1` upon encountering an error.

9. File and Directory Attributes (The `stat` Family)

Beyond the raw data contained inside a file, the operating system maintains extensive metadata regarding the file's type, ownership, permissions, and history. To retrieve this metadata, Linux provides the `stat` family of system calls: `stat`, `fstat`, and `lstat`.

All three functions populate a specialized data structure, `struct stat`, with the requested file's information.

- `int stat(const char *file_name, struct stat *buf);` identifies the file using its absolute or relative pathname.
- `int fstat(int filedes, struct stat *buf);` identifies a file that is already open, using its file descriptor.
- `int lstat(const char *file_name, struct stat *buf);` acts exactly like `stat`, except when the pathname points to a symbolic link. In that scenario, `lstat` returns the metadata about the symbolic link itself, whereas `stat` would resolve the link and return metadata about the target file.

The `struct stat` is a comprehensive blueprint of a file's existence within the inode table. Key fields include:

- `st_dev` and `st_ino` : The hardware device ID and the specific inode number identifying the file.
- `st_mode` : An integer detailing the file's protection permissions and its fundamental type.
- `st_nlink` : The total number of hard links pointing to this inode.
- `st_uid` and `st_gid` : The User ID and Group ID of the file's owner.
- `st_rdev` : The device type, applicable only if the inode represents a special device node.
- `st_size` : The total logical size of the file, measured in bytes.
- `st_blocks` : The total number of physical storage blocks allocated to the file. While these blocks are typically measured in 512-byte increments, this can vary by system implementation and is not universally portable.
- `st_blksize` : The optimal I/O block size for read and write operations to maximize performance on the hosting filesystem.
- `st_atime` , `st_mtime` , and `st_ctime` : Timestamps recording the file's history.

The `st_mode` field is particularly dense, as it contains both the permissions bitmask and the file type. Developers use pre-defined macro functions to extract boolean values regarding the file type from `st_mode` . These include `S_ISREG()` (is it a regular file?), `S_ISDIR()` (is it a directory?), `S_ISCHR()` (character device), `S_ISBLK()` (block device), `S_ISFIFO()` (pipe/FIFO), `S_ISLNK()` (symbolic link), and `S_ISSOCK()` (socket).

The `st_mode` field also dictates execution privileges through the **set-user-ID** (`S_ISUID`) and **set-group-ID** (`S_ISGID`) bits. When an executable file has the set-user-ID bit toggled on, launching that file causes the operating system to temporarily switch the *effective* user ID of the running process to match the `st_uid` (the owner) of the file, rather than the ID of the user who executed it. A classic example is the `passwd` utility; its owner is `root` , and its set-user-ID bit is active. When a standard user runs `passwd` , their effective user ID elevates to `root` just for that program's duration, granting them the necessary system permissions to modify the highly protected `/etc/passwd` file. The set-group-ID bit performs the exact same function for the effective group ID, matching it to the `st_gid` field.

To evaluate the read, write, and execute permissions stored in `st_mode`, developers compare the field against standardized octal constants. For the owner, these are `S_IRUSR` (read), `S_IWUSR` (write), and `S_IXUSR` (execute). The group equivalents replace `USR` with `GRP` (`S_IRGRP`, etc.), and the permissions for all other users replace it with `OTH` (`S_IROTH`, etc.).

10. Modifying File Attributes and State

Because the file system is dynamic, standard C libraries provide numerous system calls to modify a file's metadata and dimensions.

Truncation and Timestamps

To manually alter the length of a file, programs utilize `int truncate(char* pathname, off_t len);` or `int ftruncate(int filedes, off_t len);`. These commands adjust the file to the exact byte size specified by `len`. If the current file is larger than `len`, the extra data at the end of the file is permanently discarded and lost. If the file is shorter than `len`, the file is stretched, and the newly added space is filled with a logical "hole" (null bytes).

The operating system tracks three distinct timestamps in the `stat` structure, which can also be modified:

1. **`st_atime` (Last Access Time):** Updated whenever the file's data is accessed, such as following a `read` or an `open` creation. It corresponds to the `-u` option in the `ls` command. Modern systems often allow administrators to disable `st_atime` updates to improve filesystem efficiency and prevent unnecessary wear on Solid State Drives (SSDs).
2. **`st_mtime` (Last Modification Time):** Updated whenever the file's actual data content is altered, such as following a `write` or a `truncate`. This is the default timestamp displayed by the `ls` command.
3. **`st_ctime` (Last Status Change Time):** Updated whenever the file's *metadata* within the inode changes. This occurs after modifying permissions (`chmod`), ownership (`chown`), link counts (`link`, `unlink`), or any operation that writes to the file, as writing implicitly updates the file size metadata. It corresponds to the `-c` option in `ls`.

Programmers can artificially set the access and modification times of a file using the `utime` system call: `int utime(char* pathname, struct utimbuf *times);`. The `utimbuf` structure takes two `time_t` variables corresponding to `actime` (access time) and `modtime` (modification time). Altering a file via

`utime` will automatically trigger the kernel to update the `st_ctime` (status change) to the current system time, as the inode has been modified.

Permissions and Ownership

When an application needs to verify if the user has permission to access a file, standard operations evaluate the *effective* user and group IDs. However, in specific security contexts, a program might need to test accessibility based strictly on the *real* user and group IDs. This is accomplished using `int access(const char* pathname, int mode);`. The `mode` parameter is formed via a bitwise OR of testing constants: `R_OK` (test for read), `W_OK` (test for write), `X_OK` (test for execute), and `F_OK` (test if the file exists at all).

When a process creates a new file, the initial permissions (requested in `open` or `creat`) are filtered through the process's `umask`. The system call `mode_t umask(mode_t cmask);` changes the bitmask used for file creation. During creation, any permission bits that are set to "on" within the `umask` will be forcefully turned "off" in the access mode of the resulting file, effectively acting as a security filter.

Permissions can be changed retroactively using `int chmod(const char* path, mode_t mode);` (for paths) or `fchmod` (for file descriptors). To execute a `chmod`, the effective user ID of the calling process must either exactly match the owner of the file, or the process must hold root privileges.

Similarly, the ownership of a file is defined upon creation; the owner is set to the effective user ID of the creating process. The group ID of a new file is dictated by POSIX standards to be either the effective group ID of the process OR the group ID of the parent directory it is created in (which sometimes depends on the parent directory's set-group-ID bit). Ownership can be transferred using the `chown`, `fchown`, or `lchown` (for symbolic links) system calls, passing the new `uid_t owner` and `gid_t group`. However, to prevent users from bypassing disk storage quotas by giving their large files away to other users, many operating systems restrict `chown` usage exclusively to the superuser; this restriction status is defined by the `POSIX_CHOWN_RESTRICTED` system constant.

11. File System Links: Hard and Symbolic

The Unix file system architecture abstracts filenames away from the actual data stored on the disk. The data and metadata are housed in the inode, while directories simply contain lists that map human-readable text strings (filenames)

to specific inode numbers. This separation allows multiple filenames, even residing in completely different directories, to point to the exact same physical inode on the disk. These pointers are known as **Hard Links**.

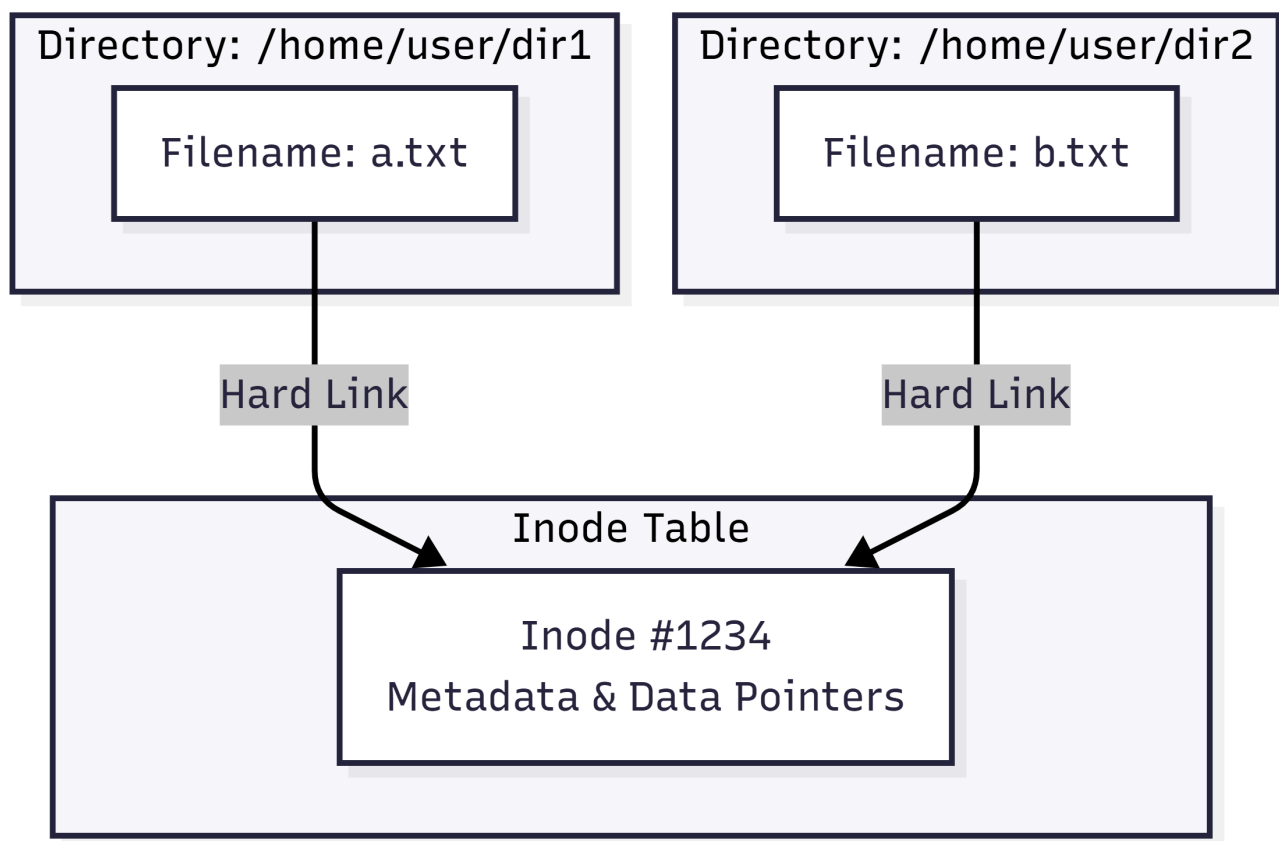


Photo: Schema of Hard Links. Two separate directory entries, `a.txt` in `dir1` and `b.txt` in `dir2`, both possess pointers directing to the exact same inode in the system's global Inode Table.

Because multiple entries can point to one file, every inode maintains an `st_nlink` counter to track how many active directory entries reference it. To create a new hard link to an existing file, developers use `int link(char* oldpath, char* newpath);`. This is an atomic operation that adds the `newpath` directory entry and strictly increments the link counter on the target inode.

To remove a file mapping, the `int unlink(char* path);` system call is used. `unlink` removes the specified directory entry and decrements the inode's link counter. A file's physical data blocks and inode are only deleted by the system when the `st_nlink` counter reaches absolute zero *and* no running processes currently hold the file open. (This mechanism allows programs to create temporary files, instantly `unlink` them so they vanish from the directory structure, and safely use them knowing the OS will automatically purge the data the moment the process terminates). The `rename` system call (`int rename(char* oldpath, char* newpath);`) also manipulates these entries, safely moving a pointer from

one name to another, replacing existing files if necessary (provided the paths do not conflict in type, such as overwriting a directory with a regular file).

Hard links have two major architectural limitations:

1. **Cross-Filesystem Limits:** Because a hard link is just an index number pointing to a local inode table, hard links cannot point to files located on completely different file systems or disk partitions.
2. **Directory Loop Limits:** Standard users are strictly forbidden from creating hard links to directories. If directory hard links were permitted, it would be trivial to create infinite loops in the file system namespace, breaking search algorithms and causing catastrophic failures. The only allowed hard links for directories are structurally generated by the system: the `.` entry (pointing to the directory itself) and the `..` entry (pointing to the parent). Consequently, every sub-directory inherently adds a hard link back to its parent.

To bypass these limitations, Unix systems utilize **Symbolic Links** (symlinks). A symbolic link is not a direct inode pointer; it is a special type of file whose literal data content is simply the text string of another file's pathname. Because it is just a text path, it can point across different file systems, and it can safely point to directories. However, symlinks *can* create logical loops, so programs traversing the file system must be coded to detect and handle them.

When executing a system call on a symbolic link, the operating system will either transparently "follow" the link (applying the action to the target file) or not follow it (applying the action to the link itself). Functions that strictly do *not* follow symbolic links include `lchown`, `lstat`, `readlink`, `remove`, `rename`, and `unlink`. A symbolic link is created via `int symlink(char* oldpath, char* newpath);`, and a program can read the literal text path stored inside a link using `int readlink(char* path, char* buf, int size);`.

12. Directory Management and Navigation

Directories themselves are fundamentally files, but to preserve structural integrity, only the kernel is permitted to write data directly to a directory. Standard users can read directories if they possess the read permission, but modifying the directory contents requires using dedicated system calls (like `open`, `creat`, `unlink`, `rename`, `mkdir`, and `rmdir`) which enforce the system's write permissions on the user.

A new, empty directory is created using `int mkdir(char* path, mode_t mode);`. The permissions specified in `mode` are subject to the calling process's `umask`, and it is vital that execution rights (which dictate "search" access for directories) are provided. A directory can only be removed via `int rmdir(char* path);`, and it will only succeed if the directory is completely empty.

Every active process maintains a "current working directory," which serves as the root for resolving any relative pathnames. The process can change its working directory using `int chdir(char* path);` (providing a string path) or `int fchdir(int filedes);` (providing a descriptor to an open directory). To ascertain where the process currently is, the C Standard Library provides `char* getcwd(char* buf, size_t size);`, which traces the path back to the root and returns the absolute pathname string into the provided buffer.

Because reading the raw binary data of a directory using standard `open()` and `read()` calls is incredibly painful and non-portable, developers use the C standard library's directory stream functions.

- `DIR *opendir(const char *pathname);` opens a directory stream, returning a pointer to a `DIR` descriptor structure (or `NULL` on error).
- `struct dirent *readdir(DIR *dir);` iterates through the open directory. Each call returns a pointer to a `dirent` structure containing information about the *next* entry in the directory, returning `NULL` when the end is reached. Note that the kernel makes absolutely no guarantees regarding the alphabetical or chronological sorting order of the entries returned.
- The `dirent` structure, defined in `<dirent.h>`, prominently features `d_ino` (the inode number) and `d_name` (a null-terminated string of the filename).
- `void rewinddir(DIR *dp);` resets the directory stream back to the very beginning.
- `void closedir(DIR *dp);` safely closes the directory stream to free system resources.

13. Practical Application: Hotel Management Algorithm

A classic example of merging these file I/O concepts is constructing a database-like application using a flat text file. Consider a file named `residents.txt`, representing a 10-room hotel. Instead of searching through the file line by line, the file is formatted with fixed-length records: every line is exactly 41 characters long,

where the first 40 characters hold the guest's name, and the 41st character is the newline (`\n`). Line 1 corresponds to room 1, line 2 to room 2, and so forth.

Because the records are a strictly fixed length, mathematical precision can be used to instantly jump to the required data using the `lseek` system call. The byte offset for any given room is calculated as `(roomno - 1) * NAMELENGTH` .

By opening the file in `O_RDONLY` mode, a function `getoccupier(int roomno)` can calculate the specific offset, invoke `lseek(infile, offset, SEEK_SET)` to snap the read pointer directly to the start of that room's string, and then execute `read(infile, namebuf, NAMELENGTH)` to extract exactly 41 bytes. The program then manually replaces the trailing newline character `namebuf[nread - 1]` with a null terminator `'\0'` to transform it into a standard, printable C-string. This demonstrates how low-level byte navigation provides the foundation for complex data management architectures.

Advanced I/O: Computer Systems & Programming

1. Introduction to Non-Blocking I/O and System Calls

In Unix and Linux operating systems, system calls that involve input/output (I/O) operations are classified as "slow" if there is a possibility that the calling process might be suspended during the execution of the call. Understanding how these slow system calls behave is fundamental to managing complex I/O operations without unintentionally halting program execution.

1.1 The Nature of Slow System Calls

When a process executes a slow system call, it risks being blocked—meaning the process is put to sleep by the kernel until the required conditions are met.

Common examples of slow system calls include:

- **read operations:** A calling process can block when attempting to read if the requested data is not currently available. This typically occurs with devices like pipes, terminal devices, or network interfaces.
- **write operations:** Similarly, a process can block on a write operation if the system cannot temporarily accept the data. A common cause for this is a lack of buffer space on the target device, such as a pipe or network interface.

- **open operations:** The process might block during the initial opening phase if specific required conditions are not satisfied. For instance, if a process calls `open` on a read-only FIFO without specifying the `O_NONBLOCK` flag, it will block if no other process has currently opened that same FIFO for writing.

1.2 Interruptions by Signals

When a process is suspended (stuck) on a slow system call, it can be interrupted if a signal is delivered to it. In this scenario, the system call does not complete successfully; instead, it returns an error and sets the global `errno` variable to `EINTR` (Interrupted system call).

Because an interruption is often an expected part of process management rather than a fatal error, these system calls can sometimes be manually restarted by the programmer. A standard pattern for handling and restarting a system call interrupted by a signal involves using a loop that checks the `errno`:

```
do {
    again = 0;
    if ((n = read(fd, buf, SIZE)) < 0) {
        if (errno == EINTR) {
            again = 1;
        } else {
            // Handle other errors
        }
    } else {
        // No error occurred, process the read data
    }
} while (again);
```

2. Implementing Non-Blocking I/O

To prevent a process from suspending indefinitely, it is possible to configure I/O operations to be non-blocking. When a file descriptor is placed in non-blocking mode, the system behavior changes drastically:

- If the requested I/O is ready, the data is simply read or written as requested.
- If the requested I/O is *not* ready, the system call does not block the process. Instead, it returns immediately with a value of `-1` and sets the `errno` to

EAGAIN (indicating that the resource is temporarily unavailable, and the process should try again later).

There are two primary methods to enable non-blocking mode:

1. **Using `open()`** : You can specify the `O_NONBLOCK` flag when initially opening the file or device. For example: `open(fifoname, O_RDONLY | O_NONBLOCK);` .
2. **Using `fcntl()`** : If the file descriptor is already open, the `fcntl()` function can be utilized to modify its flags and apply `O_NONBLOCK` dynamically. For example: `fcntl(fd, F_SETFL, O_NONBLOCK);` .

3. The Need for I/O Multiplexing

Problems arise when a single process must handle input from multiple sources. Suppose a process needs to read from two distinct file descriptors. If the process attempts a standard blocking `read()` on the first file descriptor (`fd1`) and no data is available, it will suspend. This blocking action prevents the process from reading from the second file descriptor (`fd2`), even if data is actively waiting on `fd2` .

3.1 The Deadlock Scenario

Consider a scenario involving a user at a terminal communicating through a process to a remote modem.

[Photo: I/O Multiplexing Deadlock Scenario]

Description: A structural diagram illustrating a potential deadlock. On the left, a yellow oval represents a "user at a terminal" communicating via file descriptor 1 (`fd1`) to a central red rectangle representing the "process communicating with a modem". On the right, this process communicates via file descriptor 2 (`fd2`) to a green oval representing the "modem", which in turn is connected to a telephone line.



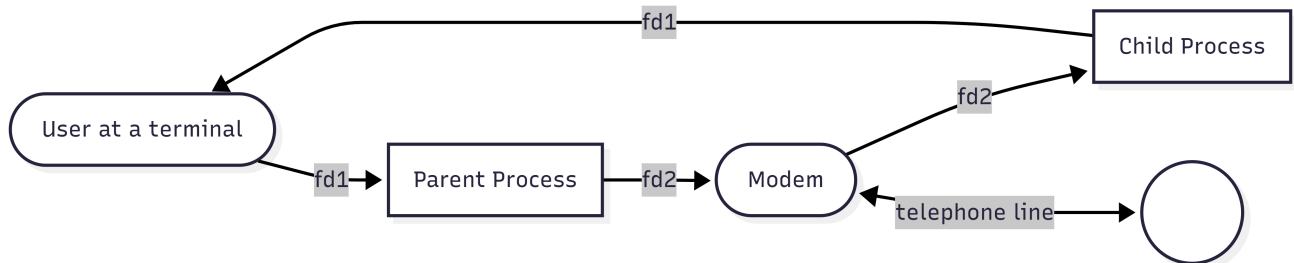
If the central process reads from `fd1` and writes to `fd2` , and simultaneously reads from `fd2` and writes to `fd1` , a deadlock can occur. The process will block on its first `read()` attempt on `fd1` if the user is not typing. Consequently, it is entirely prevented from reading any incoming data from the modem on `fd2` .

3.2 Workaround 1: Using Multiple Processes

One possible solution to the multiplexing problem is to divide the workload using the `fork()` system call to create two distinct processes (a parent and a child).

[Photo: Two-Process Fork Workaround]

Description: A diagram illustrating the fork solution. The single central process from the previous diagram is split into two red rectangles: a parent process ("padre") and a child process ("figlio"). The parent handles data flowing from the terminal (fd1) to the modem (fd2), while the child handles data flowing from the modem (fd2) to the terminal (fd1).



In this architecture, each process is exclusively responsible for managing communication in a single direction. However, this introduces complex inter-process communication and synchronization issues:

- If the remote process disconnects, causing the child process to terminate, the parent process will receive a `SIGCHLD` signal.
- Conversely, if the user disconnects and the parent terminates, the parent must explicitly notify the child to shut down (for example, by sending a `SIGUSR1` signal). Ultimately, while functional, this approach makes the program's structure significantly more complicated.

3.3 Workaround 2: Polling

A single-process alternative is to use polling. This involves setting both file descriptors (`fd1` and `fd2`) to non-blocking mode and utilizing a continuous loop.

In this loop, the process attempts to `read(fd1, ...)`. If data is present, it reads and writes it to `fd2`. If no data is present, the non-blocking call returns immediately. The process then attempts to `read(fd2, ...)` and similarly processes data or returns immediately. To prevent the infinite loop from completely hogging the system's CPU resources, a brief `sleep` command is usually inserted at the end of the loop.

While polling boasts the lowest possible latency for detecting new data, it is generally avoided in professional software development because it is highly

inefficient and wastes significant CPU time actively checking for updates.

4. The `select()` System Call

In many scenarios, the most effective and elegant solution for managing I/O multiplexing is the `select()` system call. This function allows a single process to simultaneously monitor a defined set of file descriptors, pausing the process's execution until one or more of those specific descriptors become ready to perform an I/O operation.

By using `select()`, an application can determine several critical pieces of information simultaneously:

- The total number of file descriptors that are currently ready for I/O.
- Which specific descriptors are ready for a **read** operation (meaning a subsequent `read()` call on that descriptor will not block).
- Which specific descriptors are ready for a **write** operation (meaning a subsequent `write()` call on that descriptor will not block).
- Which specific descriptors have encountered an **exception** condition, such as the arrival of out-of-band data on a network socket.

Armed with this information, the application can perform appropriate I/O functions strictly on the ready descriptors, ensuring the process is never unexpectedly blocked.

4.1 Prototype and Arguments

The `select()` function requires the `<sys/select.h>` header and uses the following C prototype:

```
#include <sys/select.h>

int select(int nfd,
           fd_set *readfds,
           fd_set *writefds,
           fd_set *exceptfds,
           struct timeval *timeout);
```

The arguments dictate precisely what `select()` monitors and for how long:

- **Descriptor Sets (`readfds` , `writefds` , `exceptfds`):** These arguments define the specific conditions to be met. They are sets of file descriptors grouped by the operation you want to monitor (reading, writing, or exceptions).
- **`nfds` :** This is a crucial integer argument that must be set to the value of the highest (maximum) file descriptor being monitored across all sets, **plus one**. For example, if you are monitoring file descriptors 0, 1, and 3, `nfds` must be $3 + 1 = 4$.
- **`timeout` :** This argument defines how long the `select()` call should wait before giving up. It utilizes the `timeval` structure, which specifies time in both seconds (`time_t tv_sec`) and microseconds (`suseconds_t tv_usec`).

4.2 Managing the Wait Time

The `timeout` argument drastically changes the behavior of `select()` :

- **Infinite Wait:** If `timeout` is passed as a `NULL` pointer, the call operates in a blocking mode with an infinite waiting time. The function will only return if at least one descriptor becomes ready for I/O or if an error occurs.
- **Polling (Non-Blocking):** If `timeout` is provided but both `timeout->tv_sec` and `timeout->tv_usec` are set to 0, the call becomes non-blocking. The system performs a quick poll: all requested descriptors are checked instantly, and the function returns immediately regardless of their state.
- **Finite Timeout:** If the `timeval` structure holds a time greater than zero (e.g., `timeout->tv_sec != 0` or `timeout->tv_usec != 0`), `select()` waits for a finite duration. The call will return whenever one of three conditions is met: at least one descriptor becomes ready for I/O, the specified timer entirely elapses, or an error occurs.

4.3 Return Values and Error Handling

When `select()` completes, it returns an integer indicating the outcome:

- $n > 0$: Success. The integer represents the total number of descriptors that are ready for I/O.
- 0: The specified timeout expired before any descriptors became ready.
- -1: An error occurred. When this happens, the global `errno` variable is set to indicate the specific issue.

Common `errno` values set by `select()` include `EBADF` (indicating there is an invalid file descriptor among the provided arguments), `EINTR` (indicating the call was interrupted by a signal before any I/O was ready), `EINVAL` (indicating invalid arguments were supplied, such as a negative number of descriptors), and `ENOMEM` (indicating the system is currently unable to allocate necessary memory).

5. Working with Descriptor Sets (`fd_set`)

The `fd_set` data type is used to construct the lists of file descriptors for `select()` to monitor. Under the hood, this structure typically acts as a bit array, where exactly one bit is allocated to represent each possible file descriptor. Because manually manipulating bits is error-prone, POSIX provides four specialized macros to manage variables of type `fd_set` :

1. **`void FD_ZERO(fd_set *fdset)`** : This macro zeroes out (initializes) the entire set, ensuring no file descriptors are currently marked. This must always be called before adding any descriptors.
2. **`void FD_SET(int fd, fd_set *fdset)`** : This sets the bit for a specific file descriptor (`fd`), adding it to the set for monitoring.
3. **`void FD_CLR(int fd, fd_set *fdset)`** : This removes a specific file descriptor from the set.
4. **`int FD_ISSET(int fd, fd_set *fdset)`** : After `select()` returns, the system modifies the sets in-place to indicate which descriptors actually became ready. This macro tests whether a specific descriptor is still set. It returns a non-zero value if the descriptor belongs to the set (meaning it is ready), and zero otherwise.

5.1 Implementing `select()` : A Structural Example

To use `select()` , you must first allocate variables of type `fd_set` (e.g., `fd_set readset, writeset;`), clear them, and populate them with the file descriptors of interest.

For instance, if a program needs to read from file descriptors 0 and 3, and write to file descriptors 1 and 2, it would construct two sets. Conceptually, the `readset` bit array would look like `1 0 0 1` (where indices 0 and 3 are set to true), and the `writeset` would look like `0 1 1 0` (where indices 1 and 2 are set to true).

The setup phase looks like this:

```

fd_set readset, writeset;

// Clear the sets
FD_ZERO(&readset);
FD_ZERO(&writeset);

// Add the requested file descriptors to the sets
FD_SET(0, &readset);
FD_SET(3, &readset);
FD_SET(1, &writeset);
FD_SET(2, &writeset);

// Call select(). The first arg is max file descriptor (3) + 1.
int n = select(3 + 1, &readset, &writeset, NULL, NULL);

```

Once `select()` returns, the program must evaluate the result. Assuming the return value n is positive (indicating readiness), the application checks which specific descriptors triggered the return using `FD_ISSET`:

```

if (n > 0) {
    if (FD_ISSET(0, &readset)) {
        // Proceed with non-blocking read on fd 0
    }
    if (FD_ISSET(3, &readset)) {
        // Proceed with non-blocking read on fd 3
    }
    if (FD_ISSET(1, &writeset)) {
        // Proceed with non-blocking write on fd 1
    }
    // ... continue checking other descriptors ...
} else {
    // Handle error or timeout
}

```

6. Record and File Locking: Introduction

While multiplexing manages single-process I/O efficiency, concurrent I/O by multiple processes introduces dangerous data integrity issues. When one process

writes to a file that is simultaneously being read by other processes, those readers might consume corrupted, partial, or incomplete data. Even worse, if multiple processes attempt to write to the same file at the exact same time, their output can be mixed unpredictably, or one process might entirely overwrite the output of another.

The structural solution to concurrency issues is **Record/File Locking**. Locking mechanisms allow an operating system to block other processes while one process accesses a file (or specific regions of it), thereby avoiding data corruption and warranting the atomicity of the I/O operation.

6.1 Types of File Locks

There are two primary categories of file locks based on the type of access required:

- **Read Lock (Shared Lock):** This type of lock allows one or more processes to read the file concurrently. It permits shared access but prevents any process from writing to the file while the read lock is active.
- **Write Lock (Exclusive Lock):** This lock grants write access to a single, exclusive process. When an exclusive write lock is active, no other processes can read or write to the locked region.

The basic rules of lock requests follow a strict compatibility matrix:

- If a file has **No Locks**, requesting a Read Lock or Write Lock is permitted (OK).
- If a file has one or more **Read Locks** ($n \geq 1$), requesting an additional Read Lock is permitted, but requesting a Write Lock is **denied**.
- If a file has a single **Write Lock**, requests for any additional locks (Read or Write) are strictly **denied**.

6.2 File Locking Modes: Advisory vs. Mandatory

File locks are enforced in two distinct ways depending on the system's configuration.

Advisory Locking was the first file locking mode implemented in Unix and Unix-like operating systems. In this mode, the kernel does not enforce locks by default during standard I/O operations. Instead, individual processes are fully responsible for asserting locks and explicitly verifying whether conditions exist that prevent access to a file. Because `read()` and `write()` system calls execute

unconditionally and are unaware of potential advisory locks, processes must use specific locking functions to check a file's state *before* attempting access.

Mandatory Locking, introduced in System V Release 4 (SVR4), takes a stricter approach. Here, the system kernel itself actively prevents a process from accessing a file (or specific parts of it) if another process holds a conflicting lock. By default, mandatory locking is disabled in the system. To enable it, the filesystem must be mounted with the `mand` option (e.g., `mount -t <vfstype> <device> <dir> -o mand`). Additionally, the files intended to be locked must have their set-group-ID bit turned on and their group-execute bit turned off.

Despite its stricter guarantees, mandatory locking introduces severe problems. Because the kernel enforces the lock unconditionally, not even the `root` user can access a locked file. This creates a massive security vulnerability: a rogue process could lock a crucial system file, making it completely inaccessible and rendering the entire system unusable. Due to the fact that mandatory locks are considered "inelegant, buggy, and subject to races on multiple operating systems," and have remained flawed for decades, support for them was entirely removed from the Linux kernel starting with version 5.15.

7. Implementing Record Locks with `fcntl()`

The standard and most robust method to implement record locking in Unix and Linux operating systems is by utilizing the `fcntl()` system call. This requires importing both the `<unistd.h>` and `<fcntl.h>` headers.

The function prototype is defined as:

```
int fcntl(int fd, int cmd, struct flock *lock);
```

The function returns `0` upon success and `-1` if an error occurs. Beyond lock management, `fcntl()` is a versatile tool used for multiple operations on a file descriptor. The `cmd` argument dictates the function's specific behavior:

- **Duplicate an existing descriptor:** Using the `F_DUPFD` command.
- **Set or get file descriptor flags:** Such as managing the close-on-exec flag using `F_GETFD` or `F_SETFD`.
- **Set or get file status flags:** Such as dynamically changing `O_RDONLY`, `O_WRONLY`, `O_APPEND`, or `O_ASYNC` modes using `F_GETFL` or `F_SETFL`.

- **Record lock management:** Using the specific locking commands `F_GETLK`, `F_SETLK`, and `F_SETLKW`.

7.1 The flock Structure

When managing locks, the third argument to `fcntl()` must be a pointer to a `struct flock`. This structure explicitly defines the precise nature, location, and scope of the requested lock. The structure contains the following key fields:

- **short l_type:** This field specifies the type of lock. It can be set to `F_RDLCK` for a read (shared) lock, `F_WRLCK` for a write (exclusive) lock, or `F_UNLCK` to explicitly release an existing lock. Note that to acquire a read lock, the file must initially be opened for reading or reading/writing; to acquire a write lock, the file must be opened for writing or reading/writing.
- **short l_whence:** This acts as the starting reference point for the lock's byte offset. It accepts standard standard positional flags: `SEEK_SET` (the start of the file), `SEEK_CUR` (the current position in the file), or `SEEK_END` (the absolute end of the file).
- **off_t l_start:** This defines the starting offset for the lock, representing a number of bytes relative to the position specified by `l_whence`. The offset is flexible and can be a negative value if needed to lock backwards from the end or current position.
- **off_t l_len:** This dictates the total number of bytes to lock. It must be a non-negative value. A critical special case exists: if `l_len` is explicitly set to `0`, the system will lock all bytes starting from the calculated location (`l_whence` plus `l_start`) continuously through to the end of the file (EOF), regardless of how much new data is appended to the file later.
- **pid_t l_pid:** This field holds the Process ID (PID) of the process that currently owns a conflicting lock. This field is only populated and relevant when using the `F_GETLK` command.

7.2 Lock Management Commands

To interact with the lock described by the `flock` structure, one of three commands is passed to `fcntl()`:

1. **F_SETLK (Non-blocking):** This command is used to request a lock (if `l_type` is `F_RDLCK` or `F_WRLCK`) or release a lock (if `l_type` is `F_UNLCK`). The lock or unlock operation strictly affects the byte sequence specified by the `l_whence`, `l_start`, and `l_len` fields. If the lock request cannot be

immediately satisfied because it conflicts with a lock held by another process, `fcntl()` will not block; instead, it returns `-1` immediately. The `errno` is then set to either `EACCES` or `EAGAIN`. Because the exact error code can vary across different Unix/Linux implementations, POSIX standards mandate that portable applications must explicitly check for both errors.

2. **F_SETLKW (Blocking):** This is the blocking counterpart to `F_SETLK` (the 'W' denotes Wait). If the requested lock cannot be granted due to a conflict, the calling process is suspended and blocked until the conflicting lock is released and the new lock is acquired. If the suspended process happens to catch a signal during this wait, the call is interrupted. After handling the signal, the function returns `-1` and `errno` takes on the value `EINTR`.
3. **F_GETLK (Test Lock):** This command checks whether a specific lock request could theoretically be granted, without actually asserting the lock. If the lock *can* be granted (no conflicts exist), the system simply modifies the `flock` structure by setting `l_type` to `F_UNLCK` while leaving the rest of the structure unchanged. Crucially, the process does not acquire the lock. Conversely, if the lock *cannot* be granted, `fcntl()` populates the `l_pid` field of the provided structure with the PID of the exact process holding the conflicting lock.

7.3 Releasing Locks Automatically vs. Manually

Locks are automatically released by the operating system under two conditions: when the process terminates, or when the process closes the specific file descriptor associated with the lock. Despite this automated cleanup, it is considered excellent programming practice to manually and explicitly release all obtained locks (by calling `F_SETLK` with `F_UNLCK`) as soon as the critical I/O operations are completed.

8. Practical Examples of Record Locking

The pseudocode logic for interacting with locks is straightforward: submit a blocking lock request; if granted, perform the required read or write operations on the file, and immediately release the lock. Below are three common implementations.

Example 1: Read lock for the entire file (Blocking Mode) To read an entire file safely without interference from writers, a process configures a read lock (`F_RDLCK`) starting from the very beginning (`SEEK_SET`, offset 0) and stretching to EOF (length 0). It waits for the lock using `F_SETLKW`.

```

struct flock lock;
lock.l_type = F_RDLCK;      // Read Lock
lock.l_whence = SEEK_SET;   // Start of the file
lock.l_start = 0;           // Offset at start of the file
lock.l_len = 0;             // Lock to EOF

// Lock file descriptor fd (blocking mode)
if (!fcntl(fd, F_SETLKW, &lock)) {
    // Read operations on file descriptor fd ...

    // Release the lock
    lock.l_type = F_UNLCK; // Unlock
    if (fcntl(fd, F_SETLK, &lock) == -1) { /* handle error */ }
}

```

Example 2: Write lock on the last 10 bytes (Blocking Mode) If a process needs to append or modify just the end of a file, it requests a write lock (`F_WRLCK`). The reference point is the end of the file (`SEEK_END`), so it applies a negative offset (`-10`) and sets the length to 0 to lock through the EOF.

```

struct flock lock;
lock.l_type = F_WRLCK;      // Write Lock
lock.l_whence = SEEK_END;   // End of the file
lock.l_start = -10;         // Offset starting 10 bytes before the
EOF
lock.l_len = 0;             // Lock to EOF

if (!fcntl(fd, F_SETLKW, &lock)) {
    // Write operations on the locked region ...
    lock.l_type = F_UNLCK;
    if (fcntl(fd, F_SETLK, &lock) == -1) { /* handle error */ }
}

```

Example 3: Write lock on the next 10 bytes (Non-Blocking Mode) To lock a specific upcoming chunk without pausing the program if it fails, a process sets `F_WRLCK` starting from the current offset (`SEEK_CUR` , offset 0) with a definitive length of 10 bytes. It uses `F_SETLK` to fail instantly if the region is currently locked by another process.

```

struct flock lock;
lock.l_type = F_WRLCK;      // Write Lock
lock.l_whence = SEEK_CUR;   // From current file offset
lock.l_start = 0;           // 0 bytes offset
lock.l_len = 10;            // Lock next 10 bytes

// Lock file descriptor fd (non-blocking mode)
if (!fcntl(fd, F_SETLK, &lock)) {
    // Write operations on the locked region ...
    lock.l_type = F_UNLCK;
    if (fcntl(fd, F_SETLK, &lock) == -1) { /* handle error */ }
}

```

9. Advanced Considerations and Alternative Functions

9.1 Lock Inheritance and Lifecycle

It is critical to understand how file locks behave when processes spawn or transform. Locks are **not** inherited by a child process created using the `fork()` system call. However, locks **are** preserved when a process replaces its core image using an `execve()` call.

There is an important exception regarding `execve()` : if the file descriptor has the close-on-exec flag explicitly set to true, the system will close the file during the execution of `execve()` , which consequently releases any associated locks.

9.2 The Standard I/O Buffer Pitfall

Developers must avoid calling standard I/O library functions (like those found in `stdio.h`) to read or write to locked regions of files. Because standard I/O functions utilize user-space buffers, they can easily cause a program to read stale data or write data out-of-sync with the strict timing of kernel-level file locks.

If standard I/O must be used, developers have a few alternatives to ensure data integrity:

- Aggressively flush the user-space buffers using `fflush()` before releasing the lock.

- Completely remove buffering on the stream by calling `setvbuf(..., NULL, _IONBF, 0)`.
- Ideally, avoid the `stdio` library entirely for critical locked regions and use direct low-level `read()` and `write()` system calls.

9.3 Simpler Lock Interfaces: `lockf()` and `flock()`

While `fcntl()` provides the most robust and complete toolset for record locking, there are two simpler alternative functions available:

1. **`lockf()`** : Defined in the POSIX.1-2001 standards, `lockf()` acts as a simplified interface to the underlying `fcntl()` machinery. It allows for the lock management of a region within a file, but strictly starting from the current file position. (More details can be found in `man 3 lockf`).
2. **`flock()`** : This function allows for lock management solely on the entire file. It does not support byte-range locking. (More details can be found in `man 2 flock`).

Both of these functions are considerably simpler to implement than `fcntl()`, but they are inherently less versatile and less complete.

CHAPTER 4: Memory Management and Syscalls

1. The Memory Layout of a Process

When a C program is compiled and loaded into memory for execution, it is not placed randomly; rather, it is systematically divided into several distinct, highly organized parts known as segments. Understanding this structural layout is foundational to mastering dynamic memory allocation.

At the lowest memory addresses sits the **Text Segment**. This area contains the actual machine code instructions that are executed by the CPU. Because this binary code does not change during execution, the memory pages making up the text segment are designed to be shareable. This means that if multiple instances of the same program are running, only a single copy of the code needs to exist in physical memory. To enforce security and stability, this segment is strictly marked as read-only, which prevents any accidental or malicious modifications to the instructions while the program is running.

Directly above the text segment are the data segments, which manage the program's global and static variables. The **Initialized Data Segment** (often simply referred to as the "data segment") stores variables that have been explicitly assigned an initial value within the program's source code. When the program is launched, the values for these variables are read directly from the executable file on disk by the `exec` system call.

In contrast, the **Uninitialized Data Segment** handles global and static variables that lack explicit initialization in the code. This segment is historically known as the **BSS** segment, an acronym for "block started by symbol". Instead of storing empty data in the executable file, the operating system's kernel intervenes right before the program begins execution to initialize all data within the BSS segment to either a numeric zero or a `NULL` pointer. A key observation regarding Linux executables is that only variables initialized to a non-zero value actually consume physical space in the executable file stored on disk. Variables destined for the BSS segment do not take up disk space, making the executable file more efficient.

Moving further up the memory address space, we encounter the **Heap**, the segment dedicated to dynamic memory. This is the memory space that a program explicitly requests during its execution for data structures whose size might not be known at compile time. When memory is requested and allocated in the heap, the overall memory space of the process actively grows, a dynamic behavior that can be monitored using system tools like the `ps` command. The heap grows structurally "upward," meaning each new dynamic memory allocation is assigned a range of memory mapped to numerically larger addresses. While it is theoretically possible to return released heap memory to the operating system—thereby shrinking the process's total memory footprint—this is almost never actually implemented in modern memory allocators. Instead, released memory is kept in a "malloc pool" for reuse by future allocations.

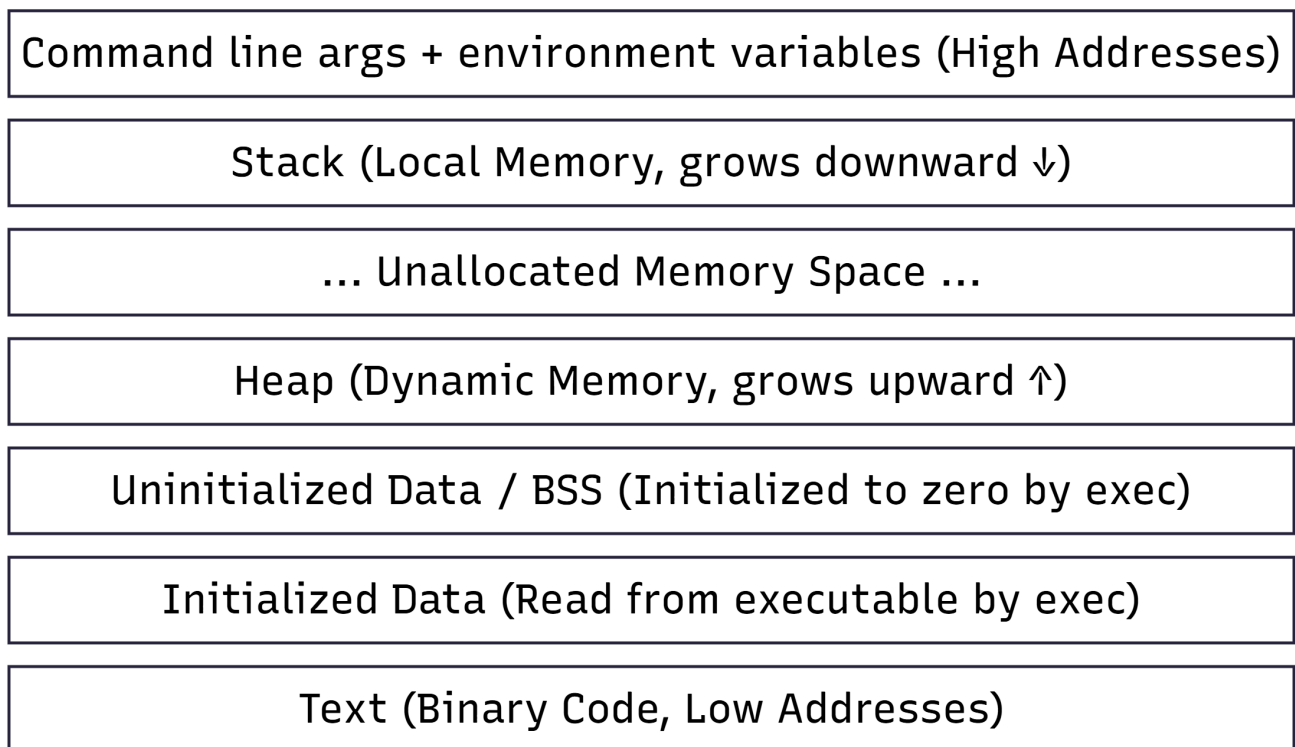
At the very top of the memory space, located at the highest addresses, is the **Stack**. The stack is a localized memory region responsible for managing automatic (or local) variables, as well as maintaining the execution context of the program. Whenever a function is called, the compiler automatically allocates space on the stack for several critical pieces of data: the parameters passed to the called function, the local variables used within that function, the function's eventual return value, and the specific memory address indicating where the CPU should resume execution once the function completes. This meticulous saving of state on the stack is the specific mechanism that allows for the execution of recursive function calls. Because stack memory is intrinsically tied to the scope of a

function, variables residing on the stack disappear entirely as soon as the containing function returns control to its caller. The freed stack space is then immediately available to be reused for subsequent function calls.

Unlike the heap, the stack typically grows "downward" toward numerically smaller addresses. Although this opposing growth theoretically means the stack and heap could eventually expand so much that they collide or overlap in the middle of the address space, the operating system kernel actively prevents this situation from occurring. Finally, situated above the stack at the absolute highest bounds of the process's address space, the system stores the command line arguments and environment variables passed to the program.

[Photo: Memory Layout of a Process]

Description: A structural diagram illustrating the memory space of a process, progressing from Low Addresses at the very bottom to High Addresses at the very top. At the lowest addresses sits the "text" segment containing binary code. Above it is the "initialized data" segment, which is read directly from the executable by exec. Next is the "uninitialized data (bss)" segment, which is initialized to zero by exec. Above the bss is the "heap", representing dynamic memory and featuring an arrow pointing upward to indicate it grows toward higher addresses. A large, unallocated memory gap (shaded in blue with dashed borders) separates the heap from the "stack" above it. The stack represents local memory and features an arrow pointing downward, indicating it grows toward lower addresses. Finally, at the highest addresses above the stack, lies the block for "Command line args + environment variables".



2. Inspecting Memory Footprints with the `size` Command

To understand how a program's segments translate to actual physical data, developers can utilize the Unix `size` command. This utility explicitly reports the byte sizes of the text, data, and bss segments contained within an executable file.

Consider an example executable file named `myprog`. By using the `ls -l` command, a developer might see that `myprog` has a total file size on disk of 12,320 bytes. However, running the command `size myprog` might output the following detailed breakdown in decimal (`dec`) and hexadecimal (`hex`) formats: 1,458 bytes for text, 276 bytes for data, and 8 bytes for bss, totaling 1,742 bytes.

There is an immediately apparent discrepancy here between the 12,320 bytes the file occupies on disk and the 1,742 bytes it actually requires when structurally loaded into memory. This massive size difference is largely attributed to the presence of the symbol table within the executable—a comprehensive list of the program's variable names and function names used strictly for debugging purposes.

To optimize disk space, developers can execute the `strip myprog` command. This command completely removes these debugging symbols from the executable file. Running `ls -l` again after stripping might reveal the file size has drastically dropped to 3,480 bytes. While this saves considerable disk space, it comes at the steep cost of making the program virtually impossible to debug.

Interestingly, even after stripping all symbols, the physical file size (3,480 bytes) remains larger than the actual loaded memory footprint reported by `size` (which remains static at 1,742 bytes). This is because the executable file must continue to retain additional, necessary metadata to function properly, such as explicit instructions regarding which shared libraries the program needs to use and link dynamically during runtime.

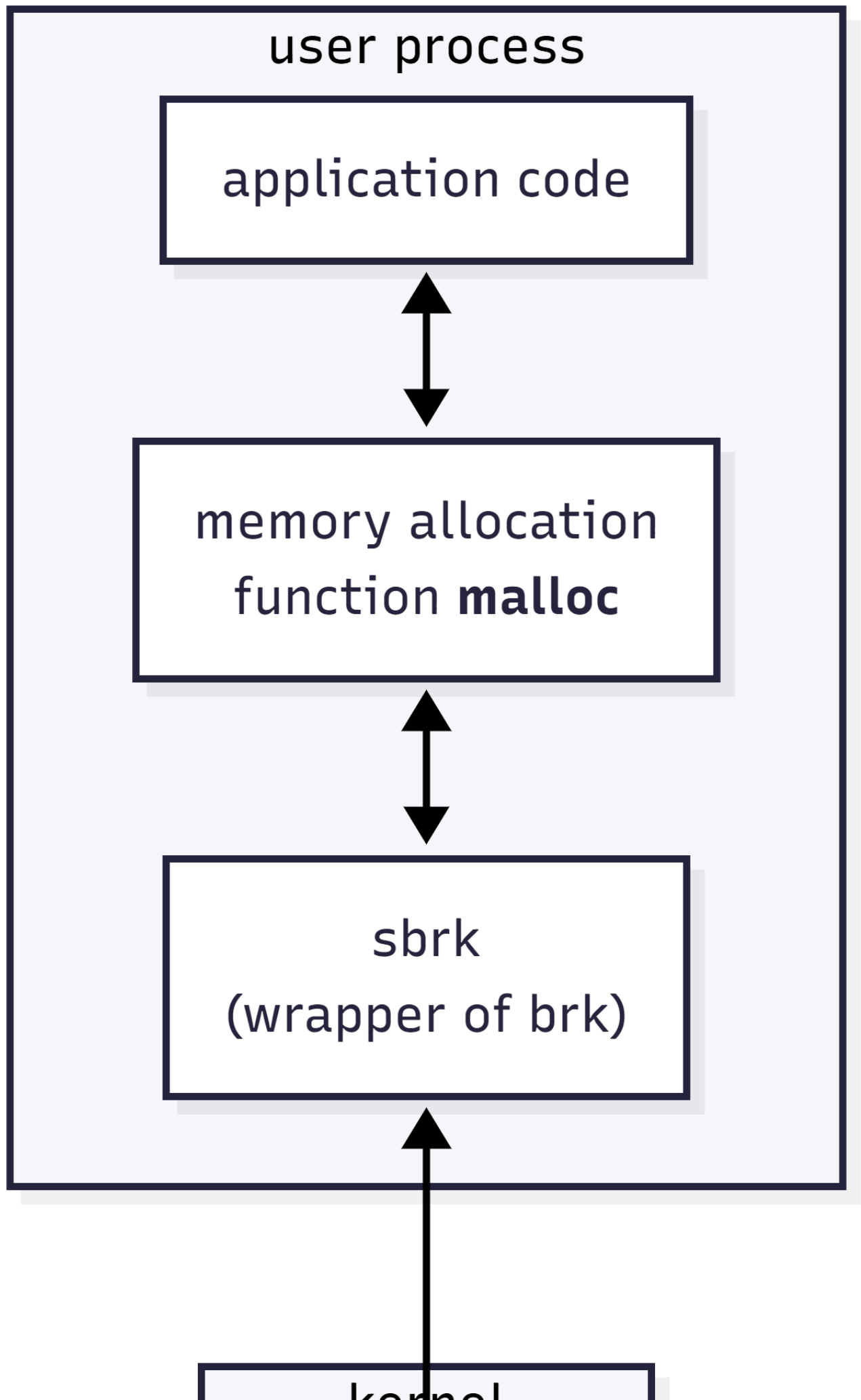
3. Memory Allocation Basics and Underlying Mechanisms

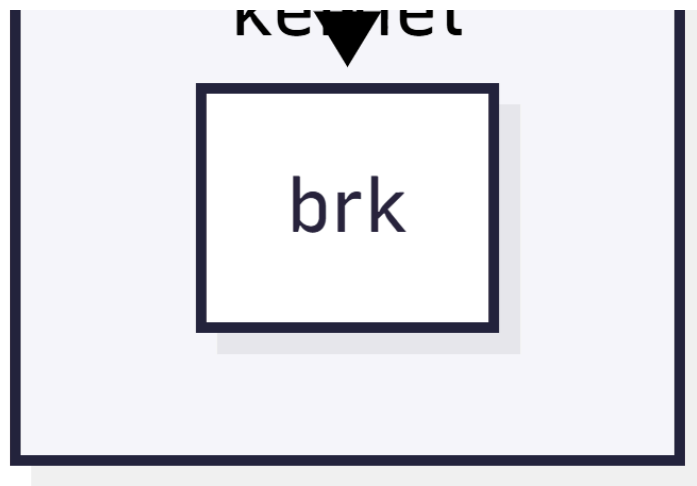
The ANSI C standard (via the standard C library) specifies three primary functions for dynamic memory allocation: `malloc`, `calloc`, and `realloc`. It is a critical distinction that these are library functions rather than system calls. Under the hood, these standard functions are implemented using the underlying `sbrk()` system call.

In practical application development, `sbrk()` is generally not called directly by the programmer. This is primarily because `sbrk()` is not designed as a general-purpose memory allocation function; its sole, highly specific job is to incrementally increase or decrease the memory space associated with a process, specifically modifying the bounds of the heap segment. Standard memory allocation functions act as sophisticated managers on top of this primitive mechanism. In fact, many standard implementations will allocate slightly more memory than the program explicitly requests. This extra space is used to maintain vital bookkeeping information hidden from the programmer, such as the exact size of the allocated block and a pointer linking to the next available block in memory. Because `malloc()` is a library function, it is entirely possible for developers to replace the standard `malloc()` with a custom implementation tailored to specific performance or architectural needs.

[Photo: Relationship between malloc and (s)brk]

Description: A structural diagram illustrating the layered relationship between the user process space and the operating system kernel. In the top "user process" box, the "application code" communicates bidirectionally with the standard "memory allocation function malloc". In turn, malloc communicates bidirectionally with sbrk, which acts as a user-space wrapper. The sbrk wrapper then crosses the boundary into the "kernel" box below, communicating directly with the brk system call to manage physical memory.





4. The Core Allocation Functions

The library calls for dynamic memory management offer different behaviors regarding initialization and sizing.

The most common function is `void *malloc(size_t size);`, which allocates a contiguous memory space of exactly `size` bytes. A critical characteristic of `malloc` is that the allocated memory is explicitly NOT initialized; its contents remain indeterminate and may contain leftover "garbage" data from previous operations.

Alternatively, `void *calloc(size_t nmemb, size_t size);` allocates memory specifically intended for an array of `nmemb` items, where each item occupies `size` bytes. Unlike `malloc`, the memory block returned by `calloc` is automatically initialized to 0 across every byte, ensuring a clean state for the new data.

To visualize how this interacts with the process layout, consider a simple program declaring `size_t mySize = 1024;` and a pointer `void *myPtr;` within the `main()` function. When `myPtr = malloc(mySize);` executes, a newly allocated block of 1024 bytes is created directly inside the heap segment. Meanwhile, the variable `myPtr` itself resides on the stack as a local variable. This stack-based pointer holds the memory address that points directly downward into the newly reserved space in the heap.

When a program's memory needs change during runtime, developers rely on `void *realloc(void *ptr, size_t newsize);`. This function allows for the modification—either increasing or decreasing—of a previously allocated memory area. In the scenario of memory growth, `realloc()` first checks if there is sufficient contiguous unallocated space immediately following the end of the existing memory region. If there is, `realloc()` simply expands the allocation into

that additional space and returns the exact same `ptr` that was originally passed to it.

However, if there is not enough adjacent space, `realloc()` must allocate an entirely new memory area of the requested size. It then automatically copies the existing contents addressed by the old pointer into the newly allocated block, safely releases the old area using `free()`, and finally returns a brand-new pointer directing to the new area. Programmers must exercise extreme caution here: because the memory area could physically move during this process, any other pointer variables that referenced the old area before the `realloc` call could become dangerous "dangling" pointers.

`realloc` also possesses specific edge-case behaviors: calling `realloc(NULL, newsize)` operates identically to `malloc(newsize)`. Conversely, calling `realloc(ptr, 0)` behaves exactly like calling `free(ptr)`. Interestingly, a pointer returned by a seemingly useless call to `malloc(0)` is legally valid and can be subsequently passed into `realloc()` later in the program.

5. Memory Release and Pool Management

When a developer no longer intends to refer to an allocated memory segment, it is absolutely imperative that the memory is released back to the system using the `void free(void *ptr);` function.

As previously noted, when memory is freed, it is not actually returned to the underlying operating system kernel. Therefore, the total memory footprint of the process does not decrease. Instead, the released memory is retained in an internal, available memory pool. This optimization ensures that the space can be rapidly reused for future allocation requests triggered by subsequent calls to the `*alloc` family of functions.

Failing to properly manage this lifecycle has severe consequences. If a process repeatedly allocates memory via `malloc()` but neglects to call `free()` on blocks it no longer needs, the process's memory usage will continuously grow. This structural flaw is widely known as a "memory leak," leading to an uncontrolled increase in memory occupation. Left unchecked, a memory leak will eventually exhaust available resources, potentially resulting in the forcible termination of the process by the kernel.

6. Critical Implementation Notes

All of the aforementioned allocation functions return a value strictly typed as `void*`, which acts as a generic, typeless pointer. In C programming, this returned pointer is typically converted into a typed pointer using the cast operator (e.g., `<type>`).

The parameter type `size_t` utilized by these functions is an unsigned integral type specifically intended to represent an amount of memory. While modern systems typically define `size_t` as an `unsigned long`, it is highly recommended to strictly use the `size_t` alias to guarantee code portability across different hardware architectures. Furthermore, it is a standard idiom to use the C `sizeof()` operator in combination with memory allocation calls to precisely determine the size in bytes of an area that will contain an array of complex elements. An example of this syntax is: `myPtr = (struct myStruct*) malloc(nelem * sizeof(struct myStruct));`.

It is guaranteed that both `calloc()` and `malloc()` will return a pointer possessing an in-memory alignment compatible with any type of variable. If the system is entirely out of memory and the call fails, they will safely return `NULL`. In the event that `realloc()` fails to find space, the original memory area is strictly preserved; it is not modified, released, or moved.

Serious application errors that cause forced interruptions of the program during memory allocation or release calls are mostly attributed to "heap corruption". The two most common causes of heap corruption are overflowing the strict limits of the allocated memory (such as writing data beyond the last allocated byte) and attempting to release the exact same pointer more than once (a double-free error). To assist developers in catching these flaws, recent versions of the GNU `libc` include an implementation of `malloc()` that can detect and report memory leaks. This diagnostic mode is enabled by setting the `MALLOC_CHECK_` environment variable, with further details available within the `malloc` manual pages.

Finally, there is a special library call named `void *alloca(size_t size);`. While `alloca()` is functionally similar to `malloc()` in that it reserves memory, the critical distinction is that the allocated memory space resides directly on the stack instead of the heap. Consequently, this memory is automatically released the moment the function that called `alloca()` returns to its caller. However, its usage is heavily discouraged; developers should consult the manual pages to check for actual usability and potential severe malfunctions associated with this function.

7. Practical Code Examples: Allocation and Initialization

To fully grasp the theoretical concepts of memory allocation, it is essential to observe how these library calls are structured in practical C code. Consider a scenario where a developer needs to manage a collection of 3D coordinates. The data structure is defined as `struct coord { int x, y, z; } .`

To allocate an array of these structures dynamically, the program must first determine the total amount of memory required. This is achieved by multiplying the desired number of elements (`count`) by the memory footprint of a single element, calculated using `amount = count * sizeof(struct coord) .` The developer then requests this space by invoking `malloc(amount)` and casting the returned `void*` pointer to the appropriate type: `coordinates = (struct coord *) malloc(amount); .`

Immediately following the allocation, robust software must verify that the allocation succeeded by checking if the pointer is `NULL` . If it is `NULL` , the program must report the error, attempt to recover, or gracefully give up and terminate. Once the memory is successfully allocated and referenced by the pointer, the C compiler allows the programmer to treat this pointer exactly as if it were a standard array. For example, a developer can access and modify the `x` , `y` , and `z` fields of a specific element using standard array indexing syntax, such as `coordinates[an_index].x` . Behind the scenes, the compiler generates the precise machine code required to perform pointer arithmetic, calculating the correct memory address for the array element and its internal structure members.

Because `malloc` does not initialize the memory it returns, developers must manually clear the data if a blank slate is required. This can be accomplished by passing the pointer and the total allocation amount to `memset(coordinates, 0, amount)` or the older `bzero(coordinates, amount)` function. Alternatively, the developer could bypass manual initialization entirely by substituting `malloc` with a call to `calloc()` , which zeroes the memory automatically.

When developers are finished utilizing the memory, it must be returned to the memory pool by calling `free(coordinates); .` While not strictly required by the language, a highly recommended best practice is to immediately assign `coordinates = NULL;` following the `free` call. This neutralizes the pointer, ensuring that subsequent accidental uses of the variable will result in a predictable null-pointer dereference rather than silently corrupting active memory.

Furthermore, an advanced approach that guarantees proper memory sizing—even if the pointer's fundamental declaration is altered during future code maintenance—is to size the allocation based on the dereferenced pointer itself rather than an explicitly named type. This elegant idiom is written as: `pointer = malloc(count * sizeof(*pointer));`.

8. Dynamic Resizing and the Danger of Dangling Pointers

When an application's data requirements grow, developers must dynamically resize existing memory blocks using `realloc`. Suppose the previously allocated coordinate array needs to double in capacity. The program establishes a `new_count = count * 2` and calculates the `new_amount` in bytes.

A critical rule when using `realloc` is to never assign its return value directly back to the original pointer variable before verifying success. Instead, developers should capture the result in a temporary pointer, such as `struct coord *newcoords = (struct coord *) realloc(coordinates, new_amount);`. If `realloc` fails and returns `NULL`, assigning that directly to `coordinates` would overwrite the only reference to the original, still-valid data block, causing an immediate and unrecoverable memory leak.

Once the program confirms `newcoords` is not `NULL`, it is safe to permanently update the primary reference by executing `coordinates = newcoords;`.

However, a severe danger lurks here: if `realloc` was forced to move the memory block to a new location to accommodate the larger size, any auxiliary pointers that were previously referencing the old memory addresses instantly become invalid "dangling references".

9. Pointer Caching vs. Array Indexing

The peril of dangling references is best illustrated by a structural flaw involving pointer caching. Consider a function designed to manage a dynamically expanding table. A developer might allocate memory for this table and then create an auxiliary pointer, `cur`, designed to point directly at a specific item within the array using the syntax `cur = &table[i];`.

The problem arises when the program later determines that the table needs to grow, triggering a call to `realloc`. If `realloc` moves the entire table to a new location in the heap, the original block is freed. Unfortunately, the auxiliary pointer `cur` is entirely unaware of this relocation and continues to point to the

obsolete, freed memory address. If the developer subsequently attempts to modify data through this dangling pointer—for example, by executing `cur->field = j;`—the program will corrupt memory, leading to unpredictable crashes.

Additionally, if this routine is recursive and calls another function that modifies the table's state globally, the pointers can easily lose synchronization.

The definitive solution to this problem is to abandon caching direct pointers to elements within dynamically resizable structures. Instead, developers should strictly update members using array indexing based on the base pointer, such as `table[i].field = j;`. Because the base pointer `table` is correctly updated immediately after the `realloc` call, dynamically indexing `table[i]` guarantees that the program accesses the correct location within the newly allocated memory block, causing the dangling pointer problem to disappear entirely.

10. Synthesizing Standard Functions

Understanding how these functions interoperate allows developers to build them from scratch. A trivial implementation of the `calloc` function perfectly demonstrates this relationship.

To build a custom `calloc(size_t nmemb, size_t size)`, a developer merely needs to replicate its two core behaviors: allocation and zero-initialization. First, the total required memory is computed via `total = nmemb * size;`. Next, the underlying `malloc(total)` function is invoked to secure the memory space, returning a pointer `p`. Crucially, the implementation must check `if (p != NULL)` to confirm the allocation succeeded. If successful, the `memset(p, '\0', total);` function is applied to forcefully fill the entire allocated block with zeros. Finally, the function returns `p`, providing the caller with a perfectly initialized memory block—or `NULL` if the initial `malloc` failed.

11. GNU Coding Standards for Memory Management

To ensure robust and predictable software behavior, Richard Stallman outlined specific, rigorous guidelines for memory allocation within the GNU Coding Standards.

First and foremost, developers must meticulously check every single call to `malloc` or `realloc` to explicitly see if it returned a zero (a `NULL` pointer), indicating an allocation failure.

Second, developers must check the return value of `realloc` even if they are attempting to make an existing memory block smaller. Because some underlying systems round memory block sizes up to a power of 2, a request for less space might cause the system to relocate the data into an entirely different block that fits the new, smaller size profile more efficiently.

Third, developers must be aware of historical flaws in standard library implementations. On standard Unix systems, `realloc` contains a dangerous bug: if the system call fails and returns zero, it can inadvertently destroy the original storage block in the process. The GNU implementation of `realloc` specifically corrects this bug; if GNU `realloc` fails, it guarantees that the original memory block remains completely unchanged and valid.

Finally, regarding memory release, developers must operate under the strict assumption that calling `free` will actively alter and destroy the contents of the freed memory block. Therefore, any critical data that a program needs to fetch or salvage from a dynamically allocated block must be retrieved strictly *before* executing the `free` call.

CHAPTER 5: Fork, Exit & Wait

Summary of Process-Related Library Calls

To manage processes effectively within a computer system, developers rely on a specific set of standard library calls. These functions form the backbone of process manipulation and synchronization:

- **Attribute Retrieval:** The functions `getpid()`, `getppid()`, and `getgid()` are utilized to provide essential process attributes, such as the Process Identifier (PID), the Parent Process Identifier (PPID), and group affiliations.
- **Process Creation:** The `fork()` system call is responsible for creating a new child process. It achieves this by "cloning" the memory and state of the calling process.
- **Program Execution:** The `exec()` family of functions is used to replace the current process's memory space, actively loading an entirely new program into memory for execution.
- **Synchronization:** The `wait()` function allows for essential synchronization between processes, typically ensuring a parent process pauses until its child has completed its execution.

- **Termination:** Finally, the `exit()` system call is used to formally end a running process.

The Fundamentals of Process Lifecycle

Process Creation

In modern operating systems, processes do not emerge spontaneously; a process is always spawned by another pre-existing process, which assumes the role of the "parent". The parent process can originate from various levels of the system architecture. It could be the operating system itself—specifically via the `init` or `systemd` process—which spawns processes to provide system facilities such as printing services. Alternatively, the parent might be a background service (daemon) like a job scheduler or `getty`, which is strictly responsible for managing user logon sessions on terminals. Lastly, any standard user process has the capability to act as a parent by spawning another user process.

Process Termination

When a process completes its lifecycle, it undergoes termination, which can occur under two primary circumstances. A process can terminate voluntarily, which typically happens when the running process explicitly executes the `exit()` command. Conversely, a process can be involuntarily killed by the system itself. This forced termination usually occurs when the system encounters unrecoverable error and fault conditions caused by the process.

Process States

To thoroughly understand process execution, one must understand the various states a process transitions through during its lifespan, all of which are continuously maintained by the operating system's kernel.

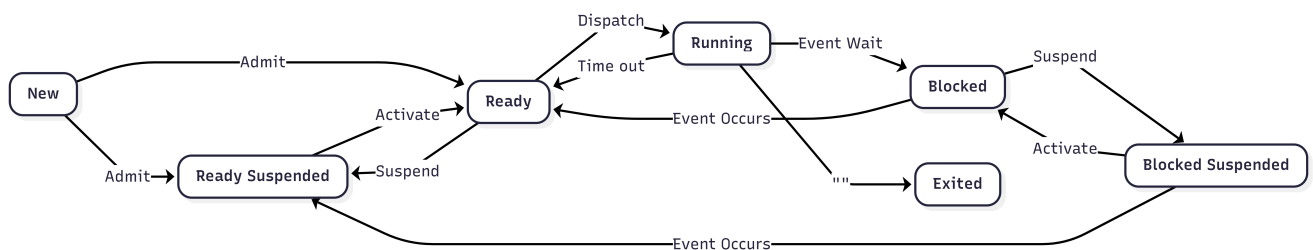
[Photo: Process States Diagram]

The structural lifecycle of a process involves several interconnected states. When a process is initially created, it exists in the **New** state. The kernel must then "Admit" the process, transitioning it into either the **Ready** state or the **Ready Suspended** state.

Once in the active **Ready** state, the kernel's scheduler can "Dispatch" the process, moving it into the **Running** state where its instructions are actually executed by the CPU. From the **Running** state, three things can happen:

1. The process finishes its task and moves to the **Exited** state.
2. The process is interrupted by a "Time out" and forced back into the **Ready** state to await its next turn on the CPU.
3. The process needs to wait for an external event (like I/O input), triggering an "Event Wait" that transitions it into the **Blocked** state.

When a process is in the **Blocked** state, it remains there until the required "Event Occurs", at which point it moves back to the **Ready** state. Furthermore, memory management may force processes out of main memory, causing state suspensions. A **Ready** process can be "Suspended" into the **Ready Suspended** state, and a **Blocked** process can be "Suspended" into the **Blocked Suspended** state. These suspended processes require an "Activate" signal to return to their normal Ready or Blocked counterparts. A process can also transition directly from **Blocked Suspended** to **Ready Suspended** if the event it was waiting for occurs while it is in suspension.



Process Identifiers (PID)

To uniquely track every active process, the operating system utilizes Process Identifiers. Each process is assigned a unique identifier, known as a PID, which is strictly a non-negative integer. In most programming environments, the PID is implemented as an unsigned number belonging to the specific data type `pid_t`. The kernel assigns these PIDs sequentially every single time a new process is created.

Alongside its own PID, every process maintains a record of its lineage through an associated Parent Process Identifier (PPID). The PPID is simply the precise PID of the parent process that generated or spawned the current process.

Certain PIDs are heavily restricted and pre-assigned by the system. PID `0` is either completely unassigned or specifically reserved and assigned to a low-level kernel process. PID `1` is exclusively reserved for `init` (or `systemd` in newer Linux distributions). The `init` process is created directly by the kernel at the very end of the system bootstrap procedure and is singularly responsible for

performing the actions necessary to bring the entire operating system up to a usable, multiuser state.

Process Identifier Functions in C

To retrieve these identifiers programmatically in the C language, developers use the standard POSIX functions provided in the `<unistd.h>` header file. The function `pid_t getpid();` is used to return the PID of the calling process, while `pid_t getppid();` returns the PID of the calling process's parent.

Here is a practical C implementation demonstrating how to fetch and print these values:

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("The process PID is %d.\n", (int)getpid());
    printf("The parent process PID is %d.\n", (int)getppid());
    return(0);
}
```

User and Group Identifiers: Managing Process Privileges

Beyond process identifiers, every process in the operating system is strictly associated with specific user identifiers (user-IDs) and group identifiers (group-IDs). These identifiers are crucial because they determine the process's privileges, dictating exactly which system calls the process has the right to invoke and which system resources (like files or hardware) it is permitted to access.

To manage security and access control effectively, the system maintains at least six distinct identifiers for every process (or more, if supplementary group IDs are considered):

- **Real User-ID and Real Group-ID:** These identifiers represent the actual user who launched the process and the primary group associated with that user. Crucially, the real user-id and real group-id values generally do not

change throughout the entire duration of a login session; only the superuser (root) possesses the power to modify them.

- **Effective User-ID, Effective Group-ID, and Supplementary Group-IDs:** These are the identifiers that the kernel actually considers when determining file access privileges during execution. While in most cases the effective user-id and effective group-id perfectly coincide with the real user-id and real group-id, they can differ. This divergence happens if the executable file being run has its *set-user-id* or *set-group-id* permission bit set to active.
- **Saved Set-User-ID and Saved Set-Group-ID:** These identifiers are essentially backup copies. They are saved by the system during the `exec()` system call and represent the exact values of the effective user-ID and effective group-ID immediately after an `exec()` is performed on an executable file that has its set-user-id or set-group-id bits active.

The Set-User-ID and Set-Group-ID Mechanism

Every file in the system (including executable program files) has an owner and a group owner. If an executable program file has the special permission bit known as *set-user-id* (or *set-group-id*) active, a privilege escalation can occur. When such a program is invoked using an `exec()` call, the process's effective user-id (or effective group-id) temporarily changes to become that of the *owner of the file*, rather than remaining as the user who actually launched it.

A classic example of this is the system's `passwd` utility, which allows users to change their account passwords. Modifying passwords requires writing to secure system files that standard users cannot normally access. However, because the `passwd` program file belongs to the superuser (`root`) and has the *set-user-ID* bit enabled, any standard user who launches this program temporarily gains superuser privileges strictly during the program's execution. While powerful, this mechanism poses a significant security danger if the elevated program contains vulnerabilities.

Examining the `passwd` executable in a terminal reveals this special bit:

```
$ ls -l /usr/bin/passwd
```

```
-rws--x--x 1 root root 19336 Sep 7 2004 /usr/bin/passwd
```

(Note the 's' in the permission string, indicating the active set-user-ID bit).

Modifying and Retrieving Process Attributes

Process IDs can be programmatically changed using specific system calls, primarily `int setuid(uid_t uid);` and `int setgid(gid_t gid);`, which set the real and effective user/group IDs. However, the operating system enforces strict rules to control whether a program is allowed to change these IDs:

1. **Superuser Privilege:** If the process's current effective-uid is `0` (meaning it has superuser privileges), then invoking `setuid()` will forcefully change the real-uid, the effective-uid, and the saved-set-uid all to the requested `uid`.
2. **Unprivileged User:** If the process does not have superuser privileges, the change is highly restricted. The system will only allow `setuid()` to change the effective-uid, and only if the requested `uid` is exactly equal to either the process's current real-uid or its saved-set-uid.
3. **Failure:** If neither of the above conditions is met, the system call completely fails, an error is returned, and the global error variable `errno` is set to `EPERM` (Operation not permitted). The exact same rules apply to group IDs using `setgid()`.

A developer has access to a wide array of functions to interact with these attributes:

- **Retrieval:** `uid_t getuid();` (real user), `gid_t getgid();` (real group), `uid_t geteuid();` (effective user), `gid_t getegid();` (effective group). The POSIX standard also provides `getresuid()` and `getresgid()` to retrieve the real, effective, and saved IDs simultaneously via pointers.
- **Setting:** Alongside `setuid()` and `setgid()`, there are functions to set exclusively the effective IDs (`seteuid()`, `setegid()`) and functions to explicitly set all three IDs at once (`setresuid()`, `setresgid()`).

Here is a practical C program demonstrating how to retrieve and print these identifiers:

```
#include <unistd.h>
#include <stdio.h>

int main() {
    uid_t uid, euid;
    gid_t gid, egid;

    uid = getuid();
```

```
euid = geteuid();
gid = getgid();
egid = getegid();

printf("real uid: %d, effective uid: %d\n", (int)uid,
(int)euid);
printf("real gid: %d, effective gid: %d\n", (int)gid,
(int)egid);
}
```

Deep Dive: The `fork()` System Call

The `fork()` system call is the cornerstone of process creation in POSIX-compliant systems. Its function signature is straightforward: `pid_t fork();`.

When `fork()` is invoked, it creates an entirely new child process by deeply cloning the memory image of the parent process. This means the parent's data segment, heap, and stack are all copied over to the child. However, modern operating systems optimize this massive data transfer by exploiting a concept called **copy-on-write**. Instead of instantly duplicating all physical memory, the system simply points the child to the parent's memory pages. These memory pages are only physically copied if and when either the parent or the child attempts to modify them, vastly improving performance.

The most unique and sometimes confusing characteristic of `fork()` is its execution flow: **it is called once by the parent process, but it returns twice**. Immediately after the `fork()` operation completes, both the newly created child process and the original parent process continue executing the code starting at the very next instruction immediately following the `fork()` call.

Because they share identical code, the program must branch its logic based on the return value of `fork()` to differentiate the parent's tasks from the child's tasks. The `fork()` call returns:

- `0` to the newly created child process.
- The actual PID of the child process to the parent process.
- A negative value (usually `-1`) if an error occurs and the child process could not be created.

Execution Flow Example

Consider this simple C program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void main() {
    printf("One\n");
    fork();
    printf("Two\n");
}
```

If you compile and run this (`./a.out`), the output will be: `One Two Two`

Visualizing the Flow: Before the `fork()` call, only Process A exists (e.g., PID=3627, PPID=3282). Its Program Counter (PC) executes `printf("One\n")`. Then, Process A reaches the `fork()` instruction. At this exact moment, the kernel splits the execution, creating Process C (the child, e.g., PID=3630, PPID=3627) while Process B (the continuation of the parent) also moves forward. After the split, both Process B and Process C independently have their Program Counters pointing to the next instruction: `printf("Two\n")`. Consequently, "Two" is printed twice.

Here is a more robust example demonstrating how to properly handle the two return paths:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

void main() {
    pid_t pid;
    printf("I am process %d.\n", (int)getpid());
    printf("Call to fork...\n");

    pid = fork();

    if(pid == 0)
        printf("I am the child process (PID: %d).\n",
```

```
(int)getpid());  
    else if(pid > 0)  
        printf("I am the parent of the process with PID %d.\n",  
pid);  
    else  
        printf("An error occurred while calling fork.\n");  
}
```

Process Inheritance

Because the child is a clone of the parent, it inherits an extensive list of process properties. However, it does not inherit everything. Understanding this boundary is critical for systems programming.

Properties **INHERITED** from the parent process include:

- Real user ID, real group ID, effective user ID, and effective group ID.
- Additional group IDs and the process group ID.
- The session ID and the control terminal.
- The set-user-ID flag and set-group-ID flag.
- The current working directory and the root directory.
- The file creation mask (umask).
- The signal mask.
- The close-on-exec flag for all open descriptors (which is copied).
- File descriptors (which are actively shared between parent and child).
- Environment variables.
- Shared memory segments.
- Established limits on system resources.

Properties **NOT INHERITED** from the parent process include:

- The `fork()` return value (which inherently must be different to branch logic).
- The process ID (the child gets its own unique PID).
- The parent process ID (the child's PPID points to the parent, while the parent's PPID points to its own creator).
- Process accounting values (such as CPU time used, which resets to zero for the child).
- File locks (locks held by the parent are not passed to the child).

- Pending alarms (any alarms set by the parent are cancelled for the child).
- The set of pending signals (the child's pending signal queue is completely emptied upon creation).

Fork Errors, Use Cases, and Open Files Management

While `fork()` is highly efficient, it is not infallible. The system call will fail and return an error under specific resource constraints: notably, when the absolute maximum number of processes allowed in the system has been reached, or when the user executing the program has reached their specific maximum allowed number of concurrent processes.

Assuming successful execution, `fork()` is primarily utilized in two major use cases:

1. **Work Distribution:** A process deliberately duplicates itself so that the parent and child can execute completely different parts of the code. This is the standard architectural model for network servers and daemons. The parent process constantly remains in a loop, waiting for incoming requests from the network. When a request arrives, the parent forks, assigns the handling of that specific request to the new child, and immediately returns to its waiting state to accept new requests without being blocked by the processing time.
2. **Program Execution:** A process wants to run a completely different program. In this case, the process forks a child, and the child immediately calls an `exec()` function to replace its cloned memory space with the new program.

File Descriptor Inheritance and Management

One of the most critical aspects of process duplication involves how the system handles file inputs and outputs. When a `fork()` occurs, all file descriptors that are currently open in the parent process are perfectly duplicated in the child process.

To understand the implications of this, we must examine how the operating system tracks open files.

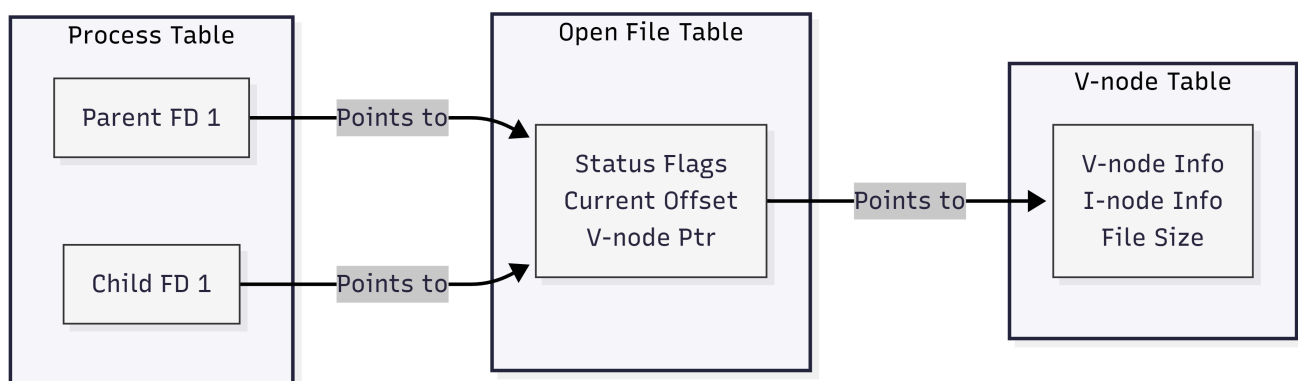
[Photo: Process, Open File, and V-node Tables Diagram]

Description: This diagram illustrates the memory structure linking processes to files before and after a `fork()`. The "Process Table" contains arrays of file

descriptors (0, 1, 2) for both the parent and the child. These descriptors hold pointers that lead to the "Open File Table". The Open File Table entries store the file's status flags, the current file offset (the byte position where the next read/write will occur), and a v-node pointer. Finally, the v-node pointer connects to the "V-node Table", which holds permanent system-wide data like v-node info, i-node info, and the total file size.

Before the `fork()`, the parent process has pointers mapping its file descriptors to specific entries in the central Open File Table. After the `fork()`, the newly created child process receives an exact copy of the parent's descriptor array.

Crucially, the child's file descriptors point to the exact same entries in the Open File Table as the parent's descriptors.



Because both parent and child processes share the same Open File Table entry, **they intrinsically share the file offset.**

The motivation for this architectural design becomes clear when evaluating common process flows. Suppose a process forks, and the parent explicitly uses `wait()` to pause until the child terminates. If the standard output (`stdout`) of this program is redirected to a file, both the parent and child will likely attempt to write to it. If the parent and child did not share the same offset, a severe data corruption issue would occur: the child would write to the file and update its own isolated offset, but when the parent subsequently resumed writing to `stdout`, it would use its old, un-updated offset, directly overwriting the child's output data.

Therefore, managing these inherited file descriptors depends entirely on the relationship between the processes:

- **Case 1: The parent process waits for the child.** In this synchronized scenario, the file descriptors are intentionally left unchanged. Any file offset changes made by the executing child process will naturally be reflected in the shared open file table entry, ensuring the parent picks up exactly where the child left off.

- **Case 2: Parent and child processes are independent.** If the processes branch off to perform entirely separate tasks concurrently, sharing the offset becomes dangerous. In this case, both processes must explicitly close the file descriptors they do not need to prevent unexpected interference, and then continue their independent execution.

(Self-Study Exercise: Write a C program that uses `open()` to open a file, `fork()`s, and uses `write()` to log different messages depending on if it is the parent or child. Observe how the messages append linearly. Then, modify the code to move the `open()` call to strictly after the `fork()`. You will see the contents of the file become corrupted or overwritten, precisely because the parent and child will now create separate open file table entries with unshared, competing offsets.)

Process Lifecycle: Termination and Synchronization

Termination Paths

The conclusion of a process's lifecycle can occur through two distinct categories of termination: normal and abnormal.

There are three ways for a process to end **NORMALLY**:

1. **Return:** Executing a standard `return` statement from the `main` function (which the C compiler intrinsically translates to an `exit()` call).
2. **The `exit()` function:** Explicitly calling `void exit(int status);`. As specified in ANSI C, this function performs high-level cleanup: it safely closes all standard I/O streams and invokes all custom exit handlers that the developer may have registered.
3. **The `_exit()` function:** Calling `void _exit(int status);`. Specified in the POSIX.1 standard, this function immediately returns control to the kernel, bypassing the high-level C library cleanups. It is inherently called as the final, absolute last operation by the standard `exit()` function to finalize the termination.

[Photo: Process Termination Flow Diagram]

Description: This flow chart demonstrates the termination hierarchy. User functions or the main function can execute a standard `return` or call `exit()`. The `exit()` function then loops through and executes all registered "exit handlers" and triggers standard I/O cleanup. Once these user-space cleanups finish, `exit()`

funnels down and calls `_exit()` . Alternatively, a user function can bypass all cleanup by calling `_exit()` directly. Regardless of the path, `_exit()` acts as the final boundary, passing the termination request directly into the operating system's kernel.

Conversely, there are two ways a process terminates **ABNORMALLY**:

1. **Uncaught Signals:** The process catches a system signal whose default, unhandled action is forced termination. This lethal signal could be generated internally by the process itself (e.g., a division by zero), externally by other processes, or directly by the kernel responding to a fault. (Note: Comprehensive details on signals can be found in the Linux manual via `man 7 signal`).
2. **The `abort()` function:** Explicitly calling `void abort();` . This function technically acts as a wrapper that forces the process to generate a `SIGABRT` signal directed at itself, effectively making it a subset of signal-based abnormal termination.

Regardless of whether the termination was normal or abnormal, the kernel executes strict administrative actions upon process death: it permanently removes the process's allocated memory segments and forcibly closes all remaining open file descriptors.

Exit Status vs. Termination Status

It is vital to distinguish between the two statuses generated at the end of a program's life:

- **Exit Status:** This is the specific integer value that a programmer explicitly passes as an argument to the `exit()` or `_exit()` functions in their code. It is a user-defined notification mechanism aimed at the parent process.
- **Termination Status:** This is the ultimate value generated and maintained by the kernel itself upon process death. In the event of a normal termination, the kernel takes the user-provided "exit status" and converts it into the final "termination status" right when `_exit` is called. In the event of an abnormal termination, the kernel generates a termination status that specifies the exact system reason (like the specific signal number) for the crash.

These definitive termination status values are stored by the kernel so they can be retrieved by the parent process using the `wait()` and `waitpid()` system calls.

Zombie Processes and `init` Reparenting

Because parents and children operate asynchronously, they rarely terminate at the exact same moment. The operating system must carefully manage these overlapping lifespans.

If the **parent process exits before its children**, the surviving children are left orphaned. The kernel continuously monitors the process table for this occurrence. If it finds orphaned children, it intervenes to maintain the system hierarchy by reparenting them: it sets the PPID of each orphaned child to `1` (assigning them to the supreme `init` or `systemd` process).

A more complex scenario arises if the **child process exits before the parent**. When a child dies, its termination status must be securely stored by the kernel so that the parent has the opportunity to retrieve it later using `wait()`. If the child ends, but the parent does not immediately "wait" for it (and does not actively handle the `SIGCHLD` signal), the child's memory footprint is wiped, but a skeleton of information is permanently kept in the kernel's memory. In this lingering state, the child process becomes a **zombie**.

While in the zombie state, the kernel maintains the dead process's PID, its termination status, and accounting information (such as the total CPU time taken by the process). The process will relentlessly remain a zombie until the parent finally executes a `wait` system call to collect this data.

If the parent of a zombie exits without ever waiting, the zombie is passed to `init` (PID 1). Fortunately, children inherited by `init` can never permanently remain zombies. The `init` process is uniquely designed to aggressively and repeatedly execute `wait()` calls whenever one of its adopted children terminates, instantly collecting the data and cleaning the zombie from the system.

Consider the following C code demonstrating zombie creation:

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/types.h>

void main() {
    pid_t pid;
    printf("I am process %d.\n", (int) getpid());
```

```

printf("Call to fork...\n");

for (int i = 0; i < 5; i++) {
    pid = fork();
    if (pid == 0) {
        printf("I am the child process (PID: %d).\n", (int)
getpid());
        exit(1); // Child dies immediately
    } else if (pid > 0) {
        printf("I am the parent of the process with PID
%d.\n", pid);
    } else {
        printf("An error occurred while calling fork.\n");
    }
}

pause(); // Parent pauses indefinitely, refusing to wait()
for its dead children
}

```

If we inspect the system processes (using a tool like `ps`) while this code runs, we see the parent (PID 8539) sleeping in state `S+`, while all five generated children (PIDs 8540-8544) are trapped in state `Z+`, explicitly marked as `<defunct>` by the system.

Synchronization: `wait()` and `waitpid()`

To retrieve termination statuses and prevent the accumulation of zombie processes, developers utilize the `wait` family of system calls:

- `pid_t wait(int *status);`
- `pid_t waitpid(pid_t pid, int *status, int options);`

When a process invokes `wait()` or `waitpid()`, the calling process immediately suspends execution (blocks) if all of its children are currently running. The function will return under two conditions:

1. It successfully retrieves a child's termination status (if a child has already terminated and its status is waiting in the kernel).
2. It returns an error (specifically, if the calling process does not have any existing children to wait for).

For both functions, the argument `status` is a pointer to an integer memory location. If this pointer is not `NULL`, the kernel places the retrieved termination status directly into this location. Upon success, the return value of the system call itself is the exact PID of the child whose status was collected. (Because the structure of the integer `status` is highly system-dependent, developers must use standardized C macros to extract specific bits, such as determining if it was a normal termination, and extracting the actual exit status value).

The Difference Between `wait` and `waitpid`: The standard `wait()` function is indiscriminate; it blocks the calling process until *any* one of its children finishes. The `waitpid()` function, however, provides highly granular control over which process to wait for and how to behave.

The behavior of `waitpid()` is dictated by its `pid` argument:

- `pid == -1`: The function behaves identically to standard `wait()`, accepting any terminating child.
- `pid > 0`: The function explicitly waits strictly for the child process whose ID exactly matches the requested `pid`.
- `pid == 0`: The function awaits the termination of any child process whose process group ID perfectly matches the process group ID of the caller.
- `pid < -1`: The function awaits the termination of any child whose process group ID is equal to the absolute value of the provided `pid` (i.e., `-pid`).

Furthermore, `waitpid()` accepts an `options` argument. Passing the macro `WNOHANG` as an option radically changes the function's behavior: instead of blocking and suspending the caller, it instructs `waitpid()` to return immediately with a status code of `0` if the specified child has not yet finished executing.

Asynchronous Notification: `SIGCHLD`

Polling for child completion via `wait()` can be inefficient. To solve this, the operating system utilizes an asynchronous notification system.

Whenever a process terminates—regardless of whether it died normally or abnormally—the kernel immediately generates and sends a `SIGCHLD` signal directly to the parent process. By default, the parent process is configured to completely ignore this signal. However, a robust program can explicitly set up a signal handler in the parent. This custom function is automatically and asynchronously invoked by the OS the very moment the `SIGCHLD` signal is received, allowing the parent to instantly execute a `wait()` call to reap the dead

child and clean up the zombie process without needing to block its primary execution flow beforehand.

(Self-Study Exercise: Write a complete program that uses `fork()` , blocks using `wait()` until the child successfully prints a message to the screen, and then prints its own parent message before gracefully exiting. Afterwards, modify the program to accept a command-line integer and generate exactly that many children in a loop to observe how `wait()` interacts with multiple terminations.)

CHAPTER 6: Exec

1. The Necessity of Process Replacement

In a Unix environment, process creation relies heavily on specific system primitives. If the `fork()` system call were the sole primitive available for creating new processes, programming would be exceptionally difficult and limited. This limitation exists because `fork()` is exclusively designed to create identical copies of the calling process.

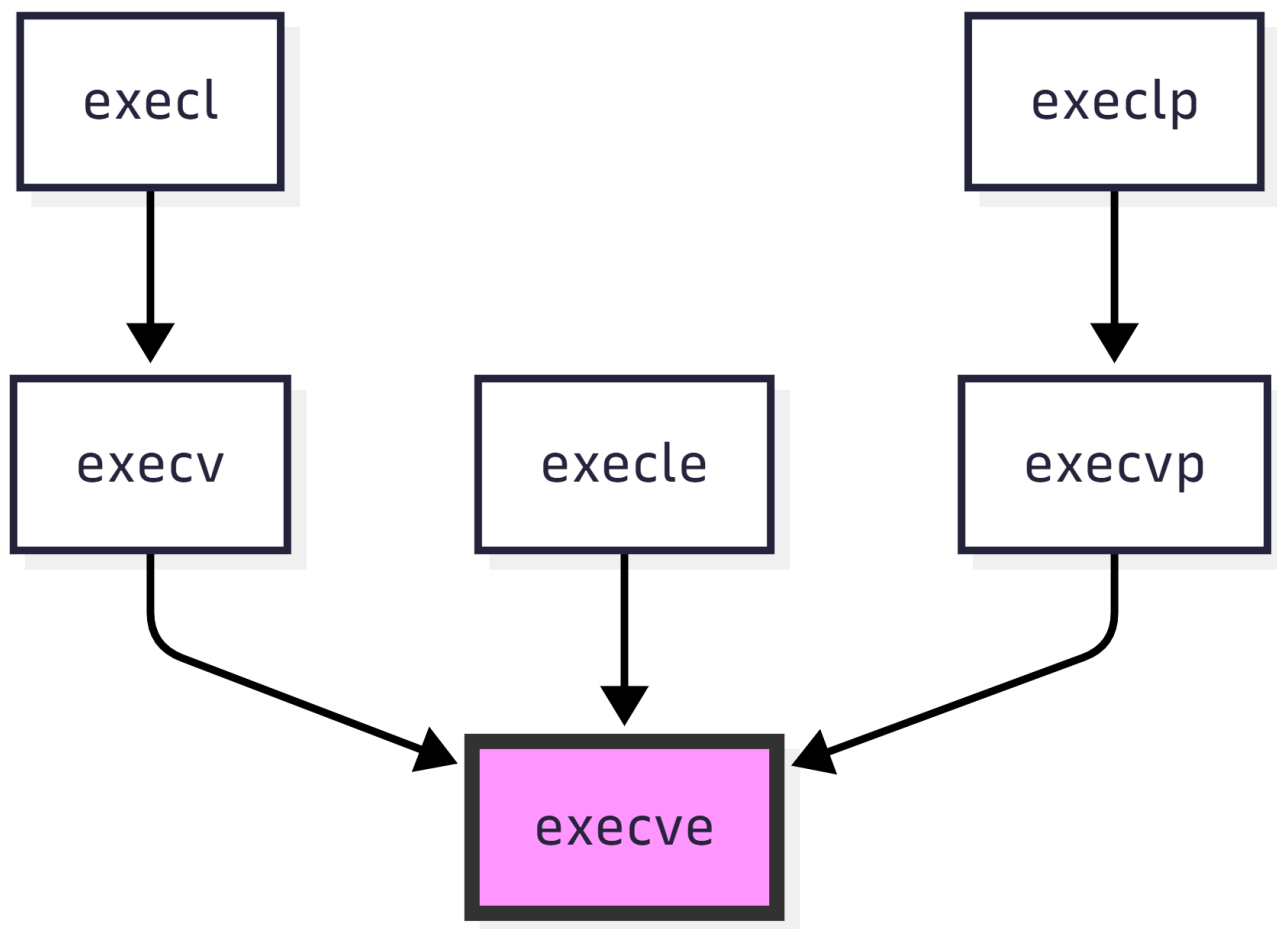
To overcome this and actually run different applications, the operating system provides the `exec()` family of primitives. The primary function of the `exec()` family is to allow a process to start the execution of a completely different program. It achieves this not by creating a new process alongside the old one, but by entirely replacing (or overwriting) the calling process with the new program.

2. The Architecture of the `exec` Family

While developers commonly refer to "calling `exec()` ", there is actually an entire family of functions designed to handle different ways of passing arguments and environments to the new program. However, beneath these varying interfaces lies a single core system call.

The Underlying Hierarchy

The family consists of six distinct functions: `execl` , `execv` , `execle` , `execvp` , `execvp` , and `execve` . Despite the variety, all of these functions ultimately route their execution through `execve()` , which is the only true system call within this entire family. The other five functions serve essentially as library wrappers that format the arguments and environment variables differently before invoking `execve()` .



Schema Translation: The structural flowchart demonstrates the internal routing of the `exec` family. The functions `execl` and `execvp` first resolve into `execv` and `execvp` respectively, or pass their data directly downward, but ultimately `execv`, `execl`, and `execvp` all converge by calling the foundational `execve` system call.

3. Memory and Execution Flow During an `exec` Call

When a process invokes any function from the `exec()` family, a drastic transformation occurs within its memory space. Because the function's purpose is replacement, the caller program is completely overwritten. This overwrite is comprehensive, wiping out and replacing the original program's text (code), data, heap, and stack segments.

Once the memory is replaced, the newly loaded program begins its execution directly from its own `main()` function. Crucially, while the program itself changes entirely, the process identity remains constant; the process ID (PID) does not change after an `exec()` call.

Code Execution Behavior

Because the original program is eradicated from memory upon a successful `exec()` call, any lines of code written sequentially after the `exec()` statement will generally never run.

Consider a standard C implementation using `execl`:

```
#include <stdio.h>
#include <unistd.h>

main() {
    printf("Executing ls\n");
    execl("/bin/ls", "ls", "-l", NULL);
    perror("The execl call generated an error\n");
    exit(1);
}
```

In this scenario, the `execl()` function replaces the original C program by overwriting it with the executable located at `/bin/ls` (passed as a parameter). The `perror()` and `exit(1)` statements immediately following `execl()` will only ever be executed if a failure or error occurs during the execution of `execl()` itself. If `execl()` succeeds, those error-handling lines cease to exist in memory.

The Program Counter (PC) Shift

[Photo: Memory Transition Before and After `exec`]

Schema Translation: When visualizing the memory layout before and after the `exec` call, we observe the Program Counter (PC) shifting contexts.

- **Before the call:** The PC points to the instructions of the original program, moving from `printf(...)` down to the `execl("/bin/ls", ...)` instruction.
- **The Transition:** The `exec` call acts as a bridge, wiping the current memory block.
- **After the call:** The memory block now entirely contains the `ls` code. The PC is reset to point to the very top (the entry point/ `main()`) of this new `ls` program instruction set.

4. The Six Variants and Their Parameters

The six versions of `exec()` are differentiated based on how they handle three main factors:

1. **Path Resolution:** Whether they require an absolute/relative path or use the `PATH` environment variable to search for the command.
2. **Environment Variables:** Whether they automatically inherit the parent's environment variables or allow the programmer to specify a custom environment.
3. **Argument Passing:** Whether command-line arguments are passed as a variable-length list (terminated by `NULL`) or as an array of character pointers.

These differences are denoted by specific suffixes added to the base `exec` name:

- **p suffix:** Indicates that the function handles the `PATH` environment variable; it will search for the command using the directories listed in `PATH`.
- **v suffix:** Indicates that arguments are passed as a vector (an array, `argv[]`).
- **e suffix:** Indicates that an array of environment pointers (`envp[]`) is explicitly passed to the function.
- *(No `v` implies a list `l` of arguments; no `e` implies inheriting the current environment).*

Parameter Matrix

Function	Needs Pathname	Needs Filename (Uses <code>PATH</code>)	Arg List (...)	Arg Vector (<code>argv[]</code>)	Inherits Environ	Custom Env (<code>envp[]</code>)
<code>execl</code>	Yes	No	Yes	No	Yes	No
<code>execlp</code>	No	Yes	Yes	No	Yes	No
<code>execle</code>	Yes	No	Yes	No	No	Yes
<code>execv</code>	Yes	No	No	Yes	Yes	No
<code>execvp</code>	No	Yes	No	Yes	Yes	No
<code>execve</code>	Yes	No	No	Yes	No	Yes

Note: All data in this table is derived from the parameter classification matrix.

C Prototypes

To utilize these variants, the following prototypes are defined in the C standard library:

- `int execl(char *pathname, char *arg0, ... );`
- `int execlp(char *filename, char *arg0, ... );`
- `int execle(char *pathname, char *arg0, ..., char* envp[]);`
- `int execv(char *pathname, char *argv[]);`
- `int execvp(char *filename, char *argv[]);`
- `int execve(char *pathname, char *argv[], char* envp[]);`

5. Process Properties Across the `exec` Boundary

Because the process ID remains the same, a newly executed program does not start completely fresh; it inherits a significant amount of context from the program that called `exec()`. However, for security and functional reasons, certain properties are strictly not inherited.

Inherited Properties

The following information and properties are smoothly inherited from the calling process to the new program during an `exec()` call:

- The Process ID (PID) and Parent Process ID (PPID).
- The real User ID (uid) and real Group ID (gid).
- Any supplementary Group IDs.
- The Process Group ID and Session ID.
- The associated control terminal.
- The current working directory and the root directory.
- The file creation mask (commonly known as `umask`).
- Any established file locks.
- The process's signal mask.

Not-Inherited Properties

Conversely, the following properties are explicitly NOT inherited or are subject to change during the `exec()` transition:

- **Effective User-ID and Effective Group-ID:** These do not simply carry over. Instead, their new values are dependent upon the `suid` (Set-User-ID) bits configured on the newly targeted executable file.

- **Open Files:** The fate of currently open file descriptors depends entirely on the "close-on-exec" flag associated with each specific descriptor.
 - If the "close-on-exec" flag is set to true for a file descriptor, that file is automatically closed when `exec()` is called.
 - If the flag is false, the file descriptor is kept open and accessible to the new program. Keeping them open is the default behavior in the operating system.

6. The Unix Process Creation Paradigm: Combining `fork` and `exec`

By themselves, `fork()` and `exec()` have distinct and somewhat limiting functions: `fork()` only creates an identical duplicate of a process, while `exec()` only overwrites a current process with a new one. However, when combined, these two system calls form the foundational programming paradigm for process management in Unix and Linux environments.

The standard pattern involves a parent process using `fork()` to create a child process. Immediately after the split, the child process uses `exec()` to execute a new program, while the parent typically waits for the child to finish.

Implementation Example

Here is the standard C implementation of this combined paradigm:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void fatal(char *s) {
    perror(s);
    exit(1);
}

main() {
    pid_t pid;

    switch(pid = fork()) {
```

```

        case -1:
            // Fork failed
            fatal("fork failed");
            break;
        case 0:
            // Child process execution
            execl("/bin/ls", "ls", "-l", NULL);
            // This line only runs if execl fails
            fatal("exec failed");
            break;
        default:
            // Parent process execution
            wait((int *)0);
            printf("ls completed\n");
            exit(0);
    }
}

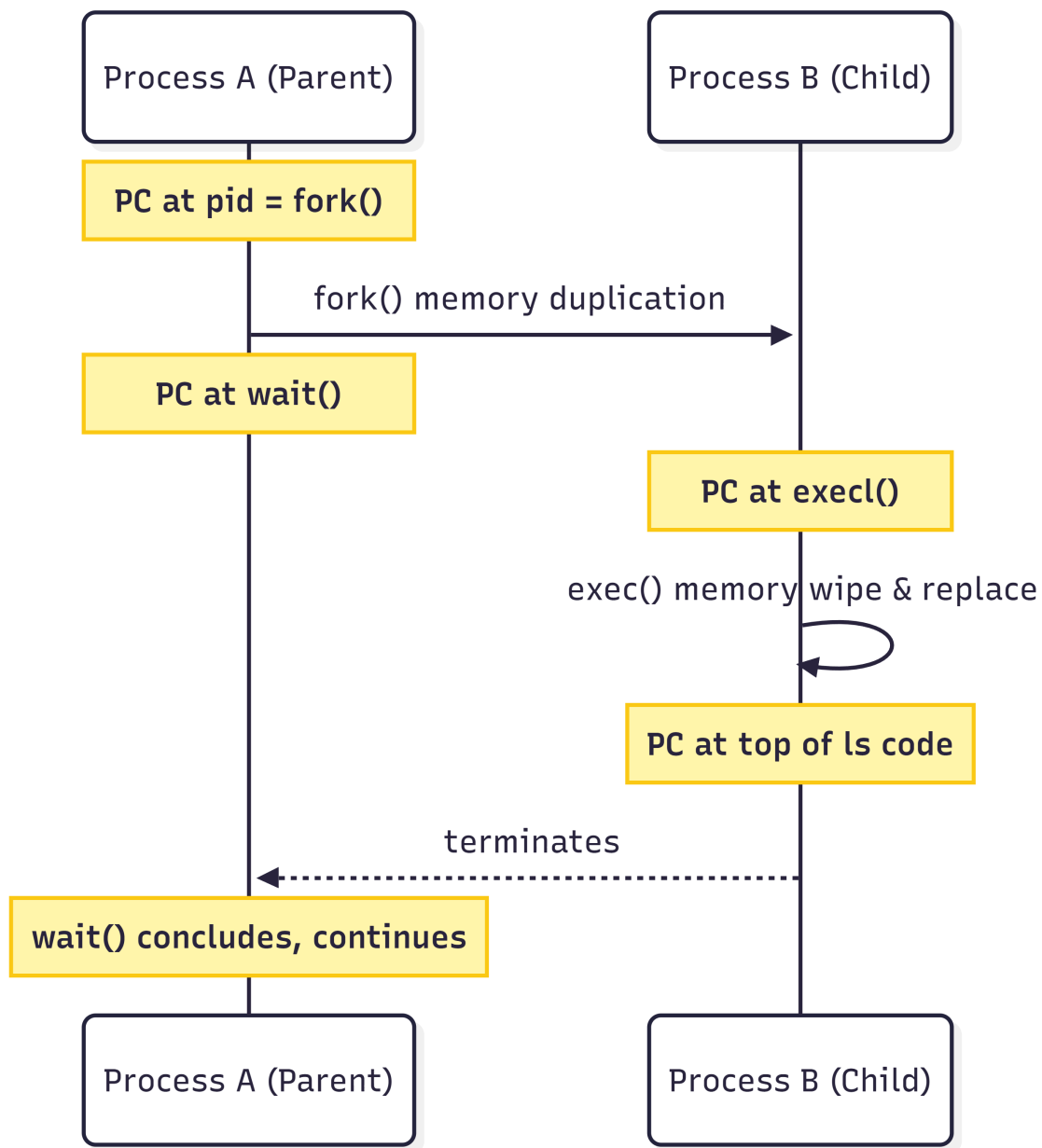
```

Note on the code: The `wait()` function in the default case ensures the parent halts its progression until the child process (running `/bin/ls`) completes.

Visualizing the `fork` and `exec` Transition

[Photo: Memory Architecture of Combined `fork` and `exec`] *Schema Translation:* When we look at the memory space and Program Counter (PC) during this combined operation, a distinct timeline emerges.

1. **Before Fork:** A single process (Process A) runs and its PC points to the `pid = fork();` instruction.
2. **The Fork Split:** Memory duplicates. We now have Process A (Parent) and Process B (Child).
3. **Divergent Paths:** Process A's PC moves to `wait((int *)0);`, effectively pausing. Simultaneously, Process B's PC moves to `execl("/bin/ls", ...);`.
4. **The Exec Transformation:** Process B undergoes the `exec` call. Its memory is entirely wiped and replaced by the `ls` program code. Process B's PC now points to the very first instruction (`/* ls code */`) of this new program, while Process A continues to wait.



7. The `system()` Library Call

For developers seeking a simpler alternative to manually writing the `fork` and `exec` pattern, ANSI C provides a standard library function called `system()`. Its prototype is: `int system(char* command);`

This function executes a specified command and automatically waits for it to terminate. While it performs a function incredibly similar to the `fork + exec` pattern, it is not a direct system call. Furthermore, it is not defined in the POSIX.1 standard (because it isn't an operating system interface), but it is defined in POSIX.2.

Why it is discouraged: The behavior of the `system()` call is highly system-dependent. On Unix systems, it essentially acts as a wrapper that calls `/bin/sh -`

`c` to run the command. Because of this dependency on the shell and the overhead it introduces, using `system()` is generally discouraged in robust systems programming.

8. Managing the Process Environment

The environment of a process is defined as a collection of strings, where each string is terminated by a null character (`\0`). The data takes the format of `identifier = value` . Structurally, this environment is represented in memory as an array of character pointers, which is ultimately terminated by a null pointer.

By default, the environment of a process is inherited directly from its parent process.

Accessing the Environment

There are two primary ways to access the environment variables from within a C program:

1. **Using `envp` :** You can add a third parameter, `char **envp` , to the `main` function signature.

```
main(int argc, char **argv, char **envp) {  
    while(*envp) {  
        printf("%s\n", (*envp)++);  
    }  
}
```

2. **Using the Global Variable:** You can access the environment directly via the globally defined variable: `extern char **environ; .`

Modifying the Environment

To search for a specific environment variable, the standard library `<stdlib.h>` provides `getenv() : char *getenv(const char *name);` This function takes the variable name as an argument and returns a pointer to its value (everything to the right of the `=` symbol). If the variable doesn't exist, it returns `NULL` .

To modify or extend the environment, you use `putenv() :`
`putenv("variable=value");`

Passing a Custom Environment via `exec`

If you need to specify a completely new environment for a program during an `exec` call, you cannot use the standard variants. You must use `execle` or `execve`, both of which accept an `envp` array as a parameter.

Example of Setting a New Environment:

```
#include <unistd.h>
#include <stdio.h>

main() {
    char *argv[2], *envp[3];

    argv[0] = "setmyenv";
    argv[1] = (char *)0;

    // Defining the custom environment
    envp[0] = "var1=value1";
    envp[1] = "var2=value2";
    envp[2] = (char *)0;

    // Executing showmyenv with the new environment
    execve("./showmyenv", argv, envp);
    perror("execve failed");
}
```

Running this executable will output only `var1=value1` and `var2=value2`, ignoring whatever environment the parent originally had.

9. Process Directory Context

Beyond environment variables, processes rely heavily on directory contexts, which are inherited from the parent process.

- **Current Working Directory:** Every process has a current working directory. To change this context programmatically, you use the `chdir` system call (`#include <unistd.h>`): `int chdir(const char *path);`
- **Root Directory:** Each process is also associated with a root directory. This directory dictates the starting point (the absolute top) of the file system as it is visible to that specific process. To change the root directory (often used for

sandboxing or security), you use the `chroot` system call: `int chroot(const char *path);`

10. Practical Implementation Challenges (Exercises)

To consolidate this theoretical knowledge, the module proposes a progressive set of practical exercises to build a simple shell-like program:

- **Exercise 1 - Basic Execution:** Write a program that takes an executable's filename as a command-line argument, runs it, and waits for termination (optionally supporting the `$PATH` variable).
- **Exercise 2 - Iterative Spawning:** Upgrade the program to support options/arguments. Encapsulate the `fork + exec` pattern inside a custom `spawn()` function. The program should iteratively read the target executable from standard input, launch it upon pressing `<enter>`, and repeat until an end-of-file (EOF) character (`ctrl+d`) is caught.
- **Exercise 3 - Shell Features:** Finally, add a shell prompt in the format `username@hostname $` before reading input. Allow the system to launch processes in the background (without waiting for termination) if the input string ends with an `&` character. As a final challenge, write a custom implementation of the standard `system()` library function that respects its prototype, ignoring the native signal handling.

CHAPTER 7: Signals

1. Introduction to Signals

Signals are fundamental mechanisms in modern operating systems, acting as software interrupts at the process level. Their primary purpose is to allow the management of asynchronous events that unexpectedly interrupt the normal instruction flow of a running process. When a signal is delivered, the handling of that specific event is executed by a dedicated function known as a **signal handler**. The behavior and implementation of signal handling across different UNIX-like systems are standardized by POSIX.1.

Characteristics of Signals

Every signal is assigned a specific identifier, which is a symbolic name that always begins with the three characters `SIG`. For example, `SIGABRT` is the designated

identifier for the abortion signal.

The total number of available signals typically ranges from 15 to 40, depending entirely on the specific UNIX version being used. For context:

- The POSIX.1-1990 standard mandates a baseline of 20 standard signals.
- The Linux kernel supports 32 standard signals, alongside an additional 32 real-time signals.

Underneath these symbolic names, signals are actually represented by positive integers. These constant definitions are typically found within system header files, specifically in `/usr/include/asm-generic/signal.h`.

Interestingly, there is a special **null signal** (Signal #0). While it has no actual effect on the process, it is highly useful: it can be sent to a process to check if a signal *could* be delivered, thereby verifying whether the target process is currently running and if the sender has the correct permissions.

2. Conditions for Signal Generation

Signals are generated by various system conditions, which broadly fall into hardware and software categories:

1. **Terminal Interactions:** Pressing special key combinations on the terminal can immediately generate signals. A common example is pressing `Ctrl-C`, which generates the `SIGINT` (Interrupt) signal.
2. **Hardware Exceptions:** When a process performs an illegal or impossible hardware operation, the hardware generates an interrupt. This interrupt is captured by the OS kernel, which then translates it into a signal and sends it to the offending process. Examples include:
 - A mathematical division by zero, which generates `SIGFPE` (Floating Point Exception).
 - An invalid reference to a memory address, which generates `SIGSEGV` (Segmentation Fault).
3. **The `kill()` System Call & Command:** Processes can manually send signals to other processes using the `kill()` system call, or a user can do so via the `kill` command (the shell interface to the system call). There is a strict limitation to this: the effective user ID (`euid`) of the process executing the `kill()` must be exactly the same as the process receiving the signal, unless the sender is the superuser (root, `euid 0`).

4. **Software Events:** Signals are routinely generated asynchronously by the operating system itself to notify processes of software-level events. Examples include the termination of a child process, which triggers `SIGCHLD`, or the expiration of an internal timer, which triggers `SIGALRM`.

3. Actions Associated with Signals

When a signal is generated and delivered to a process, the process must react. There are three primary actions that can be associated with a signal:

Action 1: Ignore the Signal

A process can choose to simply ignore incoming signals. However, there are exceptions: `SIGKILL` and `SIGSTOP` can **never** be ignored. This strict rule exists to ensure that the system superuser always retains the unassailable ability to kill or suspend any errant process. Furthermore, if a process attempts to ignore hardware-generated signals (like memory faults), the resulting behavior is completely undefined according to POSIX standards.

Action 2: Execution of the Default Action

If a process does not explicitly define how to handle a signal, the OS executes a default action. For many "critical" signals, this default action is the immediate termination of the process. Frequently, this termination triggers the generation of a **core dump file** (a memory snapshot) in the current working directory for debugging purposes. However, a core file will *not* be generated under the following conditions:

- The `set-user-id` or `set-group-id` bits are set, and the process's uid/gid are different from the file owner/group.
- There is a lack of write permissions for the process in the running directory.
- The resulting core file would be too large.

Action 3: Catching the Signal

A process can actively intercept a signal by installing a custom function called a **signal handler**. When the signal arrives, the kernel interrupts the main program execution and explicitly calls this handler function on behalf of the process.

Signal handlers must execute quickly. For example, upon catching a `SIGCHLD` signal (indicating a child process died), a suitable action is for the handler to run

`waitpid()` . If a `SIGTERM` (standard termination request) is caught, the handler might safely remove temporary files and save persistent data before gracefully exiting.

Theoretical Workflow: The Interruption Process

When a signal interrupts normal execution, the system follows a specific contextual jump.

[Photo: Flow of Execution during Signal Delivery]

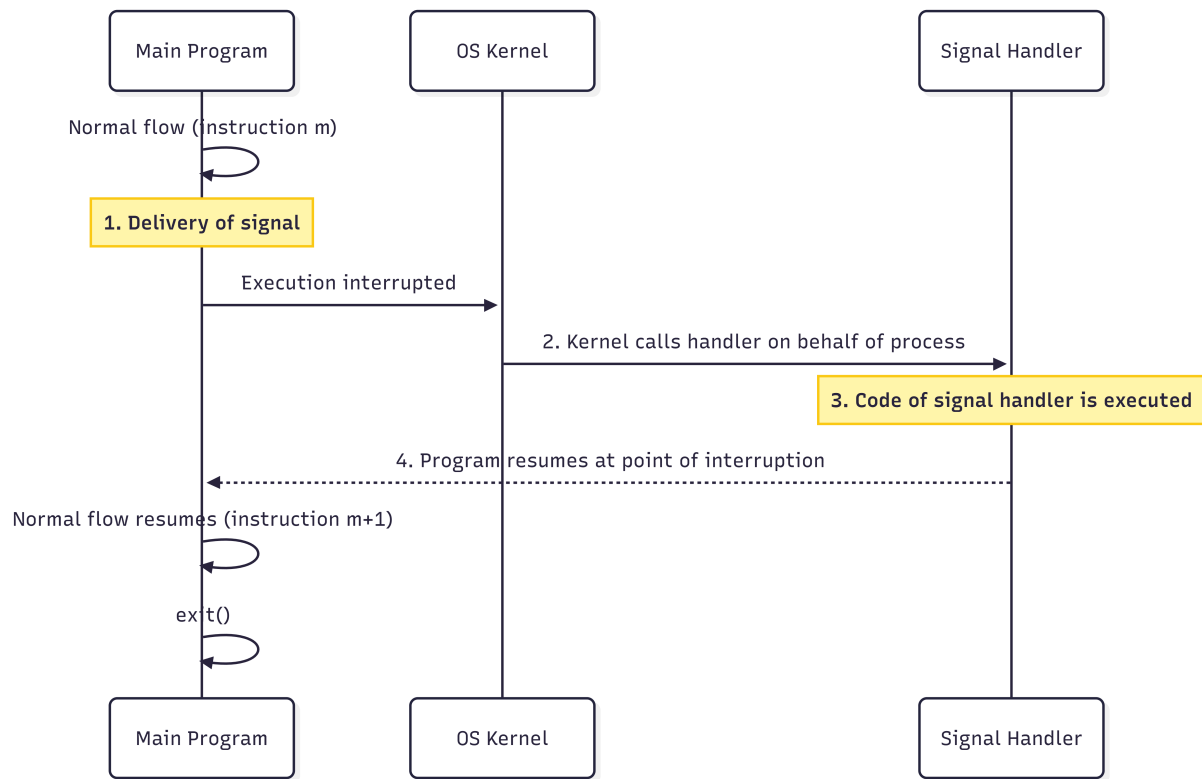
Detailed Contextual Description of the Structure:

The execution flow begins inside the **Main program**. The program executes sequentially downwards (e.g., executing `instruction m` , with the next scheduled line being `instruction m+1`). At this precise moment (Step 1), a signal is delivered to the process. The normal flow of execution is immediately suspended right after `instruction m` .

In Step 2, the **Kernel** steps in and executes a context switch, calling the installed **Signal handler** on behalf of the process.

During Step 3, the code defined within the **Signal handler** block is executed from start to return.

Finally, in Step 4, once the handler issues a `return` , the execution flow jumps back into the **Main program** exactly at the point of interruption, resuming smoothly at `instruction m+1` and continuing downwards until program termination (`exit()`).



4. Dictionary of Important Signals

The following is an exhaustive breakdown of the most critical signals recognized by the operating system, organized by their default behaviors and causes:

- **SIGABRT** : Indicates abnormal termination and generates a core file. It is explicitly generated by calling the `abort()` function.
- **SIGALRM** : Causes process termination. It is generated when a timer, set via the `alarm()` or `setitimer()` functions, expires.
- **SIGBUS** : A Non-POSIX signal that causes termination and generates a core dump. It indicates a hardware fault, the exact nature of which is defined by the OS.
- **SIGCHLD** : Sent to a parent process when one of its child processes terminates. The default action is to ignore it. The parent process *must* define a signal handler that calls `wait()` or `waitpid()` to reap the child and prevent zombies.
- **SIGFPE** : Causes termination and generates a core file. Raised during arithmetic exceptions, such as a division by 0.
- **SIGHUP** : Causes process termination. It is sent to a process if its controlling terminal is disconnected.
- **SIGILL** : Causes termination and generates a core file. Raised when a process attempts to perform an illegal hardware action.

- **SIGINT** : Causes process termination. Delivered when a process receives an explicit break character (like `Ctrl-C`) directly from the terminal interface.
- **SIGIO** : A Non-POSIX signal indicating an asynchronous I/O event. The default action varies between termination or being ignored.
- **SIGKILL** : The forced, unblockable way to kill a process. Causes immediate termination.
- **SIGPIPE** : Causes process termination. Raised when a process attempts to write to a pipe or socket where the reading end has already finished or closed.
- **SIGSEGV** : Causes termination and generates a core dump. Raised when a process executes a strictly invalid memory reference.
- **SIGTSTP** (noted as **SIGSTP** in source): The default action is to stop (suspend) the process. It is delivered when a process receives a suspend character (like `Ctrl-Z`) from the terminal.
- **SIGSYS** : Causes termination and generates a core dump. Raised during an invalid system call invocation, such as passing an incorrect parameter.
- **SIGTERM** : A standard termination request signal, normally generated by executing a standard `kill` command. Causes process termination.
- **SIGURG** : A Non-POSIX signal whose default action is to be ignored. It signifies that an urgent condition has occurred, such as the arrival of out-of-band data over a network connection.
- **SIGUSR1** & **SIGUSR2** : User-definable signals whose default action is to cause process termination. Developers can use these for application-specific IPC.

5. System Library Calls for Signal Handling

To define custom behaviors for asynchronous events, processes utilize specific system library calls. The foundational, albeit older, function for this is `signal()`, which traditionally possesses a somewhat complex C prototype: `void (*signal(int signo, void (*func)(int)))(int);`.

A more readable and modern alternative definition uses a `typedef`: `typedef void sighandler_t(int);` followed by `sighandler_t *signal(int, sighandler_t*);`.

When invoking this function, two primary arguments are required:

- **signo** : The integer identifier of the specific signal to handle.
- **func** : The action to be performed when the signal is caught. This can be a pointer to a custom signal handler function to process the event in userspace,

or one of the predefined constants `SIG_IGN` (to ignore the signal) or `SIG_DFL` (to execute the default action). Note that custom handling and ignoring are not applicable to `SIGKILL` and `SIGSTOP`.

If the setup is successful, `signal()` returns a pointer to the previous signal handler. In the event of an error, it returns the constant `SIG_ERR` and sets the global `errno` variable accordingly. These constants (typically defined in `/usr/include/asm-generic/signal-defs.h`) act as pointers to functions taking an integer and returning nothing, cast in such a way that they cannot accidentally conflict with actual memory addresses of user-defined handlers (e.g., `SIG_DFL` is cast as 0, `SIG_IGN` as 1, and `SIG_ERR` as -1).

6. Mechanisms for Signal Generation

Processes have several system calls at their disposal to actively generate and send signals.

The `kill()` and `raise()` Functions

The `kill()` system call—`int kill(pid_t pid, int signo);`—is used to send a signal to a specific process or an entire group of processes. The destination is determined by the `pid` argument:

- `pid > 0`: The signal is sent exactly to the process identified by `pid`.
- `pid == 0`: The signal is broadcast to all processes belonging to the same process group as the calling process.
- `pid < -1`: The signal is sent to the specific process group identified by the absolute value of the PID (`-pid`).
- `pid == -1`: The behavior for this value is officially undefined.

Security permissions tightly govern `kill()`. The superuser can send signals to any process. For standard users, the real or effective user ID (`uid`) of the source process must exactly match the real or effective `uid` of the destination process. POSIX.1 also defines signal 0 as the "null signal"; sending it performs all standard error and permission checking mechanisms without actually delivering a signal, making it a clever way to verify if a process currently exists (though PIDs can be reused by the OS).

To send a signal to the calling process itself, one can use `int raise(int signo);`.

Timers: `alarm()` , `setitimer()` , and `pause()`

Timers allow processes to schedule future signals. The `unsigned int alarm(unsigned int sec);` function establishes a timer that will deliver a `SIGALRM` to the process once the specified seconds have elapsed. It is important to note that this system is not real-time; the OS only guarantees that the pause will be *at least* `sec` seconds, as system scheduling mechanisms can delay execution. A process can only have one active alarm at a time. If an alarm is already running when `alarm()` is called again, the function returns the number of seconds remaining on the old timer, and if `sec` is set to zero, any pre-existing alarm is completely canceled. Because the default action of `SIGALRM` is process termination, a custom handler is usually installed.

For more granular control over timers, developers use `getitimer()` and `setitimer()`. Conversely, if a process simply needs to wait for any signal to arrive, it can invoke `int pause();`, which suspends the process entirely until a signal is caught, subsequently returning `-1` and setting `errno` to `EINTR`.

7. Process Sleep and Abortion

The `unsigned int sleep(unsigned int seconds);` function forces the calling process to suspend execution until either the specified time passes (returning `0`) or a signal is caught and handled without terminating the process (returning the remaining time). While it functionally resembles `alarm()`, it is explicitly implemented without utilizing `alarm()` to prevent timer conflicts.

For critical failures, `void abort();` is used to send a `SIGABRT` to the calling process. If the handler is set to `SIG_DFL`, it terminates the process. Setting it to `SIG_IGN` is explicitly not allowed. If a custom handler is defined, the signal is caught, the cleanup code executes, and if the handler returns, the signal is automatically raised again to guarantee termination (unless the handler explicitly calls `exit` or `_exit`).

8. Signal Inheritance During Process Creation

When new processes are spawned, signal configurations are inherited differently depending on the method:

- `fork()` : Signal handlers defined in the parent process are perfectly copied and preserved in the child process.

- **exec()** : Because the process image is entirely replaced, custom signal handlers cannot be preserved (the handler's code will no longer exist in the new child program). Therefore, any signal previously set to a custom handler is reset to the default action, while signals configured to be ignored or use default actions are left unchanged.
- **Background Processes**: As a special case, when an interactive process is launched in the background, the OS automatically sets the `SIGINT` and `SIGQUIT` signals to be ignored.

9. Reentrant Signals and Functions

A major complication introduced by signal handlers is the risk of data corruption. Because signals interrupt the normal sequential flow of instructions, a program might jump to handler code while in the middle of updating a data structure. If the handler modifies that same structure and returns, an inconsistent state or software bug can occur. A classic example is a `SIGSEGV` occurring if a signal interrupts `malloc` (which heavily manages the heap) and the signal handler attempts to call `malloc` again concurrently.

This necessitates the concept of **reentrant functions**—functions that can be called safely by multiple tasks concurrently without causing data corruption. Non-reentrant functions cannot be safely shared unless protected by mutual exclusion (like semaphores or disabling interrupts). To be strictly reentrant, a function must:

- Not maintain or return pointers to static data.
- Only use local data, or protect global data by making local copies.
- Never invoke other non-reentrant functions.

POSIX.1 defines a strict list of functions guaranteed to be reentrant, including `_exit`, `access`, `alarm`, `read`, `write`, `wait`, `sleep`, and `kill`, among others. Functions are generally excluded from this list if they rely on static data structures, invoke memory management like `malloc()` / `free()`, or belong to the standard I/O library (which heavily relies on internal buffering).

Notably, developers must rigorously avoid calling `printf()` inside signal handlers because its internal buffers can become corrupted if interrupted mid-use.

Furthermore, if a handler uses a reentrant function that modifies `errno`, it is best practice to temporarily save and restore the value of `errno` to avoid breaking the main program's error checking.

10. The Evolution of POSIX Standards

Historically, signals in early UNIX systems were "unreliable." They could be lost without the process ever knowing, and once a signal was caught, the handler was immediately reset to default, requiring the programmer to manually re-establish the handler at the top of the function. This created a dangerous race condition where a second signal could arrive before the handler was re-installed, killing the process unexpectedly.

POSIX revolutionized this by enforcing reliable signal management. Under POSIX, signal handlers explicitly remain installed after catching a signal, and developers can thoroughly manage which signals are blocked, delivered, or queued.

Additionally, POSIX addressed the behavior of "slow" system calls—operations like `read()` and `write()` on terminals/pipes/networks, `pause()`, `wait()`, and certain IPC/`ioctl` calls that can block indefinitely. To prevent a signal from being delayed for long periods while waiting for a system call to finish, POSIX ensures these blocking calls are interrupted. Upon interruption, the system call terminates prematurely, returns `-1`, and sets `errno` to `EINTR`. Programmers must explicitly handle this error code or opt to have the system automatically restart the interrupted call by utilizing the `SA_RESTART` flag in `sigaction()`.

11. Reliable Signal Management

To understand POSIX signal management, we must distinguish between generation and delivery:

- **Generation:** Occurs when the event causing the signal happens (e.g., an invalid memory reference causing `SIGSEGV`), triggering a flag to be set in the process control block.
- **Delivery:** Occurs when the process actually takes the action associated with the signal.
- **Pending:** The time interval between generation and delivery.

Processes (except for `SIGKILL` and `SIGSTOP`) can explicitly block the delivery of signals using a **Signal Mask**—a system-level bitmap containing one bit for every possible signal. If a signal is blocked, it remains completely pending until the process unblocks it or changes the action to "ignore". The list of currently pending signals can be retrieved using `int sigpending(sigset_t *set);`. The mask itself can be checked and updated using `sigprocmask()`. If a blocked signal is triggered multiple times, POSIX does not require them to queue; they are

generally only delivered once when unblocked. Additionally, POSIX does not strictly enforce delivery priority, but advises that critical signals like `SIGSEGV` be processed first.

Advanced Handling with `sigaction()`

In modern UNIX systems, the robust `sigaction()` function fundamentally replaces and often underpins `signal()`. `int sigaction(int signo, struct sigaction *newact, struct sigaction *oldact);`

This function utilizes a dedicated struct:

```
struct sigaction {
    void (*sa_handler)(); /* signal handler (or SIG_IGN/SIG_DFL)
    */
    sigset_t sa_mask;      /* additional signals to block during
    handling */
    int sa_flags;          /* options, e.g, SA_RESTART */
}
```

When the `sa_handler` executes, the current process mask is temporarily saved. The system then adds the signal currently being handled (`signo`), alongside any signals specified in the `sa_mask` , to the process mask, effectively blocking them until the handler returns and the original mask is restored. The `sa_flags` field accommodates non-standard extensions like `SA_RESTART` (forces automatic restart of slow system calls) and `SA_INTERRUPT` (eliminates automatic restarts).

The `sigsuspend()` Call

For complex synchronization, `int sigsuspend(sigset_t *sigmask);` is utilized. It atomically sets the process mask to `*sigmask` and completely suspends the process until either a signal in the mask is caught or the process terminates. When it eventually returns, it always outputs `-1` with `errno` set to `EINTR`.

12. Course Practical Exercises

The source notes assign four practical exercises to synthesize these concepts:

- **Exercise 1:** Write a program that captures user-defined signals (`SIGUSR1` , `SIGUSR2`) and prints a reception message when triggered via the `kill` shell command.

- **Exercise 2:** Create a program that spawns `NUMCHLD` child processes, ensuring each terminates with a different exit status. The parent must intercept `SIGCHLD` for every termination, print a warning, and finally output a summary list of all PIDs and exit statuses.
- **Exercise 3:** Re-implement Exercise 2 explicitly utilizing `sigaction()` instead of `signal()`, ensuring the struct is initialized to 0 and checking all return codes.
- **Exercise 4 (Homework):** Develop a custom implementation of the standard `system()` function. The application must appropriately handle signals by temporarily ignoring `SIGINT` and `SIGQUIT`, while explicitly blocking `SIGCHLD` during execution.

CHAPTER 8: Pipe and FIFO

Interprocess communication (IPC) is a foundational concept in operating systems, allowing independent or related processes to share data and synchronize actions. This guide explores the oldest, and arguably the most widely used, form of IPC in Unix-like systems: the Pipe, along with its named counterpart, the FIFO.

1. Introduction to Standard Pipes

A pipe serves as a dedicated communication channel established directly between two processes. When utilizing standard pipes, the data stream adheres to three strict structural characteristics:

- **Unidirectional:** Data flows in exactly one direction.
- **FIFO (First-In, First-Out):** The first byte written into the pipe is the first byte read out of it.
- **Unstructured:** The channel carries a raw stream of bytes without inherent record boundaries or formatting.

To conceptualize this, imagine a physical tube: multiple producer processes can push data into one opening, and consumer processes extract that exact data stream from the opposite opening, strictly adhering to the first-in, first-out sequence.

Despite their utility, standard pipes possess two significant limitations:

1. They operate strictly in **half-duplex** mode, meaning communication is restricted to a one-way street.

2. They are usable **only** between processes that share a common "ancestor". This is because standard pipes exist purely in memory and rely on process inheritance to be accessed.

2. Creating a Pipe

Standard pipes are initialized using the `pipe()` system call.

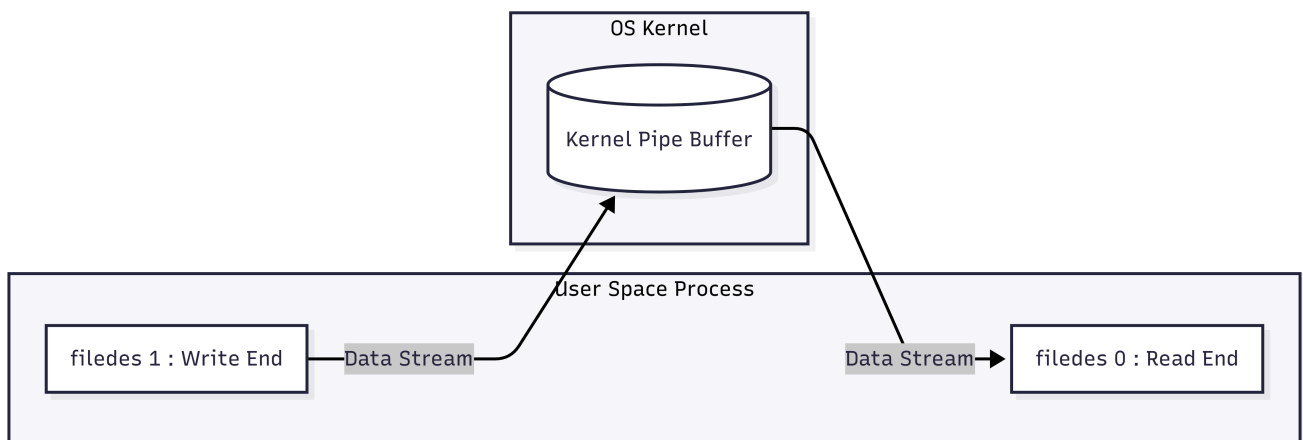
The function signatures are as follows:

```
int pipe(int filedес[2]); or int pipe(int *filedes);
```

When successfully executed, the `pipe()` system call populates the array provided in the argument with two distinct file descriptors:

- `filedes[0]` : This file descriptor is exclusively opened for **reading**.
- `filedes[1]` : This file descriptor is exclusively opened for **writing**.

Consequently, the output generated and pushed into the write end (`filedes[1]`) immediately becomes the input available at the read end (`filedes[0]`).



[Photo: Pipe Memory Mapping] This structural schematic illustrates a user process containing an array `fd` with indices `0` and `1`. A directional arrow originates from `fd[1]` and points downwards into a rectangular structure labeled `pipe` located securely within the `kernel` boundary. A second directional arrow loops out of the kernel's `pipe` structure and points directly into `fd[0]`. This visually confirms that while the application interacts with local file descriptors, the actual data buffering and routing occur safely within kernel space.

3. Practical Usage: Parent-Child Communication

Creating a pipe within a single, isolated process is generally useless since a process does not need an IPC mechanism to talk to itself. The standard operational paradigm involves using a pipe in conjunction with process creation.

Normally, a process will first call `pipe()` to establish the kernel channel, and immediately followed this by calling `fork()`. Because `fork()` creates an exact duplicate of the parent process, the file descriptors pointing to the pipe are duplicated into the child process. As a result, a shared communication channel, physically allocated within the kernel, perfectly connects the parent and child processes.

Best Practices: Closing Unused Endpoints

Because standard pipes are strictly half-duplex, a bidirectional setup where both processes keep both read and write ends open is highly discouraged. Unused channels must be deliberately closed.

If Process A intends to send data to Process B:

- Process A acts as the writer and must close its read endpoint using `close(fd[0]);`.
- Process B acts as the reader and must close its write endpoint using `close(fd[1]);`.
- Once isolated, Process A writes to the pipe, and the data is safely read by Process B.

4. Standard I/O Operations on Pipes

Because the pipe endpoints are standard file descriptors, they can be manipulated using normal I/O system calls like `read()` and `write()`.

Reading from a Pipe

When a process executes `read()` on a pipe descriptor:

- **If the write endpoint is still open:** The call successfully retrieves the available data and returns the number of bytes read. If the pipe is currently empty (no data available), the `read()` call will block and suspend the process until data is written to the opposite end, or until the pipe is entirely closed.
- **If the write endpoint has been closed:** The call will first return any remaining available data in the buffer alongside the byte count. Once the buffer is entirely empty, all subsequent `read()` calls will return `0`, which serves as the standard indicator for the "end of file" (EOF).

Writing to a Pipe

When a process executes `write()` on a pipe descriptor:

- **If the read endpoint is open:** The data is pushed into the pipe and buffered by the kernel until the consumer process reads it. The `write()` call may return immediately, or it may block depending entirely on the amount of data being written and the available space in the kernel buffer.
- **If the read endpoint has been closed:** Writing to a pipe that has no active readers generates a `SIGPIPE` signal.
 - If the process ignores or catches this signal, the `write()` system call will abort, return `-1`, and set the internal `errno` variable to `EPIPE`.
 - If the signal is not caught, the default action of `SIGPIPE` is to abruptly terminate the writing process.

5. Pipe Metadata, Buffering, and Atomicity

Pipes possess distinct metadata that can be queried. Executing `fstat()`, `lstat()`, or `stat()` on a pipe's file descriptor will describe the entity as a FIFO, identifiable programmatically via the `S_ISFIFO` macro.

Buffer Capacity and Atomicity Limits

The kernel imposes limits on pipe buffer sizes, which dictate how data writes are handled in concurrent scenarios. The constant `PIPE_BUF` (defined by including `limits.h`) specifies the exact size threshold of the pipe's buffer concerning atomic operations.

- **Atomic Writes:** Any `write()` request requesting to write an amount of data smaller than or equal to `PIPE_BUF` is guaranteed to be executed atomically. The data will be written in one continuous contiguous block. (Note: While atomic regarding data placement, the write system call itself can still technically be interrupted by a system signal).
- **Non-Atomic Writes:** Any `write()` request larger than the `PIPE_BUF` limit may be executed non-atomically. In scenarios with multiple writers pushing large datasets to the same pipe, this non-atomic behavior means the distinct data chunks from different writers can become interleaved and corrupted.

Dynamic Pipe Sizing (Linux)

Modern operating systems allow for the inspection and modification of these buffer limits. Since Linux kernel version 2.6.35, the default capacity allocated to a standard pipe is 16 pages of memory.

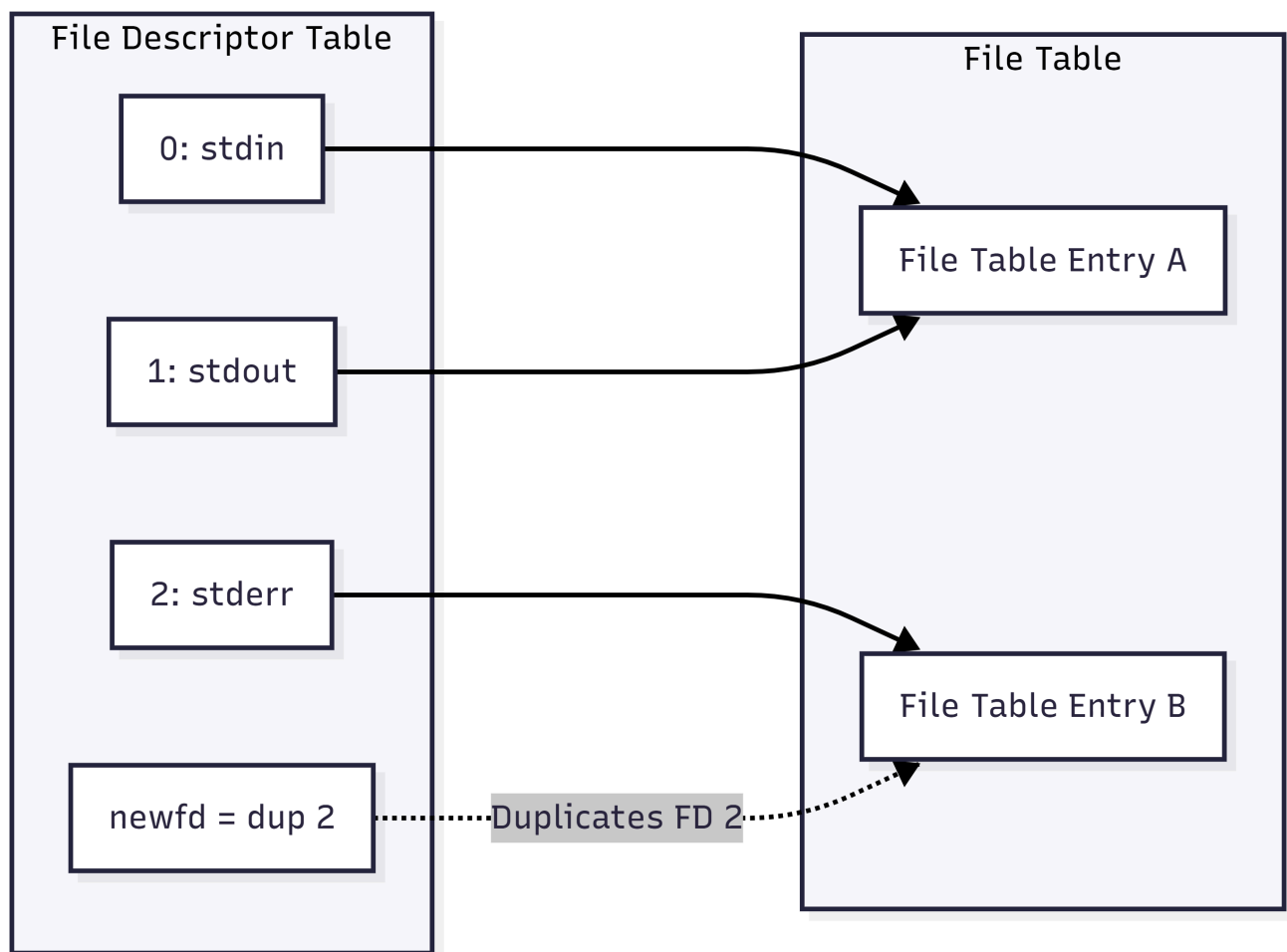
However, this capacity is not entirely fixed. It can be dynamically queried and altered using the `fcntl` system call by passing the `F_GETPIPE_SZ` flag (to read the size) and the `F_SETPIPE_SZ` flag (to modify the size). Importantly, increasing the pipe's capacity using `F_SETPIPE_SZ` does not require root/administrator privileges, provided the user is not attempting to increase the size beyond the absolute maximum limit defined by the host OS.

6. File Descriptor Duplication

To fully harness the power of pipes, particularly for redirecting standard input and output, operating systems provide mechanisms to duplicate existing file descriptors. This duplication is achieved using standard system functions:

- `int dup(int filedes);`
- `int dup2(int filedes, int filedes2);`

Both of these functions create a new file descriptor that points to the exact same underlying file table entry as the original, provided file descriptor. The operating system's file table entry contains a dedicated field that tracks a reference count, recording the total number of file descriptors that currently point to it.



[Photo: Copying the File Descriptor] This schema represents the

relationship between the File Descriptor Table and the kernel's internal File Table. When `newfd = dup(2)` is called, a new entry is created in the application's file descriptor table. Instead of opening a new file, this new descriptor is directed to point to the exact same File Table entry that file descriptor `2` currently references, effectively incrementing the reference count of that file table entry.

The `dup()` and `dup2()` Mechanisms

The two functions handle the assignment of the new descriptor slightly differently:

- **`dup()`** : This function automatically searches the file descriptor table and selects the lowest numbered free (unused) file descriptor. It assigns this newly selected descriptor to the original file table entry and returns the selected descriptor integer to the calling process.
- **`dup2()`** : This function provides more control by accepting the desired integer value for the new descriptor via the `filedes2` argument. If the file descriptor specified by `filedes2` is already open and in use, the system will automatically close it first before replacing it with the duplicate descriptor. It then returns the selected file descriptor.

7. Practical Application: Connecting Standard Streams

A classic problem in systems programming is managing standard output. For instance, consider a program (`prog1`) that writes an overwhelming amount of text to the standard output. If you want to display this output one page at a time (similar to utilizing the `more` command) without modifying the original source code of `prog1` , you can leverage pipes and descriptor duplication.

The solution involves using a third, managing program that orchestrates the connection:

1. The managing program creates a pipe and then spawns a child process using `fork()` .
2. **In the parent process:** The read end of the pipe and the standard output (`stdout` , usually `fd 1`) are explicitly closed. Standard output is then reassigned to the write end of the pipe using `dup2()` (or by combining `close()` and `dup()`). Finally, the parent process calls `exec()` to run `prog1` . Because `exec()` preserves file descriptors, `prog1` unknowingly writes its standard output directly into the pipe.

3. **In the child process:** The write end of the pipe and standard input (`stdin` , usually `fd 0`) are closed. Standard input is reassigned to the read end of the pipe using `dup2()` . The child then calls `exec()` to run `/usr/bin/more` . The `more` utility will now read its standard input directly from the pipe, seamlessly paging the output generated by `prog1` .

8. High-Level Pipes: `popen` and `pclose`

Because creating a pipe, forking a child, duplicating descriptors, and executing a shell command is such a common sequence of operations, the Standard C Library provides a streamlined function to handle this entirely.

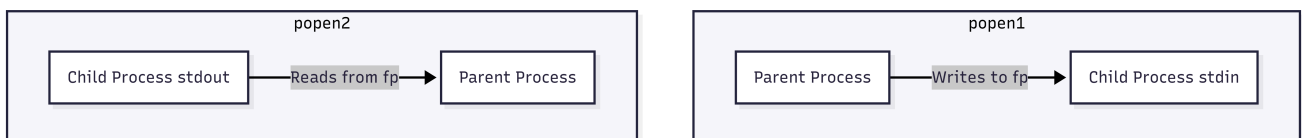
```
FILE *popen(char *cmdstring, char *type);
```

When invoked, `popen()` performs the following sequence:

1. Creates a standard pipe.
2. Creates a child process using `fork()` .
3. Automatically closes the unused ends of the pipe based on the requested mode.
4. Executes a shell environment (specifically using `sh -c cmdstring`) to run the requested command string.

The function abstracts raw file descriptors away, returning a standard I/O file pointer (`FILE *`). The connection direction depends entirely on the `type` argument provided:

- **If `type = "r"` :** The returned file pointer is used for reading. It is physically connected to the standard output of the child process running the command.
- **If `type = "w"` :** The returned file pointer is used for writing. It is securely connected to the standard input of the child process running the command.



To clean up resources established by this function, developers must use `pclose()` instead of the standard `fclose()` .

```
int pclose(FILE *fp);
```

This function safely closes the standard I/O file pointer returned by `popen()` and automatically blocks (using a `wait` mechanism) to capture and return the

termination status of the shell that was used to execute the command.

9. Interprocess Communication with Named Pipes (FIFOs)

While standard normal pipes are highly efficient, they are burdened by a severe limitation: they can only be used by processes that share a common ancestor, as this is the only biological way for file descriptors to be inherited.

To facilitate communication between completely unrelated processes, Unix provides **Named Pipes**, universally referred to as **FIFOs**.

Like standard pipes, FIFOs remain unidirectional. While a process theoretically could open a FIFO for both reading and writing simultaneously, doing so is problematic and highly discouraged; robust bidirectional communication requires the creation of two separate FIFOs.

The defining characteristic of a FIFO is that it exploits the standard file system interface. FIFOs possess a tangible "name" or pathname in the file system, complete with a directory entry and a dedicated inode. However, crucially, they do not allocate or use any physical data blocks on the disk; the data remains buffered in memory. When querying a FIFO with `stat()`, `lstat()`, or `fstat()`, the system will return true for the `S_ISFIFO` macro, correctly identifying its nature.

Managing Named Pipes

A FIFO is explicitly created programmatically using the `mkfifo()` function:

```
int mkfifo(char* pathname, mode_t mode);
```

This function creates the FIFO at the specified file path. The `mode` parameter defines the file access permissions using a standard octal number, operating identically to the permission models used in `open()` and `creat()`.

Once established on the file system, all subsequent input/output operations are performed exactly as they would be with standard files, utilizing `open()`, `read()`, `write()`, and `close()`. When the IPC channel is no longer required, the FIFO is permanently deleted from the filesystem using the standard `unlink()` call, respecting normal file access rights.

10. FIFO Operational States and Blocking Behavior

Because FIFOs decouple the creation of the pipe from the connection of processes, the `open()` and I/O system calls must handle scenarios where one end of the channel is missing. This behavior is heavily dictated by the presence or absence of the `O_NONBLOCK` flag.

Opening a FIFO

- **Without `O_NONBLOCK` flag (Default Blocking Mode):**
 - If a process opens the FIFO strictly for reading (read-only), the `open()` call will block and suspend the process until another independent process opens that same FIFO for writing.
 - If a process opens the FIFO strictly for writing (write-only), the `open()` call will block until another process opens it for reading.
- **With `O_NONBLOCK` flag:**
 - If the FIFO is opened read-only, the `open()` call succeeds and returns immediately, even if no writer is currently attached.
 - If the FIFO is opened for writing only, and absolutely no other process has opened it for reading, the `open()` call fails, returns immediately, and throws an `ENXIO` error.

Writing to a FIFO

When utilizing `write()` on a FIFO, the same `PIPE_BUF` atomicity limits apply as with regular pipes. Furthermore, if a process attempts to write to a FIFO and no other process currently has it open for reading, the system immediately generates a `SIGPIPE` signal. By default, this signal will terminate the writing process; if the signal is ignored or caught by a handler, the `write()` function will abort, return `-1`, and set `errno` to `EPIPE`.

Non-Blocking Behavior Summary Matrix

The specific effects of the `O_NONBLOCK` flag on Pipes and FIFOs are summarized below:

Condition	Default Behaviour (Blocking)	With <code>O_NONBLOCK</code> Flag
Open FIFO read-only (when no other process has it open to write)	Wait until a process opens the FIFO for writing.	Immediate return with no error.
Open FIFO write-only (when no other process has it open to read)	Wait until a process opens the FIFO for reading.	Immediate return with error, <code>errno</code> equal to <code>ENXIO</code> .
Read from empty pipe or FIFO	Wait until there is data left in the FIFO or no process has the FIFO open for writing.	Immediate return, return value equal to 0. <i>(Note: In POSIX standards, returning 0 generally indicates EOF when no writers exist. A non-blocking read on an empty pipe with active writers typically yields -1/EAGAIN, though the provided text emphasizes a 0 return value here).</i>
Write to full pipe or FIFO	Wait for space available to write, then write.	Immediate return, return value equal to 0.

11. Practical FIFO Applications: Complex Shell Pipelines

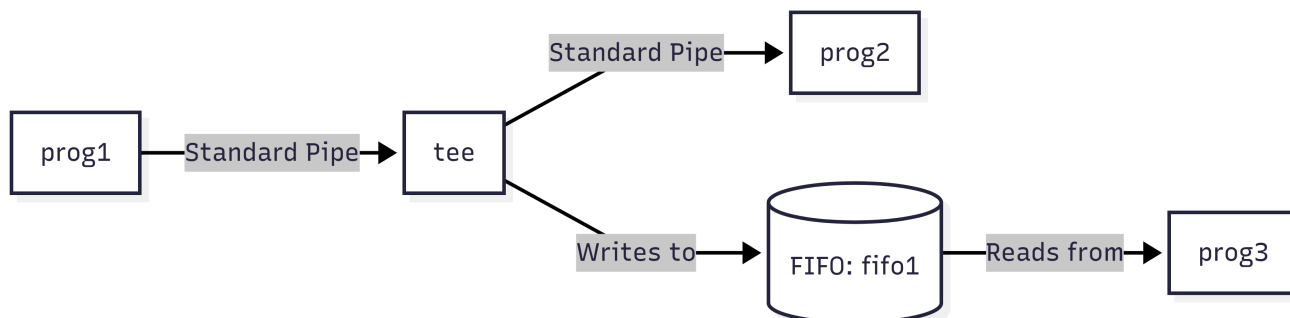
Beyond simple interprocess communication, FIFOs are exceptionally useful for orchestrating complex shell workflows. A primary use case is the ability to move data seamlessly from one active shell pipeline to another entirely separate pipeline, all without the need to create temporary intermediate files on the disk.

Consider a scenario where the output of a program must be processed by two different programs simultaneously. Using a standard pipe restricts you to a linear flow. However, by introducing a FIFO, you can branch the data stream.

This is accomplished through the following sequence of shell commands:

1. `mkfifo fifo1` (This creates the named pipe on the filesystem).

2. `prog3 < fifo1 &` (This launches `prog3` in the background, instructing it to read its standard input directly from `fifo1`).
3. `prog1 | tee fifo1 | prog2` (This executes `prog1` , piping its output into `tee` . The `tee` utility splits the stream, writing one copy into the named `fifo1` and piping the other copy into `prog2`).



[Photo: Data branching using a FIFO] This schematic illustrates the branching pipeline. The standard output of `prog1` flows into the `tee` command. `tee` acts as a splitter, sending one identical data stream via a standard pipe to `prog2` , and simultaneously writing a second identical stream into the named filesystem structure `FIFO: fifo1` . Meanwhile, `prog3` , running asynchronously, reads its input directly from this FIFO, effectively allowing `prog2` and `prog3` to process `prog1` 's output in parallel without intermediate disk storage.

12. Client-Server Architecture using FIFOs

FIFOs serve as an excellent foundational mechanism for building robust client-server applications on a local machine.

The Request Mechanism

The foundational step in this architecture requires the server to create a central FIFO. The pathname of this specific FIFO must be "well-known," meaning it is pre-configured or publicized so that all prospective clients know exactly where to locate it on the filesystem.

The workflow for submitting requests is straightforward:

- Multiple clients open the well-known FIFO and write their specific requests into it.
- The server process continuously monitors and reads these incoming requests from the same well-known FIFO.

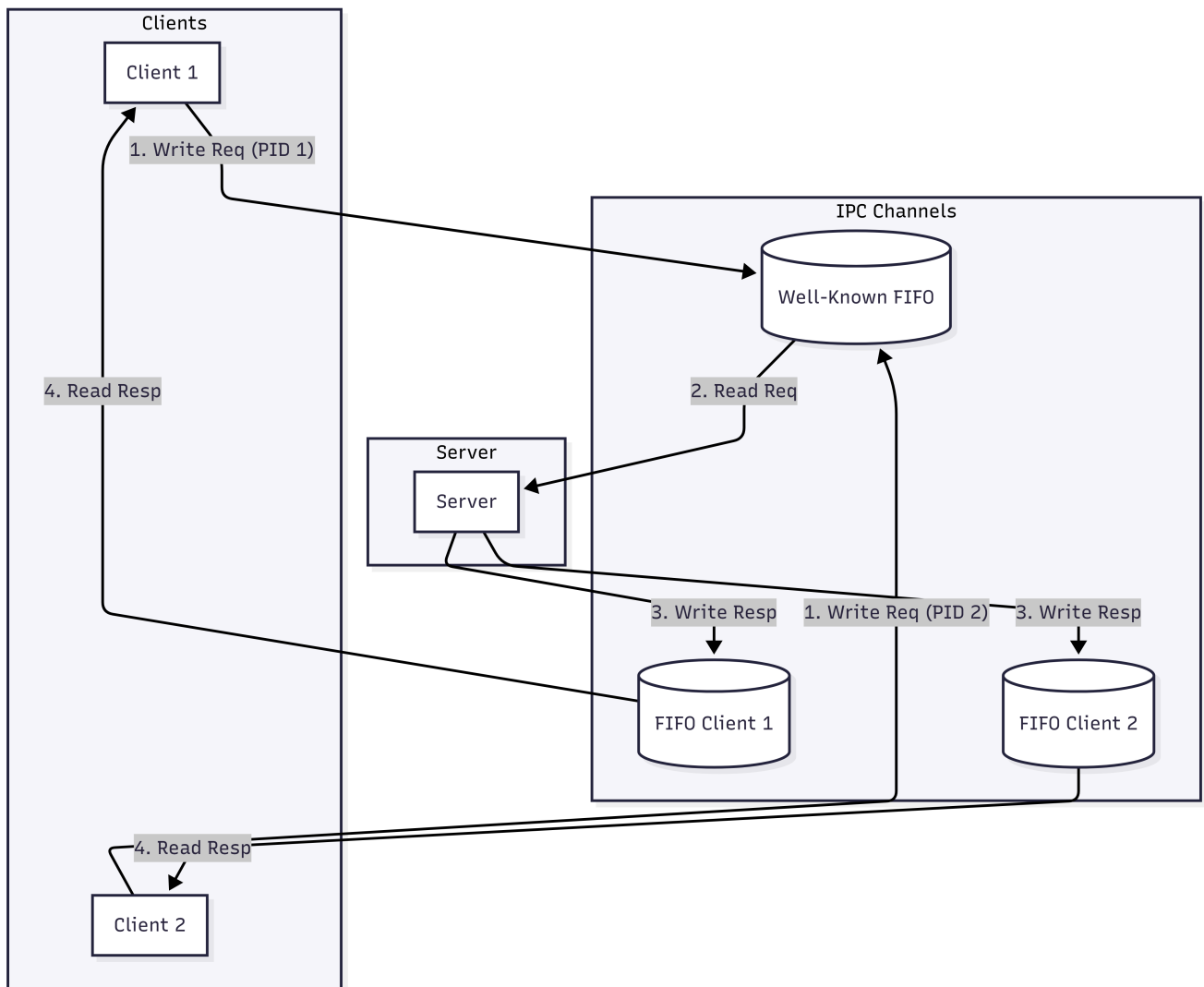
The Response Problem and Solution

While submitting requests is simple, returning responses poses a significant architectural challenge: How can the server accurately respond to individual clients?

You cannot use the same "well-known" FIFO to broadcast responses because the clients would have no mechanism to determine which pieces of data belong to them, nor would they know exactly when to read their specific response without intercepting data meant for others.

The definitive solution is to utilize dynamically generated, client-specific FIFOs.

1. Before submitting a request, each client creates its own unique FIFO (a "client FIFO") dedicated solely to receiving its response. To ensure the server can identify it, the client embeds its unique process ID (PID) into the name of this new FIFO, and then opens it for reading.
2. The client then writes its request to the server's well-known FIFO, ensuring it includes its PID within the data payload of the request itself.
3. The server reads the request, parses the PID, and dynamically reconstructs the file path to that specific client's FIFO.
4. The server then opens that specific client FIFO for writing and securely transmits the calculated response directly to the client.



Critical Caveats for Server Stability

When implementing this robust architecture, two major system-level caveats must be addressed to prevent the server from crashing or hanging:

- **Handling SIGPIPE :** The server process absolutely must capture or ignore the SIGPIPE signal. Because clients operate independently, a client might unexpectedly terminate or preemptively close its client FIFO before the server has the opportunity to write the response. If the server attempts to write to a closed client FIFO without handling SIGPIPE , the operating system's default action handler will instantly terminate the server process entirely.
- **The Read/Write Open Trick:** When the server initially opens its own "well-known" FIFO to listen for requests, it should purposefully open it for *both* reading and writing (O_RDWR), even though the server only intends to read from it. If the server opens it strictly for reading, as soon as the final connected client finishes and disconnects, the server will read an End-Of-File (EOF) marker. This causes the server to stop reading and shut down its listening loop instead of remaining active to wait for new clients. By keeping a write file

descriptor open on its own FIFO, the server ensures the channel always has at least one active writer (itself), which guarantees the read action remains continuously blocking and never triggers an EOF.

13. Practical Exercises for Mastery

To solidify the concepts of interprocess communication using pipes and FIFOs, the following practical programming exercises are recommended:

- **Exercise 1: FIFO Configuration Testing:** Write a C program named `test_fifo.c` designed to systematically verify the four possible operational configurations of a FIFO. The program should test opening the FIFO for reading versus writing, and it should test each state with and without the `O_NONBLOCK` flag applied, observing the resulting blocking behavior.
- **Exercise 2: The Producer-Consumer Paradigm:** Write a program that perfectly exemplifies the classic Producer-Consumer interaction model.
 - Utilize a named pipe (FIFO) as the central buffer.
 - The producer process must generate and write integers into the pipe, while the consumer process extracts and prints them.
 - To increase complexity, expand the system to include multiple manufacturers (producers) simultaneously writing to the same FIFO.
- **Exercise 3: Dedicated Forked Servers:** Write a program that extends the client-server communication scheme detailed previously. Instead of a single server handling all requests iteratively, have the main server spawn a dedicated child server (via the `fork()` system call) for every incoming client connection. Establish a private FIFO between the client (acting as the writer) and its newly dedicated server (acting as the reader). The client should read text line-by-line from standard input (`stdin`) and write it into the FIFO, while the dedicated server extracts the text from the FIFO and prints it directly to its standard output (`stdout`).

CHAPTER 9: Shared Memory

1. Introduction to Inter-Process Communication (IPC)

Inter-Process Communication (IPC) is a critical component in operating systems that allows processes to communicate and synchronize their actions. The System V (SYSV) implementation provides several advanced facilities for IPC, primarily

categorized into three mechanisms: Message Queues (which are outside the scope of this guide), Shared Memory, and Semaphores.

In the System V implementation, these three facilities share several common conceptual foundations regarding how they are named and managed. First, they exhibit a persistent lifetime, meaning the IPC objects remain in the system until explicitly destroyed or the system reboots. Second, they rely on the use of keys, specific identifiers, and dedicated data structures to manage access and metadata. Every System V IPC object is referenced by a non-negative integer known as a "key". The application program is entirely responsible for creating and managing this key, which uses the data type `key_t`.

The IPC Client-Server Paradigm and Keys

Keys act as the fundamental identifier for IPC resources, operating conceptually similar to how file descriptors work for file I/O. They provide the necessary routing to access specific kernel data structures, such as `struct shmid_ds` for shared memory segments or `struct semid_ds` for semaphores. Attempting to access an IPC resource improperly or with an invalid key will result in a return value of `-1`. Access rights for these mechanisms, such as read and write permissions, are securely stored within a permission structure named `struct ipc_perm`. Working with these structures requires including the `<sys/ipc.h>` and `<sys/types.h>` header files in your C programs.

In a standard Client-Server paradigm, keys can be generated and agreed upon in three primary ways:

- The server program can create a completely new IPC entity by specifying its key as `IPC_PRIVATE`. This macro instructs the operating system to generate a new, unique key that the user did not manually specify. However, because this key is inherently private to the creator, the client process must be explicitly informed of it, which is often accomplished by having the server write the newly generated key to a file that the client can read.
- The server and client processes can simply agree in advance on a hardcoded, pre-defined key value.
- The server and client can agree on a shared pathname pointing to an existing file in the file system, combined with a specific project-ID (an integer ranging from 0 to 255). They can then use the `ftok()` system call to convert these two agreed-upon values into a unique IPC key.

The `ftok()` function acts much like a hash function, mapping file names and project IDs into integers of type `key_t`. Its signature is `key_t ftok(const char *pathname, int proj_id)`. It successfully converts the provided pathname and project identifier into an IPC-key. For this to work, the file specified by the pathname (e.g., `/tmp/ad.tempfile`) must be readable by the process invoking the function. The operating system utilizes the metadata of that file, specifically its inode, to generate the resulting key. If `ftok()` fails, it returns `-1`.

2. System V Shared Memory

A shared memory region is a distinct portion of physical memory configured to be shared concurrently by multiple processes. By mapping this physical memory into the individual virtual address spaces of different processes, data structures placed within this region can be accessed concurrently without the overhead of copying data back and forth.

Conceptually, if Process A has a memory map ranging from address `0` to `0x30000`, a shared memory region can be mapped into its address space at a specific address (e.g., `0x30000` to `0x50000`). Concurrently, Process B can map the exact same physical shared memory region into its own memory map, perhaps at a different virtual address range like `0x50000` to `0x70000`. Because multiple processes can access this data simultaneously, strict synchronization is required to prevent data corruption; this synchronization is traditionally achieved using semaphores.

The Shared Memory API

To utilize System V Shared Memory, a process must interact with a specific set of system calls, requiring the inclusion of `<sys/ipc.h>` and `<sys/shm.h>`.

1. Creating or Accessing a Segment: `shmget()`

The `shmget()` function is used to allocate a shared memory segment or obtain the identifier of an existing one. Its signature is `int shmget(key_t key, size_t size, int shmflg)`. It returns the shared memory identifier associated with the provided `key` argument. The operating system allocates the segment such that its size is equal to the requested `size` rounded up to the nearest multiple of the system's page size. The `shmflg` argument dictates the access rights and creation behavior. The macro `IPC_CREAT` is used to request the creation of a new segment. The macro `IPC_EXCL` does nothing on its own, but when used in bitwise combination with `IPC_CREAT`, it dictates that the function must fail if a shared

memory segment for the given key already exists. If this collision occurs, `shmget()` fails and sets `errno` to `EEXIST`.

2. Attaching the Segment: `shmat()`

Once a segment is created, a process must attach it to its own virtual address space using `void *shmat(int shmid, const void *shmaddr, int shmflg)`. This attaches the segment identified by `shmid`. The `shmaddr` parameter allows the programmer to specify exactly where in the virtual memory the segment should be mapped. If `shmaddr` is provided, it must be a strictly page-aligned address. However, it is a highly frequent and recommended choice to set `shmaddr` to `NULL`, which allows the operating system to automatically choose a suitable, unused address to attach the segment. The function returns the exact address where the segment was attached, or `NULL` on failure.

3. Detaching the Segment: `shmdt()`

When a process is finished using the shared memory, it should detach it using `int shmdt(const void *shmaddr)`. This detaches the segment located at the specified `shmaddr` from the calling process's address space. It is critical to note that detaching a segment does *not* destroy it; the segment persists in physical memory.

4. Controlling the Segment: `shmctl()`

To modify, query, or destroy a shared memory segment, the `int shmctl(int shmid, int cmd, struct shmid_ds *buf)` system call is utilized. It performs the control operation specified by the `cmd` argument on the segment identified by `shmid`. The `buf` argument is a pointer to a `shmid_ds` structure (or `NULL` if not needed). The underlying kernel data structure `shmid_ds` tracks vital metadata, including ownership and permissions (`shm_perm`), the segment size in bytes (`shm_segsz`), timestamps for the last attach (`shm_atime`), last detach (`shm_dtime`), and last change (`shm_ctime`), as well as the Process ID of the creator (`shm_cpid`), the PID of the last process to call `shmat` or `shmdt` (`shm_lpid`), and the total number of current attaches (`shm_nattch`).

The most frequently used commands (`cmd`) for `shmctl()` are:

- `IPC_STAT`: Copies the information from the kernel data structure associated with `shmid` into the `shmid_ds` structure pointed to by the `buf` argument.

- `IPC_SET` : Writes the values of specific members of the user-provided `shmid_ds` structure (pointed to by `buf`) back into the kernel data structure, which also automatically updates its `shm_ctime` timestamp.
- `IPC_RMID` : Marks the shared memory segment to be destroyed. Importantly, the actual destruction is deferred; the segment will only be completely destroyed and its memory freed after the final process detaches from it (i.e., when `shm_nattch` reaches zero).

Usage Patterns and Best Practices

A typical lifecycle for shared memory involves a specific sequence of operations. First, a single process acts as the creator and allocates the segment using `shmget()` (often using `IPC_PRIVATE`, e.g., `shmget(IPC_PRIVATE, 10, 0666)`). Following creation, all processes that need access will attach the segment using `shmat(id, (void*)0, 0)`. Once processing is complete, all participating processes must detach the segment using `shmdt()`. Finally, a single designated process is responsible for marking the segment for removal using `shmctl(id, IPC_RMID, 0)`.

When utilizing shared memory, there are several strict architectural constraints to consider:

- Because segments can be attached at completely different virtual addresses in different processes, pointers within the shared segment are inherently unsafe. Developers must use offsets (relative addresses) rather than precise absolute addresses. For this reason, simple arrays are vastly easier to manage in shared memory than pointer-heavy data structures like linked lists or trees.
- The total number of shared memory segments available in a system is strictly limited by the OS. Therefore, they cannot be used as a drop-in replacement for `malloc()`.
- Shared memory provides no built-in synchronization; concurrent access *must* be heavily protected by mutual exclusion primitives like semaphores.
- Regarding process lifecycle: after a `fork()` call, the child process automatically inherits the attached shared memory segments. However, upon calling `exec()`, all shared segments are immediately detached from the process space.

Memory Mapping and File I/O (`mmap`)

Traditional file reading and writing rely on system calls like `open`, `read`, `write`, and `close`. This traditional approach forces explicit data transfers between buffers residing in the process address space and file blocks managed by the OS, which generates overhead due to multiple memory copy operations and frequent system calls.

An alternative and highly efficient approach is mapping the file directly into an empty region of the process's virtual address space. When mapped, Input/Output operations are handled directly by the Virtual Memory Manager (VMM). The system implicitly pages-in portions of the file as memory is read, and writes to this memory are eventually paged-out back to the disk. The primary result is a drastic reduction in kernel interaction, making the I/O significantly faster.

This is accomplished using the `mmap()` function: `void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset)`. It creates a new mapping in the caller's virtual address space.

- `addr`: A suggested starting address for the mapping, which can be `NULL` to let the OS decide.
- `length`: The total number of bytes to map.
- `prot`: Memory protection flags that dictate allowed accesses (these can be combined using bitwise OR `|`). Options include `PROT_READ` (allow read), `PROT_WRITE` (allow write), `PROT_EXEC` (executable memory), and `PROT_NONE` (no access, merely reserving the space).
- `flags`: Controls the core behavior of the mapping. `MAP_SHARED` dictates that any memory updates are visible to other processes mapping the same region and are written back to the underlying file. Alternatively, `MAP_PRIVATE` dictates that writing does not affect the original file (identified by the file descriptor `fd`). Under `MAP_PRIVATE`, after the first write operation, the kernel creates a copy of the written page (a "copy-on-write" mechanism), ensuring changes remain completely private to the process.
- `fd`: The open file descriptor to be mapped.
- `offset`: The specific offset within the file where the mapping should begin.

If successful, `mmap` returns a pointer to the newly mapped memory; on error, it returns the macro `MAP_FAILED`. To delete a mapping, `int munmap(void *addr, size_t length)` is used. It removes the mappings for the specified address range. Once unmapped, further references to addresses within that specific range will generate invalid memory reference faults (segmentation faults). It is completely

permissible to unmap a subset of a mapped region by providing a smaller length. `munmap` returns `0` on success and `-1` on error.

Introduction to POSIX IPC

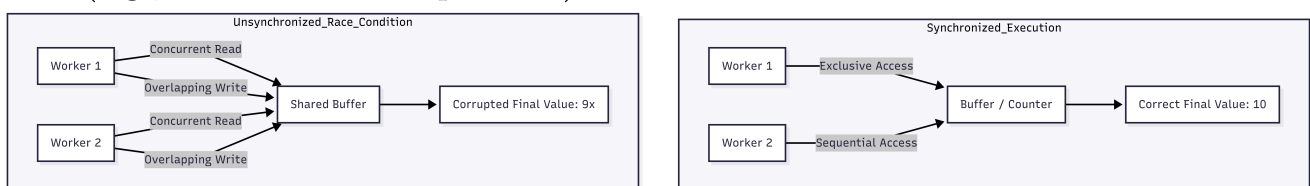
The POSIX.1b standard introduced three IPC mechanisms that are direct equivalents to those available in System V: Message Queues, Semaphores, and Shared Memory. The primary advantage of POSIX IPC is its significantly simpler Application Programming Interface (API). However, it comes with the drawback of less portability across older systems. To utilize POSIX shared memory, a developer relies on system calls like `shm_open()`, `shm_unlink()`, `ftruncate()`, and `close()`. Crucially, any shared memory created using POSIX IPC mechanisms still needs to be mapped into and out of the process's address space using the standard `mmap()` and `munmap()` functions.

3. Race Conditions and Semaphores

Understanding Race Conditions

When multiple processes or threads execute concurrently and share access to common resources—such as a shared memory segment—without proper synchronization, they can introduce a critical flaw known as a race condition. A race condition occurs when the final outcome of the program depends heavily on the precise, non-deterministic timing or interleaving of execution between the competing processes.

To visualize this risk, consider a comparative diagram illustrating memory updates across concurrent processes. In a properly synchronized scenario, multiple workers access a shared variable or counter sequentially, ensuring that increments (such as adding numbers like 8 and 10) are atomically preserved, resulting in a correct final total (e.g., 10). Conversely, when synchronization is absent, concurrent read-modify-write operations overlap. Process A and Process B might read the same initial value simultaneously, modify it locally, and write back conflicting updates, causing intermediate changes to be overwritten and resulting in corrupted final data (e.g., an erroneous output of 9).



Fundamentals of Semaphores

To prevent race conditions and guarantee the consistency of shared resources, operating systems use synchronization primitives known as semaphores. A semaphore is fundamentally a protected, non-negative integer whose value is never allowed to fall below zero. Two atomic operations can be performed on a semaphore:

- **UP (also known as SIGNAL, or V):** Increments the semaphore value by one. If there are processes blocked waiting on this semaphore, the operation wakes one of them up.
- **DOWN (also known as WAIT, or P):** Decrements the semaphore value by one. If the value of the semaphore is currently zero, the calling process will block and suspend execution until the value becomes greater than zero.

Formally, the mathematical definition of these operations can be expressed as:

- **Wait/Down (P):**

$$s = s - 1$$
$$\text{if } (s < 0) \rightarrow \text{block}()$$

- **Signal/Up (V):**

$$s = s + 1$$
$$\text{if } (s \geq 0) \rightarrow \text{wake}()$$

Classical Synchronization Problems

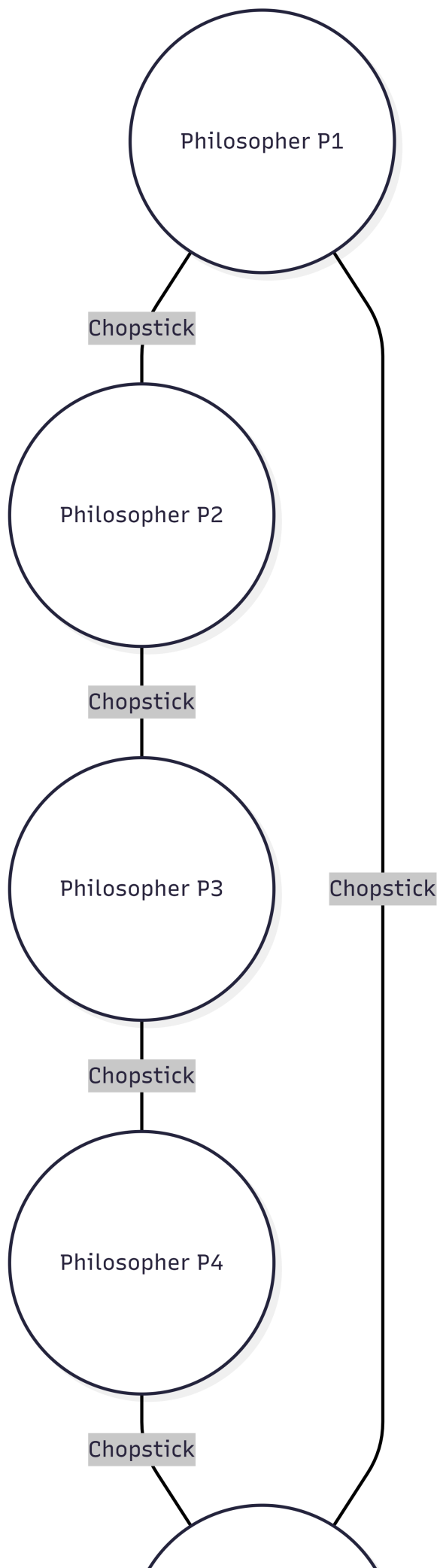
Concurrency challenges manifest in several classic theoretical problems, which serve as foundational blueprints for designing synchronized software systems.

1. Deadlock

A deadlock occurs when two or more competing processes are permanently blocked, each waiting for a resource that is held by another process in the set. For instance, consider Process A holding lock S_1 while attempting to acquire lock S_2 , while concurrently Process B holds lock S_2 and attempts to acquire lock S_1 . Because neither process can proceed without releasing its current lock, both execution threads hang indefinitely.

2. The Dining Philosophers Problem

This classical problem models the allocation of limited resources among concurrent processes to prevent deadlocks. K philosophers are seated around a circular table, with a single chopstick placed between each adjacent pair of philosophers. A philosopher may eat only if they successfully pick up *both* the left and right chopsticks adjacent to them. Since a single chopstick can be picked up by either of its two adjacent neighbors (but not both simultaneously), uncoordinated strategies can easily lead to a deadlock where every philosopher picks up their left chopstick and waits indefinitely for the right one.





3. The Sleeping Barber Problem

The Sleeping Barber problem illustrates management of bounded waiting rooms and synchronization between service providers and clients. A barbershop consists of one barber, one barber chair, and N chairs in a waiting room. If there are no customers present, the barber falls asleep in their chair. When a customer enters and finds the barber sleeping, the customer wakes the barber up. If the barber is busy cutting hair, arriving customers will either sit in an empty waiting chair or leave immediately if all waiting chairs are occupied.

4. The Producer-Consumer Problem

The Producer-Consumer problem involves a producer process generating items and placing them into a shared buffer of n slots, while a consumer process removes items from the buffer. To manage this bounded buffer safely, three distinct semaphores are required:

- n : Tracks the number of currently filled items in the buffer.
- e : Tracks the number of free, available entries in the buffer.
- s : A binary semaphore (mutex) that controls exclusive access to the buffer itself.

The pseudocode for the producer and consumer tasks demonstrates this precise synchronization:

Producer Pseudocode:

```
semaphore s = 1, n = 0, e = SIZE;
void producer(void) {
    while (1) {
        produce_item();
        P(e); // Wait for a free slot
        P(s); // Enter critical section
        enter_item();
        V(s); // Leave critical section
    }
}
```

```

        V(n); // Signal that a new item is available
    }
}

```

Consumer Pseudocode:

```

semaphore s = 1, n = 0, e = SIZE;
void consumer(void) {
    while (1) {
        P(n); // Wait for an available item
        P(s); // Enter critical section
        remove_item();
        V(s); // Leave critical section
        V(e); // Signal that a slot is now free
    }
}

```

5. Readers and Writers Problem

In the Readers and Writers problem, two types of workers interact with a shared buffer. Readers can inspect data items but cannot modify them, whereas writers can both read and write. Multiple concurrent readers are permitted to inspect the buffer simultaneously, but a writer requires exclusive, uninterrupted access to prevent corruption.

In reader-preference implementations, readers maintain a counter (`rd_count`) protected by a mutex. The first reader to arrive acquires the writer lock (`writeLock`), blocking any incoming writers until all readers have finished. Subsequent readers enter without acquiring the lock. The final reader to leave releases the writer lock.

Writer Task:

```

writerTask() {
    while (1) {
        P(writeLock);
        <<< perform write >>>
        V(writeLock);
    }
}

```

```

    }
}

```

Reader Task:

```

readerTask() {
    while (1) {
        P(mutex);
        rd_count++;
        if (rd_count == 1) P(writeLock); // 1st Reader gets
writer lock
        V(mutex);

        <<< perform task (reading) >>>

        P(mutex);
        rd_count--;
        if (!rd_count) V(writeLock); // Last Reader unlocks
writers
        V(mutex);
    }
}

```

4. System V and POSIX Semaphores API

System V Semaphores

System V provides a comprehensive API for managing semaphore sets, allowing multiple semaphores to be manipulated atomically as a single unit.

1. Creating or Accessing a Semaphore Set: `semget()` The `semget()` system call is used to obtain the identifier of a semaphore set associated with a specific key. Its signature is `int semget(key_t key, int count, int flag);`. The `key` parameter can be a fixed value, created using `ftok()`, or specified as `IPC_PRIVATE`. The `count` parameter defines the exact number of individual semaphores contained within the set. The `flag` parameter combines creation macros like `IPC_CREAT` with the least 9 bits representing access permissions.

2. Performing Semaphore Operations: `semop()` The `semop()` system call performs operations on selected members of the semaphore set indicated by `semid`. Its signature is `int semop(int semid, struct sembuf *sops, unsigned nsops);`. The `sops` argument points to an array of `sembuf` structures, where each element specifies an operation on a specific semaphore. The structure contains:

- `unsigned short sem_num`: The specific semaphore number within the set.
- `short sem_op`: The operation integer. If `sem_op > 0`, its value is added to the current semaphore value, waking up any suspended processes. If `sem_op == 0`, the calling process blocks until the semaphore value reaches zero (unless `IPC_NOWAIT` is set). If `sem_op < 0`, the process blocks until the semaphore value becomes greater than or equal to the absolute value of `sem_op`, and then subtracts the absolute value from the semaphore.
- `short sem_flg`: Control flags, such as `IPC_NOWAIT` (prevent blocking, returning `-1` and setting `errno` to `EAGAIN` if the action cannot be performed immediately) and `IPC_UNDO` (automatically undo operations when the process exits).

The entire array of operations contained in `sops` is performed in sequence, atomically, ensuring that either all operations succeed as a complete unit or none are executed.

3. Controlling Semaphores: `semctl()` The `semctl()` function performs control operations specified by `cmd` on the semaphore set identified by `semid` and a specific semaphore index `snum`. Its signature is `int semctl(int semid, int snum, int cmd, ...);`. Common commands include `IPC_SETVAL` or `IPC_GETVAL` to set or retrieve the value of a semaphore, and `IPC_RMID` to completely remove the semaphore set from the system.

System V IPC Management Commands Operating systems provide command-line utilities to inspect and manage System V IPC objects:

- `ipcs`: Displays active information regarding System V inter-process communication facilities.
- `ipcmk`: Allows manual creation of System V IPC objects from the terminal.
- `ipcrm`: Removes System V IPC objects and their associated data structures from the system.

POSIX Semaphores

POSIX.1b introduced alternative semaphore mechanisms designed to offer a simpler API. POSIX semaphores can be implemented either as anonymous (unnamed) semaphores stored in shared memory or as named semaphores identified by a filesystem-like string path.

Anonymous POSIX Semaphores An anonymous semaphore is initialized using `int sem_init(sem_t *sem, int pshared, unsigned int value);`. The `pshared` argument determines whether the semaphore is shared between threads within a single process (`pshared = 0`, located at an address visible to all threads) or among multiple processes (`pshared != 0`, requiring the semaphore to be located within a region of shared memory).

The primary operations for POSIX semaphores include:

- `sem_wait(sem_t *sem)` : Performs a blocking $P()$ (wait) operation.
- `sem_trywait(sem_t *sem)` : Performs a non-blocking variant of the $P()$ operation, failing immediately if the semaphore cannot be decremented.
- `sem_post(sem_t *sem)` : Performs a V (signal) operation, incrementing the semaphore and waking waiting processes.
- `sem_destroy(sem_t *sem)` : Destroys the semaphore when it is no longer needed.

Named POSIX Semaphores Named semaphores are created or opened using `sem_open()`. Their signatures are:

- `sem_t *sem_open(const char *name, int oflag);`
- `sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);`

The `oflag` argument controls creation behavior, utilizing flags like `O_CREAT` to create the semaphore or `O_EXCL` to ensure failure if the semaphore already exists. Named semaphores are identified by a name consisting of a slash followed by a sequence of characters. They maintain an inode entry in a virtual filesystem simulated by the kernel (visible under `/dev/shm`), though they occupy no actual disk space.

To close a named semaphore handle, a process calls `int sem_close(sem_t *sem)`, which frees system resources similarly to closing a file descriptor. To completely remove the named semaphore from the system namespace, a process calls `int sem_unlink(const char *name)`. The current value of a semaphore can be queried using `int sem_getvalue(sem_t *sem, int *sval)`.

Pitfalls and Risks of Named POSIX Semaphores

Named semaphores introduce two major architectural risks that developers must carefully manage:

1. Deadlock Risk on Process Termination If a process holding or waiting on a semaphore crashes or receives an uncatchable signal, there is no automatic recovery mechanism to wake up other waiting processes, potentially causing indefinite blocking. While anonymous semaphores residing in shared memory restrict this risk to the specific subset of processes sharing that memory segment, named semaphores are system-wide, persistent entities that can impact any running process.

2. Name-Based Conflicts and Unlink Race Conditions If a named semaphore is unlinked via `sem_unlink()` while processes still hold open handles to it, subtle race conditions can occur. For example, if Process A creates a semaphore named `/mysem`, unlinks it, but keeps its handle open, the name disappears from the virtual filesystem while Process A continues using its active reference. If Process B attempts to open `/mysem` during this window, the kernel treats it as non-existent and creates a brand-new, independent semaphore under the exact same name. This results in unrelated processes operating on completely separate semaphore states while believing they share synchronization, breaking application consistency.

CHAPTER 10: Sockets

Introduction and History of Sockets

The concept of the "socket" was originally invented at the University of California, Berkeley, as a core component of the BSD (Berkeley Software Distribution) flavor of the Unix operating system. It rapidly established itself as the standard methodology for facilitating communication over the internet. Today, the Berkeley Socket API is universally supported across almost all modern operating systems, including Windows, Linux, Mac, and Android. Because the interface was later adopted by the POSIX standard, modern sockets are widely considered to be POSIX-compliant.

The historical timeline of socket development spans several decades:

- **1971:** The specific term "socket" first appeared in the ARPANET document RPC147.

- **1983:** The 4.2BSD release officially included Berkeley Sockets. This implementation was funded by DARPA to create a functional API for their new suite of network protocols.
- **1989:** The University of California, Berkeley (UCB) released their network library to the public, entirely free of licensing constraints.
- **1991:** Microsoft Windows defined and introduced its own socket-based network API, known as WinSock.

The Basic Ideas of Network Communication

Standard operating systems rely heavily on File I/O operations—such as `open`, `close`, `read`, `write`, `seek`, and `fcntl`—to interact with local data. Network communication expands upon this paradigm by utilizing standard file descriptors for network connections. In this model, standard functions like `read()` and `write()` are extended to work directly on these network descriptors. However, creating and managing network connections requires advanced capabilities (such as addressing and protocol selection) which do not neatly fit into the traditional 'open/read/write/close' paradigm. This discrepancy necessitated the creation of the dedicated Socket API.

A **socket** serves as the fundamental building block for all network communication. Fundamentally, a socket is an endpoint of communication to which a specific name (or address) may be bound. Every socket currently in use is characterized by a specific type and is associated with one or more running processes.

[Photo: Network Communication] This structural diagram illustrates the endpoint nature of sockets. On one side of a network cloud, "Host X" (with an IP address of 146.86.5.2) utilizes a socket specifically bound to the combination of its IP and port 1625 (represented as 146.86.5.2/1625). This socket connects across the network to a "web server" (IP 161.25.19.8) using a socket bound to port 80 (represented as 161.25.19.8/80), forming a complete communication pipeline.

The Client/Server Paradigm

Socket use is heavily predicated on the Client/Server paradigm. In this model, the **Server** performs a "passive open," meaning it sets up a socket and actively listens for incoming requests generated by clients. The **Client**, conversely, performs an "active open" by reaching out to initiate a connection with a specific server.

[Photo: Client/Server Communication Paradigm] A visual representation showing a single, centralized "SERVER" box receiving inbound "request" arrows

from multiple peripheral "CLIENTS" circles, and sending outbound "response" arrows back to them, demonstrating the one-to-many relationship of a passive listener and active requestors.

Common examples of servers relying on this paradigm include file servers, web servers, print servers, mail servers, name servers, and X window servers.

Communication Domains

Sockets do not exist in a vacuum; they operate strictly within designated "communication domains". A communication domain is an abstract concept introduced to bundle together the common properties of processes that are communicating through sockets, particularly dictating how sockets are named and addressed. Sockets normally only exchange data with other sockets located within the exact same domain.

The 4.4BSD Inter-Process Communication (IPC) facilities historically supported four separate communication domains:

1. **Internet Domain (IPv4/IPv6):** Used by processes communicating over the internet using standard protocols like TCP and UDP. For example, all IPv4 sockets communicate using a combination of IP addresses and Ports, and they can only communicate with other IPv4 sockets.
2. **UNIX Domain:** Used for localized, on-system communication (IPC). In this domain, sockets are named using standard UNIX file path names (e.g., `/dev/foo`). These act as highly efficient alternatives to standard UNIX Pipes.
3. **NS Domain:** Historically used by processes communicating via the Xerox standard communication protocols, though this is now completely obsolete.
4. **ISO OSI Domain:** Utilized for OSI standard protocols.

Socket Types

Sockets are strictly typed according to the communication properties that are visible to the user. The system presumes that processes will only communicate between sockets that share the same type. There are four primary types of sockets currently available:

1. **Stream Sockets:** A stream socket provides a flow of data that is bidirectional (you can both read and write from it), reliable (all transmitted data is guaranteed to arrive), sequenced (data arrives in the exact order it was sent), and unduplicated (no data is delivered or accepted twice). Importantly, stream

sockets have *no record boundaries*; the data acts as a continuous stream, meaning a single large `receive` operation can read data accumulated from multiple separate `send` operations. Aside from the bidirectional nature of the data flow, a connected pair of stream sockets provides an interface nearly identical to that of UNIX pipes. Stream sockets are an abstraction that can be implemented by different underlying protocols (such as TCP) that meet these strict requirements. When using a stream socket, the user simply specifies the protocol, and the socket autonomously handles all packet encapsulation, decapsulation, and header construction.

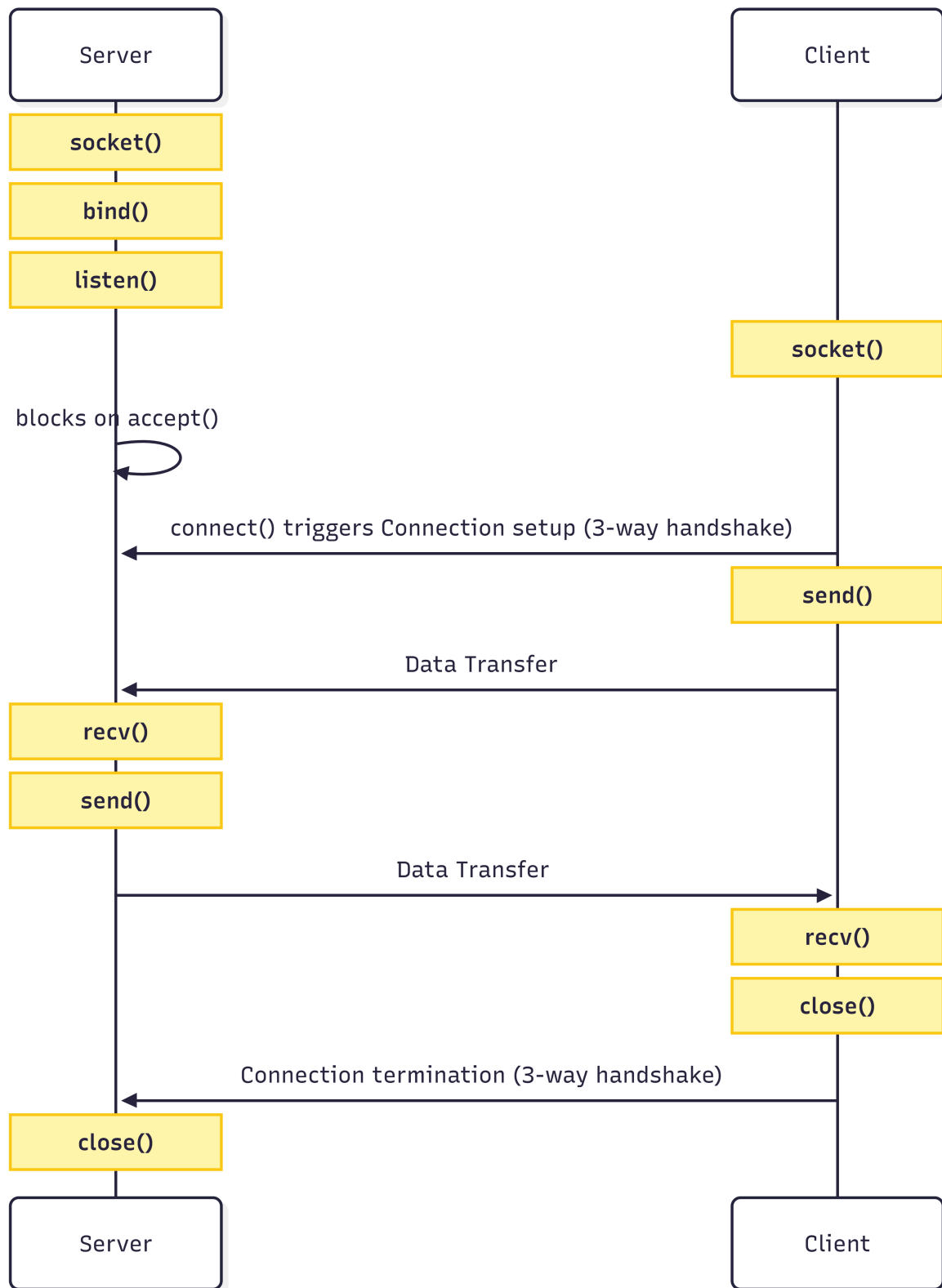
2. **Datagram Sockets:** A datagram socket supports a bidirectional flow of data, but unlike stream sockets, it is *not* promised to be sequenced, reliable, or unduplicated. Messages sent via datagram sockets may be dropped, duplicated, and potentially delivered in an entirely different order than they were sent. A defining characteristic of a datagram socket is that *record boundaries are strictly preserved*. Every individual `send` operation corresponds directly to a specific `receive` operation to ensure data is read in a distinct datagram-like fashion. Like stream sockets, users specify the protocol (typically UDP), and the socket handles the low-level packet construction.
3. **Raw Sockets:** A raw socket bypasses the standard encapsulations to provide users with direct access to the underlying communication protocols that support socket abstractions. Users are responsible for building their own packets from scratch, including the creation of custom network headers. These sockets are inherently datagram-oriented and are not intended for general users. They are highly useful for network inspection (building sniffers), developing entirely new communication protocols, or gaining access to esoteric facilities of existing protocols (like ICMP).
4. **Sequenced Packet Sockets:** This type is functionally similar to a stream socket (reliable, sequenced), with the distinct exception that *record boundaries are preserved*. Each `send` directly corresponds to a `receive` to read data in a packet-like fashion. This was provided as part of the obsolete NS socket abstraction and is rarely used nowadays.

The Stream Sockets Flow (Based on TCP)

To fully understand how Stream Sockets operate, we must look at the lifecycle of a TCP connection between a client and a server.

[Photo: Stream Sockets Flow based on TCP] This structural diagram outlines the function calls required on both the Server side and Client side to

establish, utilize, and terminate a stream socket connection.



Chapter 2: Socket Creation, Naming, and Binding

Initializing a Socket

A socket is created initially without a name or address; it acts merely as a reference point until properly configured. To create a socket, a process uses the `socket()` system call, which returns a small integer known as a file descriptor. This descriptor is subsequently used to reference the socket in all future network operations.

The syntax for creating a socket is `s = socket(domain, type, protocol);`.

- **Domain:** Specifies the communication domain. Common domains include `AF_UNIX` (for Unix Domain IPC), `AF_INET` (for IPv4 Internet Domain), `AF_INET6` (for IPv6 Internet Domain), and the obsolete `AF_NS` (NS Domain).
- **Type:** Dictates the communication properties, such as `SOCK_STREAM` (for reliable, stream-based connections), `SOCK_DGRAM` (for datagrams), `SOCK_RAW` (for raw protocol access), or `SOCK_SEQPACKET` (for sequenced packets).
- **Protocol:** Requests a specific underlying protocol to implement the socket type. Common protocols include `IPPROTO_TCP` (which strictly requires `SOCK_STREAM`), `IPPROTO_UDP` (requires `SOCK_DGRAM`), `IPPROTO_ICMP` and `IPPROTO_RAW` (both require `SOCK_RAW`), and `IPPROTO_IPV6` (typically `SOCK_RAW`).

If the protocol argument is left unspecified (by passing a value of `0`), the system automatically selects the default protocol from those available within the chosen communication domain that match the specified socket type. For instance, `s = socket(AF_INET, SOCK_STREAM, 0);` creates an IPv4 stream socket and automatically defaults to TCP. Similarly, `s = socket(AF_INET, SOCK_DGRAM, 0);` defaults to UDP, and `s = socket(AF_UNIX, SOCK_DGRAM, 0);` creates a local UNIX datagram socket. In practice, using `0` as the default protocol is correct for almost every standard situation.

Socket Names and Uniqueness

Until a socket is assigned a name, processes have no way to reference it, which means no messages can be sent or received. Communicating processes establish a connection by forming an **association**—a pairing of local and remote names.

To ensure data routes correctly, these association tuples must be strictly unique across the system. In the Internet domain, an association is composed of a five-part tuple: `<protocol, local address, local port, foreign address, foreign port>`. There can never be duplicate tuples in the network stack. In the UNIX domain, the association relies on file paths rather than IP addresses:

`<protocol, local pathname, foreign pathname>`. When bound, there may never be a duplicate UNIX domain association.

Binding Names to Sockets

The `bind(s, name, namelen);` system call allows a process to specify its "half" of the association tuple (the local machine's identity). For Internet domains, this is the `<local address, local port>` pair; for UNIX domains, it is the `<local pathname>`. Later, functions like `connect()` and `accept()` are used to complete the socket's full association by defining the remote half.

Because sockets operate across different domains, the `name` argument passed to `bind()` is a variable-length byte sequence (a structure) that changes depending on the required protocol.

IPv4 Addressing (`sockaddr_in`)

In the Internet domain, names are encoded using the `sockaddr_in` structure (where `"_in"` stands for Internet Domain), defined within the `<netinet/in.h>` header.

```
struct sockaddr_in {
    sa_family_t    sin_family; // Address family: AF_INET for
    // IPv4
    in_port_t      sin_port;   // Port number (16-bit) in network
    // byte order
    struct in_addr sin_addr;    // IPv4 address in network byte
    // order
    char           sin_zero[8]; // Padding to make struct size
    // equal to generic sockaddr
};

struct in_addr {
    uint32_t s_addr; // 32-bit IPv4 address, in network byte
    // order
};
```

When preparing this structure, standard practice involves setting the `sin_family` to `AF_INET`, using `htons()` to convert the port to network byte order, and using `INADDR_ANY` to bind the socket to all available local network interfaces. Crucially,

the `sin_zero` array exists purely for padding (backward compatibility with the generic `sockaddr` size) and should be zeroed out using `memset()`. Once populated, the structure is cast to a generic `(struct sockaddr *)` when passed to `bind()`.

IPv6 Addressing (`sockaddr_in6`)

For IPv6 addresses, the system relies on the `sockaddr_in6` structure, also found in `<netinet/in.h>`.

```
struct sockaddr_in6 {
    sa_family_t    sin6_family;    // Address family: AF_INET6
    in_port_t      sin6_port;      // Port number (16-bit,
network byte order)
    uint32_t       sin6_flowinfo;  // Flow information (usually
0)
    struct in6_addr sin6_addr;     // IPv6 address (128-bit)
    uint32_t       sin6_scope_id;  // Scope ID (useful for link-
local addresses)
};

struct in6_addr {
    unsigned char  s6_addr[16];    // 128-bit IPv6 address array
};
```

UNIX Domain Addressing (`sockaddr_un`)

In the UNIX domain (IPC), names contain a pathname and a family descriptor, encoded by the `sockaddr_un` structure defined in `<sys/un.h>`.

```
struct sockaddr_un {
    sa_family_t sun_family; // Address family: AF_UNIX (or
AF_LOCAL)
    char        sun_path[]; // Path to socket file in the
filesystem (null terminated)
};
```

When binding a UNIX socket, the `addr.sun_path` is actually created as a tangible socket file in the host system's file space. Therefore, the calling process

must possess write permissions in the target directory. The file is essentially an i-node stored on disk—persisting even after a system restart, much like named pipes and differing from named POSIX semaphores—despite not utilizing any actual data blocks. For a `bind()` to succeed on a local UNIX name, that file *must not exist* prior to the call. Once communication is permanently finished, this file should be manually deleted. Due to the variable length of the path string, calculating the `namelen` argument for `bind()` requires dynamically sizing it:

```
strlen(addr.sun_path) + sizeof(addr.sun_family) .
```

Chapter 3: Stream Connection Setup and Data Transfer

Establishing a stream connection is an inherently asymmetric process; one process assumes the role of the client, while the other acts as the server.

Client-Side Connection (`connect()`)

The client requests services from a server by initiating a connection to a specific address using the `connect()` system call. The server argument passed to `connect()` contains either the Internet address and port or the UNIX pathname of the target server.

Similar to `bind()` , the size calculation for the length parameter differs by domain: IPv4 uses standard `sizeof(server)` , whereas UNIX domain calls must again calculate the string length plus family size. Interestingly, if the client process has an unbound socket (meaning it never explicitly executed `bind()`) when it calls `connect()` , the operating system will automatically select and bind an ephemeral name/port to the socket behind the scenes.

If the connection is unsuccessful, `connect()` returns an error. However, any name that was automatically bound by the system remains attached to the socket even after a failure. If successful, the socket is officially associated with the server, and data transfer may commence.

Server-Side Connection (`listen()` and `accept()`)

Before a server can receive a client's connection, it must perform two critical steps after binding its socket to a well-known address.

1. **`listen(s, backlog);`** : The server informs the operating system that its socket is ready to accept incoming connection requests. The second parameter

dictates the maximum number of outstanding connections that the kernel will queue up while awaiting acceptance. For example, `listen(s, 5);` permits a queue of up to 5 unaccepted requests.

2. **`accept(s, (struct sockaddr *)&from, &fromlen);`** : This function pulls a pending connection from the queue. `accept()` is a blocking call; it completely halts the process until a connection becomes available or until the system call is interrupted by a signal.

Upon successful receipt of a connection, `accept()` returns a brand-new file descriptor (`newsock`) representing that specific client connection. The original socket (`s`) continues to act strictly as a passive listener for other new clients. Furthermore, `accept()` populates the `from` structure with the address of the remote socket connecting to it. The `fromlen` parameter is a value-result argument: on input, it dictates how much space is available in the `from` struct, and on output, it is modified to reflect the actual size of the name returned by the client. If the server does not care about the client's address, it can simply pass a null pointer for `from`.

Note on Server Filtering: There is no built-in mechanism within `accept()` to specify that it should only accept connections from a specific source. If a server wishes to restrict access, it must fully accept the connection first and subsequently perform IP or path filtering manually before proceeding.

Data Transfer and Socket Termination

Once a stream connection is fully established, processes can rely on normal file I/O system calls like `read(s, buf, sizeof(buf));` and `write(s, buf, sizeof(buf));`. However, the Socket API provides specialized functions: `send(s, buf, len, flags);` and `recv(s, buf, len, flags);`.

These functions allow developers to pass specific routing and behavioral flags:

- **`MSG_OOB`** : Sends or receives out-of-band (urgent) data. Certain protocols, like TCP, support sending a small amount of urgent data separately from the normal data stream, signaling to the receiver that this specific byte requires immediate attention.
- **`MSG_PEEK`** : Used strictly with `recv()`, this flag allows the user to look at the data currently present in the buffer. The data is returned to the user but remains on the system queue as "unread". A subsequent `read()` or `recv()` call will pull the exact same data.

- **MSG_DONTROUTE** : Tells the system to send data without utilizing standard packet routing protocols. Data is sent directly to the local link and is only delivered if the destination is explicitly on that local link (distance of 1). This is heavily used for UDP, as TCP inherently establishes routes during its handshake process.

When a socket is no longer of interest and communication is complete, it must be destroyed. The standard method is calling `close(s)`, which entirely terminates the connection and frees the descriptor. However, it is also possible to execute a "half-close" using `shutdown(s, how)`. Depending on the `how` flag, `shutdown` can make the socket unidirectional. The options include `SHUT_RD` (closes the read end), `SHUT_WR` (closes the write end), or `SHUT_RDWR` (closes both ends). Importantly, even if `shutdown()` is used, the file descriptor remains allocated and open; the programmer must still call `close(s)` later to properly free system resources.

Chapter 4: Datagram Sockets (UDP)

While stream sockets provide a reliable, sequenced pipeline for data, datagram sockets prioritize speed and boundary preservation over strict reliability. Operating primarily over the User Datagram Protocol (UDP), datagram sockets are connectionless by design. This means that client and server addresses are passed along with each individual message sent from one process to another, facilitating true peer-to-peer communication without the overhead of maintaining a persistent virtual circuit.

The Datagram Communication Flow

Because datagram sockets are connectionless, the standard stream-oriented functions like `listen()` and `accept()` are completely bypassed. There is no queue of incoming connections to manage. Instead, processes read and write packets directly to and from the socket.

When operating in a purely connectionless manner, developers use specialized I/O functions:

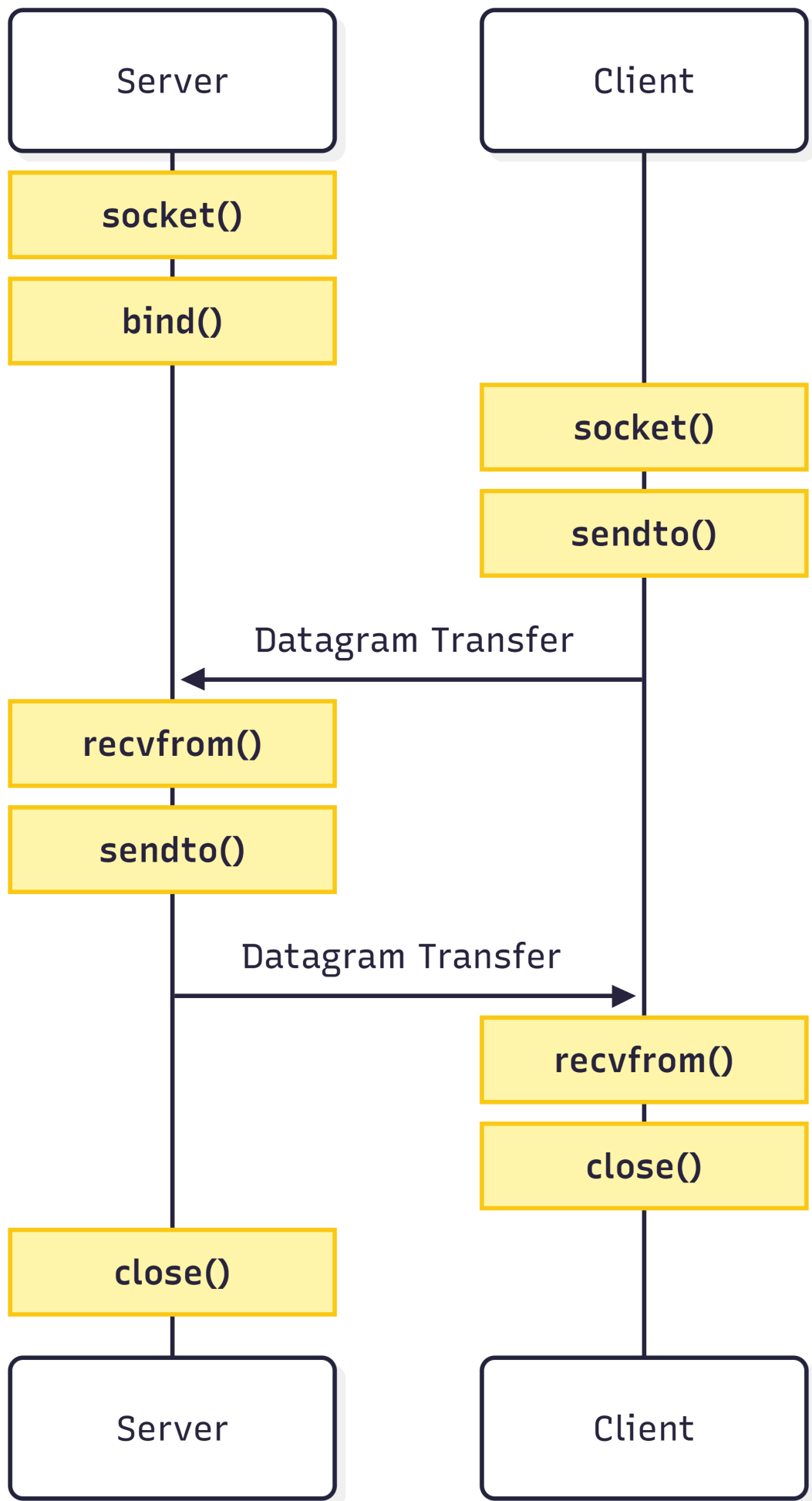
- **sendto()** : This function is used to transmit data, requiring the programmer to explicitly pass the exact destination address (e.g., a `sockaddr_in` struct) alongside the payload for every single transmission.
- **recvfrom()** : This function receives messages on an unconnected datagram socket. When a packet arrives, the operating system populates a provided

`sockaddr` struct with the sender's identity, allowing the receiving process to know exactly where the data originated. By changing the address structures used in these functions, a single datagram socket can seamlessly multiplex communication with multiple different sources and destinations.

[Photo: Datagram Sockets Flow]

This flowchart illustrates the lifecycle of a datagram (UDP) interaction. The server binds a socket and immediately drops into a loop of `recvfrom()` and `sendto()`. The client simply creates a socket and begins transmitting via

`sendto()` , *awaiting replies with* `recvfrom()` .



"Connecting" a Datagram Socket

Although UDP is connectionless, the `connect()` system call can still be used on a datagram socket. However, its behavior is fundamentally different from when it is applied to a TCP stream socket.

When `connect()` is called on a UDP socket, the function returns immediately without initiating any network handshake. The operating system simply records the peer's remote address locally. Only one connected address is permitted per datagram socket at any given time; executing a second `connect()` call simply overrides the previous destination.

The primary advantage of "connecting" a datagram socket is convenience. Once an address is registered via `connect()`, the programmer is no longer required to use `sendto()` and `recvfrom()`. Instead, they can utilize standard `read()` / `write()` or `send()` / `recv()` functions, as the kernel already knows the intended destination and source for all packets moving through that specific socket descriptor.

Chapter 5: Name Resolution and Network Formatting

To construct functional socket addresses, applications must routinely map human-readable hostnames (like `[www.google.com](https://www.google.com)`) to raw IP addresses, and service names (like `http`) to specific port numbers. The `netdb` library provides the critical routines required to perform these mappings.

The BSD Socket API Resolution (Legacy)

Historically, the BSD API relied on a set of independent functions to resolve different network components.

Host Names

The `gethostbyname(const char *name)` function takes a standard hostname string and returns a pointer to a `hostent` structure. There is also a `gethostbyaddr()` function intended to map raw IP addresses back into a `hostent` structure, though this is largely considered obsolete today.

```
struct hostent {  
    char *h_name;           // Official name of the host
```

```

    char **h_aliases;    // Null-terminated array of alternate
names
    int     h_addrtype;  // Host address type (e.g., AF_INET)
    int     h_length;    // Length of address (4 bytes for IPv4)
    char **h_addr_list;  // Null-terminated list of network
addresses
};
#define h_addr h_addr_list[0] // Macro for convenience to get
first address

```

Service Names

Similarly, `getservbyname(const char *name, const char *proto)` maps a service name (e.g., "telnet") and protocol (e.g., "tcp") to a `servent` structure. Its counterpart, `getservbyport()`, works in reverse.

```

struct servent {
    char *s_name;    // Official service name
    char **s_aliases; // Alias list
    int    s_port;    // Port number (returned in network byte
order)
    char *s_proto;    // Protocol to use
};

```

Protocol Names

If a developer does not know the integer identifier for a specific protocol to pass into the `socket()` creation call, they can use `getprotobyname()` (e.g., passing "tcp") to retrieve a `protoent` structure, which contains the `p_proto` integer.

The POSIX Resolution API (Modern)

Modern POSIX systems combine the functionality of host and service resolution into a single, unified interface: `getaddrinfo()`.

```

int getaddrinfo(const char *node, const char *service,
                const struct addrinfo *hints, struct addrinfo
**res);

```

This function resolves both hostnames (the `node`) and ports (the `service`) simultaneously. The programmer can provide a `hints` structure to specify preferences (e.g., requesting only IPv4 addresses). The results are returned as a dynamically allocated linked list of `addrinfo` structures pointed to by `res`. Because `getaddrinfo` allocates this memory on the heap, the programmer must explicitly free it using `freeaddrinfo()` when finished. The inverse function, `getnameinfo()`, converts raw addresses and ports back into strings.

```
struct addrinfo {
    int          ai_flags;      // Input flags (AI_PASSIVE,
    etc.)
    int          ai_family;     // Address family (AF_INET,
    AF_INET6)
    int          ai_socktype;   // Socket type (SOCK_STREAM,
    etc.)
    int          ai_protocol;   // Protocol (IPPROTO_TCP,
    etc.)
    socklen_t    ai_addrlen;    // Length of ai_addr
    struct sockaddr *ai_addr;    // Pointer to generic sockaddr
    char         *ai_canonname; // Canonical name for hostname
    struct addrinfo *ai_next;    // Pointer to next item in
    linked list
};
```

Endianness and Network Byte Order

Endianness refers to the sequential order in which bytes of a multi-byte data field are arranged in computer memory.

- **Big Endian:** The most significant byte is stored at the lowest memory address (making it the first byte to be read).
- **Little Endian:** The least significant byte is stored at the lowest memory address. Most modern consumer processors, including Intel CPUs and Apple Silicon, utilize a Little Endian architecture.

Despite the prevalence of Little Endian host machines, standard **Network Byte Order** is strictly **Big Endian**. Consequently, a programmer cannot simply transmit multi-byte values (like shorts, ints, or longs) derived directly from local

memory into a packet header. Doing so without conversion will completely corrupt routing data.

To solve this, the API provides macros to convert data between the host's order and the network's order. These conversions must be applied to every multi-byte value used to build packet headers, such as ports and IP addresses. If the host machine happens to already be Big Endian, these macros are implemented to do nothing, guaranteeing that the code remains highly portable across any architecture.

- `htonl(val)` : **Host TO Network Long** (converts a 32-bit quantity).
- `htons(val)` : **Host TO Network Short** (converts a 16-bit quantity, typical for ports).
- `ntohl(val)` : **Network TO Host Long** (converts a 32-bit quantity back to host order).
- `ntohs(val)` : **Network TO Host Short** (converts a 16-bit quantity back to host order).

Note on Payload Data: The actual payload data (the content of the packet) can theoretically be transmitted as-is without any endian conversion, because it is generally sent byte-by-byte (e.g., cast as an array of characters). However, this only functions correctly if both communicating peers utilize the exact same byte order. To avoid catastrophic portability issues, it is strongly recommended to always convert multi-byte payload values before transmission and after reception.

IP Formatting Helpers

To aid in building IP addresses, `<arpa/inet.h>` provides translation tools:

- `inet_pton(int af, const char *src, void *dst)` : Converts a readable string representing an IP (like "192.168.0.1") directly into a network byte order binary format, storing it in `dst`.
- `inet_ntop(int af, const void *src, char *dst, socklen_t size)` : The reverse operation, translating binary network addresses back into human-readable strings.

Chapter 6: Multiplexing and Advanced I/O

When a server must handle multiple client connections simultaneously, relying on blocking read/write calls is fatal to performance; a process blocked waiting for

data from Client A cannot respond to data arriving from Client B. The best solution for I/O multiplexing is the `select()` system call.

The `select()` System Call

`select()` enables a process to simultaneously monitor a set of file descriptors, intentionally pausing execution only until one or more of those descriptors become ready for a specific I/O operation.

```
int select(int nfd, fd_set *readfds, fd_set *writefds,
           fd_set *exceptfds, struct timeval *timeout);
```

Using `select()` provides three critical pieces of information:

1. The total number of file descriptors that are currently ready for I/O.
2. Which specific descriptors are ready for a **read** (calling `read()` on them will not block).
3. Which specific descriptors are ready for a **write** (calling `write()` on them will not block).
4. Which specific descriptors have an **exception** condition. In the context of the socket library, the only exceptional condition currently implemented is the arrival of Out-Of-Band (OOB) TCP data.

Descriptor Sets (`fd_set`)

The sets of file descriptors to be checked are managed using the `fd_set` data type. Typically, this structure uses a bitmask where one bit represents one specific file descriptor. Because directly manipulating bitmasks is error-prone, developers must use a standardized suite of macros to manage these sets:

- `FD_ZERO(fd_set *fdset)` : Completely zeroes out (initializes) the set.
- `FD_SET(int fd, fd_set *fdset)` : Adds a specific descriptor `fd` to the set.
- `FD_CLR(int fd, fd_set *fdset)` : Removes a specific descriptor from the set.
- `FD_ISSET(int fd, fd_set *fdset)` : A testing macro that returns non-zero if the requested descriptor is currently flagged within the set. This is heavily used after `select()` returns to identify exactly which sockets triggered the wake-up.

The `nfds` argument passed to `select()` must be set to the value of the highest file descriptor being monitored, plus one (`max_fd + 1`).

Managing the Timeout

The final argument to `select()` is a pointer to a `timeval` structure, which determines how long the call will wait before returning:

- **Infinite Wait:** If `timeout` is explicitly set to `NULL`, the call blocks indefinitely until at least one descriptor is ready or an error occurs.
- **Polling Mode:** If the timeout structure is passed with both seconds and microseconds set to `0`, `select()` operates as a non-blocking poll. It rapidly checks all requested descriptors and returns immediately, regardless of readiness.
- **Finite Wait:** If the timeout is set to a value greater than zero, the function will block until a descriptor is ready, an error occurs, or the specified timer fully elapses (resulting in a return value of `0`).

Chapter 7: Socket Options and Graceful Shutdown

Socket Shutdown vs. Close

Normally, a socket is terminated using the standard `close()` function, which completely destroys the connection. However, it is possible to perform a half-close, rendering the socket strictly half-duplex, by using the `shutdown(s, how)` function.

The `how` parameter dictates the behavior:

- `SHUT_RD` : Closes the read end, preventing further data reception.
- `SHUT_WR` : Closes the write end, preventing further data transmission.
- `SHUT_RDWR` : Closes both ends, effectively halting all communication.

It is highly important to understand that `shutdown()` does not free the file descriptor; the descriptor remains open, and the programmer is still required to call `close()` later to clean up system resources. Furthermore, if a remote peer closes its write end of a stream socket (e.g., TCP), any subsequent `read()` operations by the local machine will return `0`. A return value of `0` is the standard indicator of an End-of-File (EOF) scenario. Code must always check for this `0` return value to detect remote closures and subsequently close the local

socket properly. (Note: Because datagram sockets are connectionless, they have no formal concept of an EOF).

Manipulating Socket Options

The API allows programmers to fine-tune socket behavior—such as altering buffer sizes or enabling specialized protocol features—using `setsockopt()` and `getsockopt()`.

These functions target options at specific stack levels. The `level` argument defines where the option applies:

- `SOL_SOCKET` : Universal options that apply to all sockets regardless of type.
- `IPPROTO_IP` / `IPPROTO_IPV6` : Options specific to the IP routing layer.
- `IPPROTO_TCP` / `IPPROTO_UDP` : Options specific to the transport protocols.

Common `SOL_SOCKET` level options include:

- `SO_REUSEPORT` : Allows multiple sockets to bind to the exact same port.
- `SO_BROADCAST` : Enables the socket to send data to network broadcast addresses.
- `SO_SNDBUF` / `SO_RCVBUF` : Allows manual resizing of the send and receive buffers.
- `SO_LINGER` : Alters the default behavior of the `close()` system call when unsent data remains in the buffer.

Deep-Dive: The `SO_LINGER` Option

By default, `SO_LINGER` is disabled (off). Under default conditions, when `close()` is called, it returns immediately to the caller. In the background, the operating system attempts a graceful shutdown by trying to flush all unsent data remaining in the local socket buffer to the remote peer before finishing the TCP FIN/ACK handshake. The major drawback here is that if the application program terminates abruptly after `close()` returns, pending data may be permanently lost because the kernel is interrupted before it finishes transmitting the buffer.

Enabling `SO_LINGER` alters this flow. By using a `struct linger` and setting `l_onoff = 1`, the programmer can deliberately delay (block) the return of the `close()` function.

- If `l_linger > 0`, `close()` will block execution for up to the specified number of seconds. This heavily increases the probability that all data will successfully transmit before the application is allowed to exit, as it gives the kernel explicit time to flush the buffer.
- If `l_linger = 0`, `close()` will immediately abort the connection, and any unsent data residing in the buffer is instantly discarded. This is a hard abort, differing significantly from the default behavior that at least attempts a background flush.

Chapter 8: Non-Blocking I/O and Socket Signals

Enabling Non-Blocking I/O

Beyond `select()`, another approach to handling asynchronous network traffic is configuring the socket descriptor itself to be entirely non-blocking. This is achieved using the `fcntl()` system call to modify the file status flags by appending the `O_NONBLOCK` flag.

Once a socket is marked non-blocking, the behavior of standard I/O functions changes drastically:

- **Write Operations:** If a function like `send()` or `write()` cannot immediately process the entire payload, it will send whatever data it can instantly fit into the buffer, and the return value will indicate the exact partial amount sent.
- **Read Operations:** If a `read()` or `recv()` is attempted but the receive buffer is completely empty, the function will not block the process. Instead, it will immediately return `-1` and set the global `errno` variable to `EAGAIN` (or `EWouldBlock` on older Unix systems, which are functionally identical in Linux).

Signal-Driven I/O (`SIGIO`)

Instead of constantly polling a non-blocking socket to see if data has arrived, a process can instruct the operating system to proactively notify it via signals when the socket has pending I/O. This relies on the `SIGIO` signal.

Setting up signal-driven I/O requires a strict three-step process:

1. **Install a Handler:** Use the `signal()` system call to bind a custom handler function to the `SIGIO` signal.
2. **Assign Ownership:** Use `fcntl()` with the `F_SETOWN` command to assign the socket's ownership to the current process ID (`getpid()`). This tells the kernel exactly which process (or process group) should receive the notification signals for this specific socket.
3. **Enable Asynchronous Notifications:** Finally, use `fcntl()` with the `F_SETFL` command to apply the `O_ASYNC` (or `FASYNC`) flag to the socket. This step officially activates the mechanism, telling the kernel to emit the `SIGIO` signal to the owner whenever new I/O is ready.

Because signals are highly asynchronous, it is possible for two distinct packets to arrive in rapid succession while generating only a single `SIGIO` interrupt. Consequently, the signal handler must loop (often combining this with `select()` and non-blocking reads) until it definitively clears all ready sockets to ensure no data is left unprocessed.

Slow vs. Fast Functions and `EINTR`

When a signal is delivered to a process, its effect heavily depends on what the process is currently executing. Socket functions are categorized into two types:

- **Fast Functions:** These execute quickly and are never interrupted by a signal. The system waits until the function completes before raising the signal.
- **Slow Functions:** These are functions that can potentially block indefinitely waiting for network interaction (e.g., `accept()` , `blocking read()` , `recvfrom()`).

If a signal arrives while a process is blocked inside a slow socket function, the function is immediately interrupted. It stops processing and returns `-1` with `errno` set to `EINTR` . In standard behavior, the user program is responsible for manually catching this error and explicitly re-initiating the interrupted I/O call. Alternatively, programmers can configure the signal handler with the `SA_RESTART` flag. This instructs the OS to automatically restart the interrupted system call seamlessly. (Note: there are strict exemptions; for instance, an interrupted `connect()` call will never automatically restart, even if `SA_RESTART` is present).

Socket-Specific Signals

Beyond `SIGIO` , the socket API can generate two other crucial signals:

- **SIGURG** : This signal indicates an urgent condition is present, specifically the arrival of out-of-band (OOB) data on a TCP connection. To receive **SIGURG** , a process must map a handler and set ownership via **F_SETOWN** , but it does *not* need to enable **O_ASYNC** (as that flag only governs general I/O readiness notifications).
- **SIGPIPE** : This highly destructive signal is automatically generated by the kernel when a process attempts to **send()** or **write()** data into a broken socket (e.g., a socket where the remote peer has closed the connection, or one that has been locally **shutdown()**). The default system action for **SIGPIPE** is to immediately terminate the offending process.

Chapter 9: Raw Socket Programming

Network Packet Encapsulation and Standard Sockets

To understand raw sockets, it is vital to first understand how network data is packaged. Data sent over a network undergoes **encapsulation**. When an application sends data, it is progressively wrapped in specific protocol headers as it travels down the network stack. The raw application data is encapsulated within a TCP or UDP header to form a segment or datagram, which is then wrapped in an IP header to form an IP packet, and finally wrapped in an Ethernet header to form a frame for physical transmission.

When working with **Standard Sockets** (like standard stream or datagram sockets), the operating system abstracts this entirely. Standard sockets can only receive frames that are explicitly addressed to them (either to their specific address, or via broadcast/multicast). Furthermore, as the packet moves up the stack, the OS automatically strips away the Ethernet, IP, and TCP/UDP headers. The socket object delivers only the raw application payload to the user. Consequently, with standard sockets, packet headers cannot be manually modified or inspected before sending or after receiving.

Advanced Functions: Promiscuous Mode and Raw Sockets

For advanced network diagnostics, security monitoring, or custom protocol development, programmers require lower-level access.

Promiscuous Mode

Promiscuous mode is the "See All, Hear All" setting for a network interface card (NIC). It instructs the network driver to accept and process *all* frames in the broadcast domain, irrespective of whether the packets are actually addressed to that specific host machine. This mode is heavily utilized for network monitoring (both legal diagnostics and illegal sniffing). You can enable promiscuous mode in two ways:

- **Programmatically:** By setting the `IFF_PROMISC` flag via system calls.
- **Command Line Interface (CLI):** Using commands such as `ifconfig <iface> promisc` or `ip link set <iface> promisc on`.

Raw Sockets and Packet Injection

While promiscuous mode ensures the hardware accepts all packets, **Raw Sockets** ensure the software can see them intact. A raw socket allows an application to receive complete packets, including all lower-level network headers, without the OS stripping them away.

Conversely, raw sockets also allow for **Packet Injection**. This allows developers to manufacture their own packets entirely from scratch—crafting custom headers and data—and inject them directly into the network. This provides a total bypass of the operating system's standard network stack. Most active network monitoring tools and hacking utilities rely on packet injection to execute Denial of Service (DOS) attacks, SYN Floods, and IP Spoofing.

Implementing PF_PACKET

In Linux, the `PF_PACKET` family provides a software interface to send and receive packets directly at Layer 2 (the Data Link layer, communicating directly with the device driver). All packets received on a `PF_PACKET` socket will be completely whole (headers included), and all packets sent will be transmitted exactly as formulated to the medium, without any modification by the kernel. `PF_PACKET` also supports advanced traffic filtering using Berkeley Packet Filters (BPF).

To create a raw socket at this level, the `socket()` system call requires specific arguments:

```
socket(PF_PACKET, SOCK_RAW, <protocol>);
```

The `<protocol>` argument specifies the type of traffic to capture:

- `ETH_P_IP` : Configures the socket to capture only IP packets.
- `ETH_P_ALL` : Configures the socket to capture absolutely all information passing through the interface.

Sniffer and Injector Workflows

Developing a custom network sniffer involves the following strict sequence of operations:

1. Create a raw socket using `socket(PF_PACKET, SOCK_RAW, ...)` .
2. Set the target network interface into promiscuous mode to capture ambient traffic.
3. Bind the raw socket to this specific interface using `bind()` .
4. Continuously receive packets on the socket using `recvfrom()` .
5. Process and inspect the received packets.
6. Close the raw socket using `close()` when finished.

Developing a packet injector follows a similar, though shorter, flow:

1. Create a raw socket using `socket()` .
2. Bind the socket to the desired outgoing interface using `bind()` .
3. Create the custom packet in memory, manually writing the Ethernet, IP, and Transport headers.
4. Send the packet to the wire using `sendto()` .
5. Close the raw socket using `close()` .

Chapter 10: Deep-Dive into the TCP/UDP/IP Protocol Suite

Understanding how to program sockets requires a fundamental understanding of the protocols that dictate how data traverses the Internet. The Internet Assigned Number Authority (IANA) is the governing body responsible for coordinating the key elements that keep the internet running. IANA maintains Domain names, manages Number Resources (such as global IP and Autonomous System (AS) pools), and handles Protocol Assignments. (You can view assigned protocols on a Unix system by inspecting the `/etc/protocols` file).

The Underlying Network and IP

At the core of the suite is the Internet Protocol (IP), which operates at the network layer. IP is fundamentally an unreliable, best-effort datagram delivery service. Because the underlying network operates on a "best-effort" basis, it is inherently prone to dropping messages, re-ordering messages, delivering duplicate copies, limiting message sizes, and delivering messages after arbitrarily long delays. End-to-end transport protocols (like TCP and UDP) sit on top of IP to mitigate or utilize these behaviors.

The User Datagram Protocol (UDP)

UDP is a transport layer protocol that acts as a simple demultiplexor. It provides a fast, minimal, unreliable, and unordered datagram service between processes.

- **Pros:** It carries significantly less overhead than TCP, making it incredibly fast and efficient for stateless communication.
- **Cons:** It lacks built-in reliability. There is no flow control, no sequence tracking (random delivery), and no acknowledgment of receipt, meaning there is no inherent way to know when to resend a lost packet. Connections exist only long enough to deliver a single packet.

UDP endpoints are identified by ports (with servers typically using well-known ports listed in `/etc/services`).

[Photo: UDP Packet Format] This structural diagram defines the UDP header layout. It is highly minimalist, featuring a Source Port Address (16 bits) and Destination Port Address (16 bits) forming the first 32-bit row. The second row contains the Checksum (an optional pseudo-header checksum to guarantee integrity) and the Length of DATA. This header sits directly above the actual DATA payload.

The Transmission Control Protocol (TCP)

TCP provides a connection-oriented, full-duplex, reliable, byte-stream delivery service. It is designed to solve the complex end-to-end issues of network routing, such as differing Round Trip Times (RTT) requiring adaptive timeouts, network delays, differing buffering capacities at destinations, and network congestion.

TCP guarantees the delivery of packets in their exact order of transmission. It achieves this sequenced delivery by adding a sequence number to every single packet. The receiver must explicitly acknowledge (ACK) that the data arrived safely; if the sender does not receive an ACK, it resends the unacknowledged data.

TCP also applies flow control to keep the sender from overrunning the receiver, and congestion control to keep the sender from overrunning the network infrastructure.

The TCP Segment Format

[Photo: TCP Segment Format]

This block diagram details the complex TCP header structure. It includes:

- *Row 1: Source Port Number (16 bits) / Destination Port Number (16 bits)*
- *Row 2: Sequence Number (32 bits)*
- *Row 3: Acknowledgement Number (32 bits)*
- *Row 4: Header Length (4 bits) / Padding/0 (6 bits) / Flags (6 bits) / Window (16 bits)*
- *Row 5: Checksum (16 bits) / Urgent Pointer (16 bits)*
- *Row 6: Options/Padding*
- *Row 7: Data Payload*

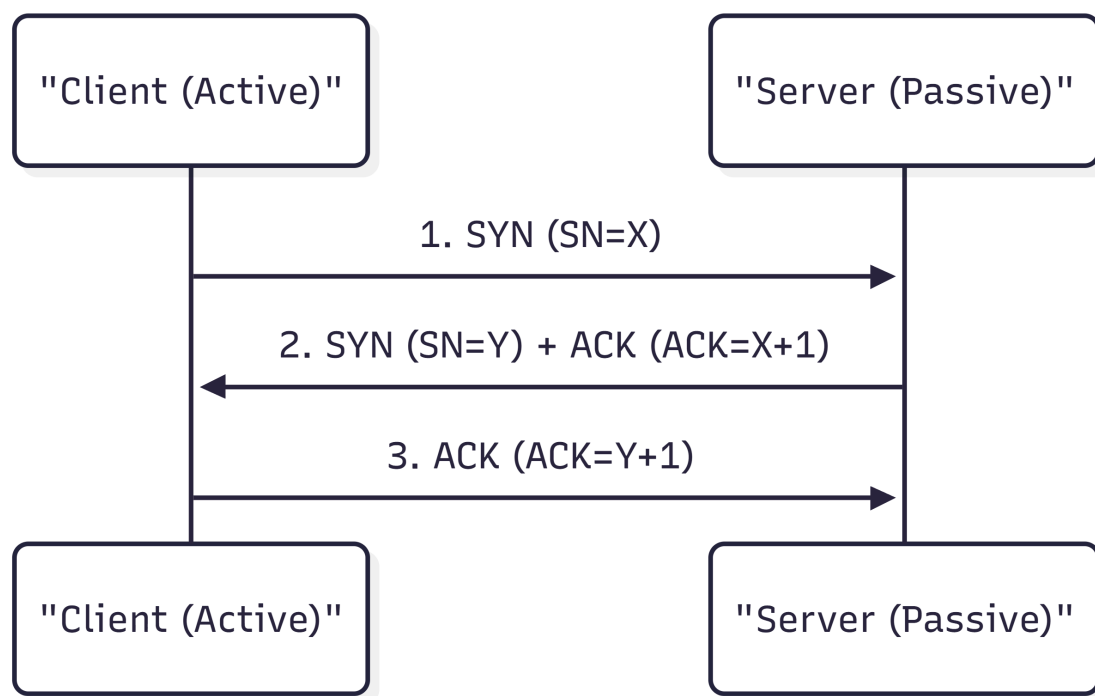
Critical TCP Header Fields:

- **Sequence Number (SN):** Refers to the specific byte-number of the *first byte* of data included in the segment.
- **Acknowledgement Number (ACK):** Indicates the byte-number of the *next expected data byte*. It signals that all bytes up through this number have successfully been received.
- **Control Flags:**
 - **URG :** Urgent data is included (triggers **SIGURG** on the receiver).
 - **ACK :** Indicates this segment is acting as an acknowledgement.
 - **RST :** Reset. An error occurred, and the session is being aborted.
 - **SYN :** Synchronize Sequence Numbers (used exclusively during connection setup).
 - **FIN :** Polite connection termination.
- **Window:** Sent with every ACK, this field tells the sender exactly how many bytes of buffer space remain available before the receiver will overflow, establishing dynamic flow control.

TCP Connection Establishment (The 3-Way Handshake)

When a client application requests a connection, TCP orchestrates a strict 3-way handshake to synchronize Sequence Numbers (SN).

1. **Client SYN:** The active participant (client) sends a SYN segment to the server. This packet contains a pseudo-randomly generated Client SN (Let's call it X), the maximum receive window for the client, and only TCP headers (no payload).
2. **Server SYN-ACK:** The passive participant (waiting server) receives the request and replies with its own SYN segment. This contains the Server's pseudo-random SN (Y), an Acknowledgement Number set to $X+1$ (confirming receipt of the client's SN), the server's receive window, and only TCP headers.
3. **Client ACK:** The client receives the server's SYN and finalizes the connection by sending an ACK segment, setting the Acknowledgement Number to $Y+1$.



TCP Buffering and Piggybacking

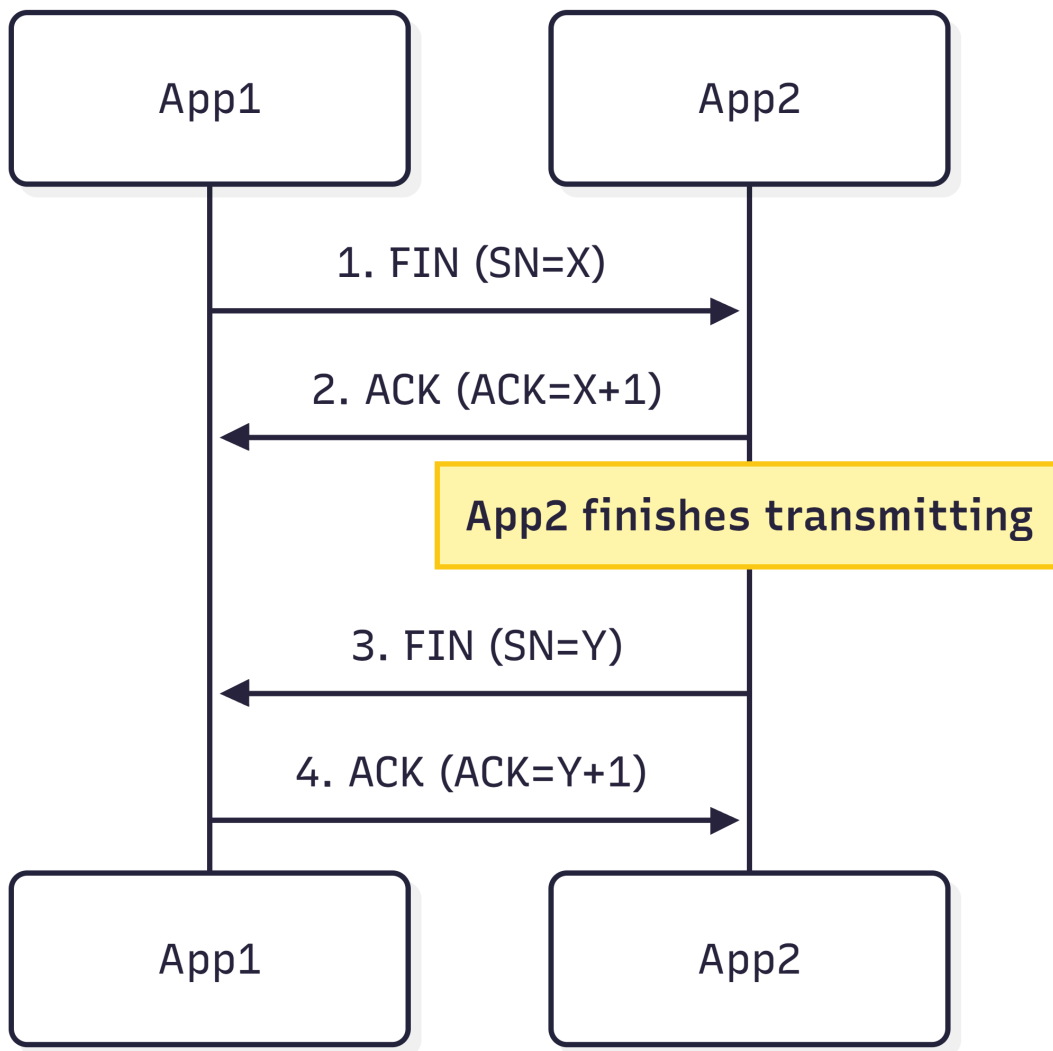
Because the TCP layer never knows exactly when the application process will ask for received data, it buffers incoming data so it is ready when requested. Both the client and server allocate these send and receive buffers automatically. When an application writes data to TCP, it sits in the send buffer until an ACK is received from the remote peer confirming delivery. The receiver does not have to acknowledge every single segment individually; it can acknowledge many segments simultaneously with a single cumulative ACK segment (especially helpful if out-of-order segments arrive and are reassembled in the buffer). Furthermore, an ACK

can also contain outgoing payload data, a highly efficient optimization known as **piggybacking**.

TCP Termination

TCP connections can be terminated via an abortive **RST** segment if an error occurs, but usually, applications execute a graceful shutdown utilizing **FIN** segments. Either end can initiate termination.

1. Application 1 (e.g., Client) decides it is done sending data and issues a **FIN** segment ($SN=X$).
2. Application 2 (e.g., Server) receives the **FIN** and acknowledges it by sending an **ACK** ($ACK=X+1$). The connection is now half-closed (App 1 cannot send data, but can still receive).
3. When Application 2 finishes its own transmissions, it sends its own **FIN** segment ($SN=Y$).
4. Application 1 receives the **FIN** and acknowledges it with a final **ACK** ($ACK=Y+1$). The connection is now fully terminated.



Chapter 11: Background Reference on Socket Structures

To guarantee absolute robustness in socket programming, developers must memorize the exact constraints of the data structures utilized across the API.

1. General Socket Address (`struct sockaddr`) This is the generic structure passed to almost all socket calls requiring an address pointer.

- `unsigned short sa_family` : Denotes the address family (e.g., `AF_INET` , `AF_UNIX` , `AF_NS` , or `AF_IMPLINK`).
- `char sa_data[14]` : 14 bytes of protocol-specific address data. Its exact contents depend entirely on the `sa_family` (for IP, it holds the port and IP address).

2. Internet Socket Address (`struct sockaddr_in`) The specific structure used for IPv4 Internet domain sockets.

- `sin_family` : Always set to `AF_INET` .
- `sin_port` : A 16-bit port number (must be converted to Network Byte Order via `htons()`).
- `sin_addr` : A 32-bit IP address structure (must be in Network Byte Order).
- `sin_zero[8]` : 8 bytes of padding. **Critical Note:** This field is technically "Not Used," but you *must* set this value (and ideally the entire structure) to `NULL` (or `\0`) using functions like `memset()` or `bzero()` before passing it into a socket function. Failing to wipe the structure can result in unexpected junk values polluting the network stack.

3. Reference Passing & Value-Result Arguments Socket address structures are an integral part of every network program. A crucial C-programming paradigm here is that these structures are *always passed by reference* (a pointer to the structure is passed, not the structure itself).

Furthermore, you must always pass the *size* of the structure as a secondary argument. When a socket function (like `accept()` or `recvfrom()`) fills in a structure with incoming remote data, the length parameter is also passed by reference (as a pointer, e.g., `&fromlen`). This allows the kernel to explicitly update the variable to reflect the true, modified size of the data it just wrote into your structure. Arguments that behave this way are known fundamentally as **value-result arguments**.